

UNIVERSIDADE ESTADUAL PAULISTA "JÚLIO DE MESQUITA FILHO"
FACULDADE DE CIÊNCIAS - CAMPUS BAURU
DEPARTAMENTO DE COMPUTAÇÃO
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

HENRIQUE TRIVELATO DE ANGELO

ADAPTAÇÃO DO ENVENENAMENTO NIGHTSHADE PARA
ARQUIVOS DE ÁUDIO

BAURU
Novembro/2025

HENRIQUE TRIVELATO DE ANGELO

**ADAPTAÇÃO DO ENVENENAMENTO NIGHTSHADE PARA
ARQUIVOS DE ÁUDIO**

Trabalho de Conclusão de Curso do Curso
de Ciência da Computação da Universidade
Estadual Paulista “Júlio de Mesquita Filho”,
Faculdade de Ciências, Campus Bauru.
Orientador: Prof. Dr. Kelton Augusto
Pontara da Costa

BAURU
Novembro/2025

A584a Angelo, Henrique
Adaptação do envenenamento Nightshade para arquivos de áudio /
Henrique Angelo. -- Bauru, 2025
110 p. : il., tabs.

Trabalho de conclusão de curso (Bacharelado - Ciência da
Computação) - Universidade Estadual Paulista (UNESP), Faculdade
de Ciências, Bauru
Orientador: Kelton Augusto Pontara da Costa

1. áudio. 2. Nightshade. 3. texto-para-fala. 4. espectrograma de
log-mel. 5. Griffin-Lim. I. Título.

Henrique Trivelato de Angelo

ADAPTAÇÃO DO ENVENENAMENTO NIGHTSHADE PARA ARQUIVOS DE ÁUDIO

Trabalho de Conclusão de Curso do Curso
de Ciência da Computação da Universidade
Estadual Paulista "Júlio de Mesquita Filho",
Faculdade de Ciências, Campus Bauru.

Banca Examinadora

Prof. Dr. Kelton Augusto Pontara da Costa

Orientador

Universidade Estadual Paulista "Júlio de Mesquita Filho"

Profa. Dra. Simone das Graças Domingues Prado

Universidade Estadual Paulista "Júlio de Mesquita Filho"

Prof. Dr. Leandro Aparecido Passos Junior

Universidade Estadual Paulista "Júlio de Mesquita Filho"

Bauru, — de Novembro de 2025.

À minha família, pelo apoio incondicional nesta jornada.

Agradecimentos

Aos meus pais, Angélica e Wenceslau, por todo o apoio incondicional, amor e incentivo ao longo de toda a minha vida, e em especial nesta reta final da graduação. Sem o suporte de vocês, eu não teria chegado até aqui.

Aos amigos feitos ao longo desses 4 anos, agradeço por me acompanharem dentro da faculdade nos momentos de estudos para provas e escrita de trabalhos assim como fora dela, nos momentos de diversão e de tristeza. Essa companhia foi fundamental para tornar esta caminhada muito mais leve.

Ao corpo docente dos departamentos de Computação e Matemática, minha gratidão pela excelência no ensino e pela dedicação que moldaram minha trajetória acadêmica.

Agradeço imensamente aos professores Dra. Simone das Graças Domingues Prado e Dr. Leandro Aparecido Passos Junior pela gentileza e prontidão em aceitar o convite para compor a banca e pela leitura atenta desta monografia.

Por fim, sou grato ao grupo de estudos de *Deepfake* da Unesp Bauru por me acolher e viabilizar essa pesquisa. Agradeço à Inaê pela valiosa ajuda na definição do tema desta pesquisa; ao Matheus pela elaboração e hospedagem do questionário online; e, principalmente, ao professor Dr. Kelton Augusto Pontara da Costa pela orientação, paciência, disponibilidade e contribuições críticas para o trabalho.

"I start chasing all the rabbits I see and end up catching none of them."
- Suruga Kanbaru.

Resumo

O *Nightshade* é uma técnica de envenenamento direcionado que busca criar uma proteção invisível para imagens antes de serem publicadas na internet. Para isso, adiciona-se, em certos pixels, ruídos imperceptíveis capazes de corromper modelos de difusão de texto para imagem (em inglês, *text-to-image*, TTI) na fase de treinamento. Este trabalho propõe uma adaptação do envenenamento *Nightshade* para expandir sua atuação no campo dos áudios. É proposta a substituição do modelo *Contrastive Language-Image Pre-Training* (CLIP) pelo modelo *Contrastive Language-Audio Pre-Training* (CLAP) para a criação aleatória de uma amostra de treinamento de áudios envenenados; a troca do modelo de difusão TTI *Stable Diffusion* pelo StyleTTS 2, um modelo de difusão de texto para fala (em inglês, *text-to-speech*, TTS), com a finalidade de gerar áudios âncoras; a adição de um passo a passo para que o envenenamento direcionado ocorra no espectrograma de log-mel de um áudio; e, por fim, a incorporação do método de Griffin-Lim para retornar os espectrogramas envenenados à sua forma de onda correspondente. Utiliza-se o *Speech Commands* para a elaboração de ataques direcionados no StyleTTS 2. Os resultados obtidos pela aplicação de um questionário online mostram que a adaptação não é capaz de efetuar um ataque direcionado e furtivo. Pelo contrário, o *fine-tuning* do StyleTTS 2 com amostras de treinamento com 5 minutos de áudios envenenados torna o modelo capaz apenas de produzir áudios incompreensíveis ao custo de ser facilmente identificado por avaliação humana, visto que o algoritmo de Griffin-Lim exacerba as perturbações adversariais já inseridas no espectrograma.

Palavras-chave: Áudio; Envenenamento; Texto para fala; *Nightshade*; Espectrograma de Log-Mel; CLAP; Griffin-Lim.

Abstract

Nightshade is a targeted poisoning technique that seeks to create invisible protection for images before they are published on the internet. To achieve this, imperceptible noise is added to specific pixels, capable of corrupting text-to-image (TTI) diffusion models during the training phase. This work proposes an adaptation of the Nightshade poisoning attack to extend its application to audio. The Contrastive Language-Image Pre-Training (CLIP) model is replaced by a Contrastive Language-Audio Pre-Training (CLAP) one to create randomized poisoned-audio training samples, and substitute the image diffusion model Stable Diffusion with StyleTTS 2 — a diffusion-based text-to-speech (TTS) model — to generate anchor audios. This work proposes a step-by-step procedure for performing targeted poisoning on an audio's log-mel spectrogram and adopt the Griffin–Lim algorithm to reconstruct poisoned spectrograms back into poisoned waveforms. Speech Commands is used to craft targeted attacks within StyleTTS 2. Finally, results from an online questionnaire indicate that the proposed adaptation fails to produce a stealthy, targeted attack: fine-tuning StyleTTS 2 with five minutes of poisoned training audio only causes the model to produce unintelligible audio, which is readily detected by human evaluators. In addition, the Griffin–Lim reconstruction further amplifies adversarial spectrogram perturbations, increasing perceptual detectability of the poisoning.

Key-words: Audio; Poison; Text-to-Speech; Nightshade; Log-Mel Spectrogram; CLAP; Griffin-Lim.

Lista de figuras

Figura 1 – Processo de difusão ao longo de T etapas	22
Figura 2 – Processo de difusão reversa ao longo de T etapas	23
Figura 3 – Exemplo de imagens envenenadas e suas respectivas imagens âncoras	28
Figura 4 – Exemplo de sangramento no ataque “dog” para “cat”	28
Figura 5 – Envenenamento a partir de estilo artístico	29
Figura 6 – Envenenamento composto de “dog” e “fantasy art”	30
Figura 7 – Degradação total do modelo	31
Figura 8 – Onda azul sofre <i>aliasing</i> , resultando na mediação da onda vermelha	33
Figura 9 – Exemplo de amostragem de áudio com profundidade de bit fixo . . .	34
Figura 10 – Exemplo de quantização de áudio com taxa de amostragem fixa . .	35
Figura 11 – Visualização do processo de STFT	38
Figura 12 – Relação logarítmica entre frequências em <i>hertz</i> e <i>mels</i>	39
Figura 13 – Bancos de filtro de mel	41
Figura 14 – Espectograma inicial $\hat{S}_{mag}^{(0)}(n_{frame}, k)$	44
Figura 15 – Exemplo de execução do algoritmo de Griffin-Lim	44
Figura 16 – Calculando proximidade de um áudio em relação a uma lista de textos	45
Figura 17 – Calculando proximidade de um áudio em relação a uma lista de textos	47
Figura 18 – Estimativa da FMP do alinhamento a_i com 5 fonemas e $L = 5$	52
Figura 19 – Arquitetura simplificada de treinamento do StyleTTS 2	53
Figura 20 – <i>Pipeline</i> de redes DNN-HMM	57
Figura 21 – <i>Pipeline</i> do ataque <i>WaveFuzz</i>	58
Figura 22 – Função para armazenar todos os caminhos dos áudios referentes a PE	63
Figura 23 – Função para gerar conceito âncora PA	64
Figura 24 – Função que repete o áudio 4 vezes a para que ele tenha 4 segundos	65
Figura 25 – Convertendo formato de onda para log espectograma mel	65
Figura 26 – Exemplo de áudio “four” do <i>Command Speechs</i>	66
Figura 27 – Dimensão e valores armazenados na matriz do log-mel espectograma	66
Figura 28 – Acoplando o espectograma de log-mel de um áudio “bed” na imagem M_{512}	67
Figura 29 – Função que adiona o espectograma de log-mel a uma imagem 512 x 512 em branco	68
Figura 30 – Código que converte imagem em escala cinza para o formato de <i>input</i> do <i>Stable Diffusion</i>	70
Figura 31 – Código referente à fase de otimização de ruídos	72
Figura 32 – Código referente à conversão do tensor Te_{adv} para a imagem $M_{512(adv)}$	72

Figura 33 – Código referente à conversão da imagem $M_{512(adv)}$ em seu respectivo áudio envenenado	73
Figura 34 – Diferenças entre o <i>Nightshade</i> original e o adaptado	74
Figura 35 – Arquivo <i>train_list.txt</i> para o ataque (<i>no</i> , <i>yes</i>)	75
Figura 36 – Código para converter texto em fonema	76
Figura 37 – Email de divulgação do questionário encaminhado pela Acessoria da FC	79
Figura 38 – Interface do questionário utilizado na avaliação humana	81
Figura 39 – Problema de <i>padding</i> no envenenamento de “ <i>go</i> ” para “ <i>yes</i> ”	84
Figura 40 – Perda média obtida durante a fase de otimização de ruídos no ataque (<i>bed</i> , <i>cat</i>)	87
Figura 41 – Ondas originais e as envenenadas de três candidatos do áudio “ <i>bed</i> ”	88
Figura 42 – Comparando onda do áudio “ <i>four</i> ” antes e depois do envenenamento com âncoras “ <i>five</i> ” e “ <i>happy</i> ”	89
Figura 43 – Perda do gerador referente ao StyleTTS 2 envenenado com $N_p = 100$ áudios	91
Figura 44 – Perda do gerador referente ao StyleTTS 2 envenenado com $N_p = 300$ áudios	91

Lista de quadros

Quadro 1 – Informações sobre algumas das profundidades de bits mais conhecidas	33
Quadro 2 – Informações sobre os <i>datasets</i> de treinamento	54

Lista de abreviaturas e siglas

AAC	<i>Advanced Audio Coding</i>
ADC	<i>Analogic Digital Conversor</i>
AIFF	<i>Audio Interchange File Format</i>
ALAC	<i>Apple Lossless Audio Codec</i>
API	<i>Application Programming Interface</i>
ASR	<i>Automatic Speech Recognition</i>
CLAP	<i>Contrastive Language-Audio Pre-Training</i>
CLIP	<i>Contrastive Language-Image Pre-Training</i>
CNN	<i>Convolutional Neural Network</i>
CVAE	<i>Conditional Variational Autoencoder</i>
DDPM	<i>Denoising Diffusion Probabilistic Model</i>
DFT	<i>Discrete Fourier Transform</i>
DNN	<i>Deep Neural Networks</i>
DPM-2	<i>Denoising Diffusion Probabilistic Model with second order solver</i>
EDM	<i>Element Differential Method</i>
FFT	<i>Fast Fourier Transform</i>
FLAC	<i>Free Lossless Audio Codec</i>
FMP	<i>Função Massa de Probabilidade</i>
GAN	<i>Generative Adversarial Network</i>
GPU	<i>Graphics Processing Units</i>
HMM	<i>Hidden Markov Models</i>
IA	Inteligência Artificial
IAGen	Inteligência Artificial Generativa

MFCC	<i>Mel Frequency Cepstral Coefficients</i>
MOS	<i>Mean Opinion Score</i>
MP3	<i>MPEG Audio Layer III</i>
ODE	<i>Ordinary Differential Equation</i>
PCM	<i>Pulse-Code Modulation</i>
PNG	<i>Portable Network Graphics</i>
RGB	<i>Red Green Blue</i>
SDE	<i>Stochastic Differential Equation</i>
SNR	<i>Signal-to-Noise Ratio</i>
SLM	<i>Speech Language Models</i>
STFT	<i>Short-Time Fourier Transform</i>
TTI	<i>Text-to-Image</i>
TTS	<i>Text-to-Speech</i>
WAV	<i>Waveform Audio File Format</i>

Sumário

1	INTRODUÇÃO	17
1.1	Problemática	18
1.2	Justificativa	19
1.3	Objetivos	19
1.3.1	Objetivo Geral	19
1.3.2	Objetivos Específicos	20
1.4	Organização da Monografia	20
2	FUNDAMENTAÇÃO TEÓRICA	21
2.1	Modelos de difusão	21
2.1.1	Formalização matemática	22
2.1.1.1	Processo de difusão	22
2.1.1.2	Processo de difusão reverso	23
2.2	Nightshade	25
2.2.1	Viabilidade de realizar envenenamento de modelos de difusão	25
2.2.2	Quem pode utilizar o <i>Nightshade</i> ?	25
2.2.3	O algoritmo de envenenamento do Nightshade	26
2.2.4	O efeito sangramento	28
2.2.5	Composição de múltiplos envenenamentos	30
2.3	Áudio	31
2.3.1	Amostragem	32
2.3.2	Quantização	33
2.3.3	Tipos de arquivo de áudio	35
2.3.4	Espectograma de log-mel	36
2.3.5	Retornando espectograma à forma de onda	41
2.3.5.1	Algoritmo de Griffin-Lim	42
2.4	CLAP	45
2.4.1	Algoritmo de treinamento dos codificadores de áudio e texto	46
2.5	StyleTTS2	47
2.5.1	O problema de expressividade e o conceito de “estilo”	48
2.5.2	Arquitetura <i>end-to-end</i> e a duração do áudio sintetizado	50
2.5.3	Treinamento adversarial com SLMs	52
2.5.4	Desempenho do modelo após o treinamento	54
2.6	Trabalhos Correlatos	56
2.6.1	Envenenamento de áudio	56

2.6.1.1	<i>VenoMave</i>	56
2.6.1.2	<i>WaveFuzz</i>	58
2.6.2	Áudios como imagens	59
3	METODOLOGIA	61
3.1	<i>Dataset</i>	61
3.2	Bibliotecas do Python	61
3.3	O algoritmo do <i>Nightshade</i> adaptado	62
3.3.1	Etapa 3	62
3.3.2	Etapa 5	63
3.3.3	Etapa 6	65
3.3.4	Etapa 7	74
3.4	Avaliando a eficácia do envenenamento	76
3.4.1	Pontuação CLAP	77
3.4.2	Valor MOS acerca da qualidade do áudio	78
3.4.3	Verificação da troca de palavras	78
3.4.4	Questionário online e coleta de informações	79
3.4.4.1	Interface do questionário	80
4	EXPERIMENTOS	82
4.1	Ambiente Experimental	82
4.2	Ataques realizados	82
4.2.1	Ataques não viáveis	83
4.2.1.1	Número insuficiente de candidatos	83
4.2.1.2	Impossibilidade de gerar áudio âncora	84
4.2.2	Ataques viáveis	85
4.3	Funções de perda durante o <i>fine-tuning</i>	90
4.4	Avaliação do StyleTTS 2 envenenado	92
4.4.1	Ataque (PE, PA) = (<i>bed, cat</i>)	94
4.4.2	Ataque (PE, PA) = (<i>down, right</i>)	94
4.4.3	Ataque (PE, PA) = (<i>four, five</i>)	95
4.4.4	Ataque (PE, PA) = (<i>four, happy</i>)	96
4.4.5	Ataque (PE, PA) = (<i>nine, five</i>)	97
4.4.6	Ataque (PE, PA) = (<i>nine, tiny</i>)	97
4.4.7	Ataque (PE, PA) = (<i>no, yes</i>)	98
4.4.8	Ataque (PE, PA) = (<i>stop, go</i>)	99
4.4.9	Ataque (PE, PA) = (<i>tree, destroy</i>)	99
4.5	Considerações Finais	100
5	CONCLUSÕES	102

5.1	Trabalhos Futuros	102
	REFERÊNCIAS	104

1 Introdução

Nos últimos anos, os estudos de Inteligência Artificial (IA) foram capazes de garantir aos computadores a habilidade de gerar textos, imagens, áudios e vídeos de alta resolução e bastante fiéis à realidade. Essa especialização é denominada de Inteligência Artificial Generativa (IAGen). Alguns dos exemplos muito conhecidos e utilizados no cotidiano são o ChatGPT ¹, Meta AI ² e o DeepSeek ³.

As aplicações da IAGen englobam diversas áreas da ciência: na Educação, o desenvolvimento de ferramentas para aprendizagem de línguas estrangeiras - por exemplo, o AI KAKU ajuda os alunos japoneses a melhorarem sua compreensão e produção de textos em inglês (GAYED et al., 2022) - na Farmácia, o desenvolvimento de novos remédios a partir de combinações moleculares que nunca foram testadas em laboratório (GANGWAL et al., 2024), no Áudio-Visual, a capacidade de atribuir vozes em diferentes línguas aos personagens fictícios sem necessidade de um artista de voz (ZHANG et al., 2024b).

Por outro lado, é possível modificar o conteúdo original de textos, imagens, áudios e vídeos com o auxílio das IAGens. Para se produzir uma *deepfake*, basta que haja troca de rostos, de falas ou de vozes com o auxílio de uma IAGen capaz de produzir textos, imagens, áudios ou vídeos falsos tão realistas quanto as mídias originais.

As primeiras *deepfakes* foram publicadas em Setembro de 2017 na plataforma *Reddit* pelo usuário "deepfake". Elas formavam uma coleção de vídeos pornográficos em que os rostos originais das mulheres foram trocadas pelos rostos de atrizes famosas (NGUYEN et al., 2022).

Atualmente, além de facilitar a difamação de quaisquer mulheres (sejam elas menores ou maiores de idade) por meio de materiais pornográficos artificiais (SANTOS, 2024), as *deepfakes* são capazes de alterar o rumo das disputas eleitorais - por exemplo, no Brasil, a partir da circulação massiva de áudios falsos no WhatsApp (QUEIROZ, 2024) - e aumentar a suscetibilidade de pessoas realizarem grandes transferências monetárias para falsos chefes (CHEN; MAGRAMO, 2024) ou falsos familiares (MACHADO, 2024).

Diante da popularização das *deepfakes*, muitos estudos emergiram de forma a contrapor o uso irresponsável das IAGens. Há aqueles que focam na elaboração

¹ Disponível em <https://openai.com/index/chatgpt/>

² Disponível em <https://www.meta.ai/> ou na própria rede social WhatsApp

³ Disponível em <https://www.deepseek.com/en>

de softwares com a capacidade de identificar se uma mídia está adulterada (REISS; CAVIA; HOSHEN, 2023). Outra frente opta por desenvolver técnicas de envenenamento de dados, que buscam deteriorar a qualidade de criação do modelo de IAGen ainda na fase de treinamento.

No domínio de imagens, o *Nightshade* (SHAN et al., 2024) é a principal técnica de envenenamento direcionado. Ele introduz ruídos imperceptíveis em imagens para corromper modelos de difusão de texto para imagem (em inglês, *text-to-image*, TTI) durante o treinamento, fazendo-os aprender associações incorretas (por exemplo, que a imagem de um cachorro corresponde ao texto gato). O objetivo é proteger a propriedade intelectual de artistas compartilhadas na internet contra o uso não autorizado de suas obras para o treinamento contínuo de IAGens.

Seguindo uma lógica similar no domínio de áudio, já existem propostas como o *VenoMove* (AGHAKHANI et al., 2023), que foca em perturbações adversárias no espectrograma, e o *WaveFuzz* (GE et al., 2022), que aplica ruído diretamente na forma de onda, para induzir falhas em modelos de reconhecimento automático de fala (em inglês, *Automatic Speech Recognition*, ASR).

1.1 Problemática

O aumento das *deepfakes* está atrelado a dois fatores: o aumento de IAGen robustas disponíveis de forma online e gratuita e a falta de proteção dos dados contra o *web scraping* (em português, raspagem da web ⁴).

Como a performance das IAGen depende diretamente da quantidade de informações disponíveis no período de treinamento ou de *fine-tuning* (em português, ajuste fino ⁵), a partir do uso ferramentas que realizam o *web scraping* de forma autônoma, é possível coletar uma imensa quantidade de dados a qualquer momento e, consequentemente, manter a alta qualidade de geração da IA com o processo de *fine-tuning*. Todos os textos, as imagens, os áudios e os vídeos publicados na internet estão sujeitos ao *web scraping*, portanto qualquer IAGen pautada na produção de *deepfake* estará apta para mudar a voz, trocar a face e alterar a fala de uma pessoa pela voz, face e fala de qualquer outra pessoa.

Comprometer o *web scraping* é uma forma de reduzir a falsa autenticidade das *deepfakes*. O envenenamento de dados é uma técnica que busca fazer, nos dados, modificações imperceptíveis capazes de lhe garantir a possibilidade de reduzir o poder de criação realista das IAGen. Quanto maior for a quantidade de dados

⁴ Tradução livre realizada pelo próprio autor

⁵ Tradução livre realizada pelo próprio autor

envenenados disponíveis na internet, pior será a qualidade das criações da IAGen que foram abastecidas pelo *web scraping*.

O problema é que atualmente não existe uma ferramenta versátil, eficiente e gratuita para realizar o envenenamento de quaisquer tipos de arquivo (imagem, texto ou vídeos) dos mais diversos tamanhos.

Para isso, este trabalho se propõe a expandir o campo de atuação do *Nightshade* para o envenenamento de áudios.

1.2 Justificativa

A escolha do *Nightshade* como base para esta adaptação é estratégica e se fundamenta em três pilares principais.

Primeiro, o *Nightshade* é considerado o estado da arte no envenenamento de modelos de difusão TTI. Sua metodologia de ataque provou ser eficaz em um domínio (imagens) que, assim como o áudio, sofre com a extração massiva de dados para treinamento de IAGens. Segundo, a ferramenta obteve grande repercussão e adoção pela comunidade artística e tecnológica, sendo amplamente discutida em plataformas como o *Reddit* ⁶. Isso indica um alto potencial de aceitação e uso de uma ferramenta derivada para a proteção de áudio.

Terceiro, e mais importante, a adaptação de um envenenamento robusto, porém restrito a imagens, para que ele também possa atuar em arquivos de áudio, é um estudo inédito. Conforme detalhado anteriormente neste trabalho, a clonagem de voz e as *deepfakes* de áudio representam uma ameaça crescente.

Portanto, a justificativa deste projeto reside na necessidade urgente de criar mecanismos de defesa para o conteúdo de áudio, aproveitando uma arquitetura de ataque - no caso, *Nightshade* - já validada e reconhecida, visando proteger criadores contra o *scraping* de dados e a clonagem de voz não autorizada.

1.3 Objetivos

1.3.1 Objetivo Geral

O objetivo principal deste projeto é propor uma técnica de envenenamento direcionado, adaptada do algoritmo *Nightshade*, capaz de inserir perturbações adversariais imperceptíveis em segmentos específicos de um sinal de áudio, visando corromper o

⁶ Disponível em: https://www.reddit.com/r/technology/comments/19bp712/nightshade_the_free_tool_that_poisons_ai_models/. Acesso em: 14 nov. 2025

treinamento de modelos de difusão de texto para fala (em inglês, *text-to-speech*, TTS), assim, proteger o conteúdo contra *scraping* de dados e clonagem de voz.

1.3.2 Objetivos Específicos

- Realizar um envenenamento em áudio como se fosse um problema de imagem;
- Efetuar mudanças mínimas na composição do ataque *Nightshade*;
- Determinar um modelo TTS de difusão que será tanto o alvo do envenenamento direcionado quanto um componente integrado na arquitetura de ataque de áudio proposta;
- Verificar se a adaptação proposta para o *Nightshade* é de fato um envenenamento direcionado que irá afetar o desempenho do modelo TTS de difusão escolhido, impedindo que ele reproduza certas palavras.

1.4 Organização da Monografia

O presente trabalho está estruturado da seguinte forma: no Capítulo 2 tem-se a fundamentação teórica, onde são explicados, em detalhes, os conceitos mais importantes para que esse trabalho pudesse se concretizar, como a ideia central de geração de dados usada em modelos de difusão; a apresentação do *Nightshade*; as conversões de áudio para imagem e vice-versa; além do funcionamento dos modelos *Contrastive Language-Audio Pre-Training* (CLAP) e StyleTTS 2 a serem utilizados na composição da adaptação proposta e os trabalhos correlatos encontrados na literatura. Já o Capítulo 3 apresenta o *dataset* utilizado para a realização do envenenamento, a metodologia empregada para a adaptação e implementação do algoritmo do *Nightshade* e as métricas utilizadas para a avaliação método de adaptação. O Capítulo 4, por sua vez, expõe os resultados obtidos nos experimentos, analisando tanto a imperceptibilidade das alterações quanto a eficácia do envenenamento. Finalmente, o Capítulo 5, apresenta as conclusões do trabalho, bem como sugestões para trabalhos futuros.

2 Fundamentação Teórica

Neste capítulo são apresentados os conceitos considerados mais importantes para este trabalho, tais como: a ideia central de geração de modelos de difusão, a apresentação do envenenamento *Nightshade*, as conversões de áudio para imagem e vice-versa, além do funcionamento dos modelos CLAP e StyleTTS 2 a serem utilizados na composição da adaptação proposta e, finalmente, os trabalhos correlatos encontrados na literatura.

2.1 Modelos de difusão

Modelos de difusão são uma classe de modelos generativos que têm ganhado destaque nos últimos anos devido à sua capacidade de gerar dados sintéticos de alta qualidade, especialmente em tarefas como geração de imagens, áudio e texto (ZHANG et al., 2023). Inspirados em processos físicos de difusão, esses modelos funcionam a partir da ideia de adicionar ruído progressivamente a um dado, como uma imagem ou um sinal de áudio, até que ele se torne puro ruído. Em seguida, um modelo é treinado para reverter esse processo e reconstruir os dados originais. Aquele que adiciona o ruído progressivamente é denominado de processo de difusão enquanto o segundo, que se pauta na retirada de ruído, processo de difusão reverso.

A formalização matemática desse processo foi inicialmente proposta por Sohl-Dickstein et al. (2015) e posteriormente refinada em trabalhos como o Modelo Probabilístico de Difusão com Remoção de Ruído (em inglês, *Denoising Diffusion Probabilistic Model*, DDPM) de Ho, Jain e Abbeel (2020). Neste último, os autores introduzem um modelo baseado em redes neurais convolucionais que prevê o ruído adicionado em cada etapa, permitindo reconstruir os dados iniciais a partir do ruído puro de forma eficiente e com alta fidelidade (subseção 2.1.1).

Sua capacidade de modelar distribuições complexas e gerar sinais altamente realistas a partir de ruído gaussiano os torna especialmente adequados para tarefas sensíveis à fidelidade perceptual. Com a possibilidade de incorporar condicionamentos textuais, espectrais ou semânticos, os modelos de difusão permitem a criação de sistemas generativos altamente controláveis e expressivos. Isso tem impulsionado inovações em áreas como acessibilidade, por meio da leitura automatizada com voz natural, gerada por meio dos modelos TTS, de acordo com (LI et al., 2023; SHEN et al., 2023); entretenimento, com geração de efeitos de fundo personalizados (KREUK et al., 2023); produção musical assistida por IA (ZHANG et al., 2024a); e até mesmo recuperação forense de áudio degradado (ZHANG; JAYASURIYA; BERISHA, 2021).

Diante disso, os modelos de difusão configuram-se como uma das abordagens mais promissoras para a próxima geração de tecnologias de processamento e criação de áudio (ZHANG et al., 2023).

2.1.1 Formalização matemática

A seguir, serão apresentados os conceitos de processo de difusão e processo de difusão reverso definidos no trabalho de Ho, Jain e Abbeel (2020).

2.1.1.1 Processo de difusão

O processo de difusão (equação 2.1) é um processo estocástico progressivo que transforma uma amostra real x_0 (uma imagem ou um áudio) em uma amostra de ruído puro x_T , por meio de uma sequência de T etapas.

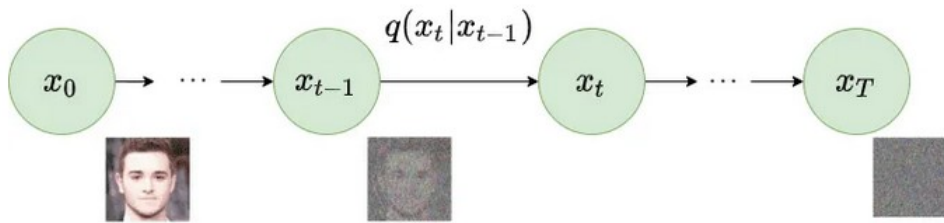
$$q(x_1, \dots, x_T | x_0) = \prod_{t=1}^T q(x_t | x_{t-1}) , \quad (2.1)$$

em que $q(\cdot)$ representa a distribuição verdadeira dos dados. A equação 2.2 define os ruídos adicionados em cada interação do processo de difusão

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; (\sqrt{1 - \beta_t})x_{t-1}, \beta_t \mathbb{I}) , \quad (2.2)$$

tal que $\mathbb{I}$ é uma matriz identidade e $\beta_t \in [0, 1]$ é o parâmetro denominado de agendamento de ruído, responsável por informar a quantidade de informação do dado x_{t-1} a qual irá se tornar ruído na etapa t .

Figura 1 – Processo de difusão ao longo de T etapas



Fonte: Ho, Jain e Abbeel (2020).

Conforme as etapas vão avançando (figura 1), maior será o valor de β_t . Segundo (GUO et al., 2025), há diversas formas de se modelar β_t . A mais conhecida é a modelagem linear de β_t de Ho, Jain e Abbeel (2020), definida na equação 2.3

$$\beta_t = \beta_1 + \frac{(t-1)}{T-1}(\beta_T - \beta_1) , \quad (2.3)$$

de forma que o incremento de β_t , isto é, $\frac{(t-1)}{T-1}(\beta_T - \beta_1)$ seja uniforme para todas as etapas.

Ao supor que $\alpha_t = 1 - \beta_t$ e $\tilde{\alpha}_t = \prod_{s=1}^t \alpha_s$, é possível reescrever a equação 2.2 como a equação 2.4

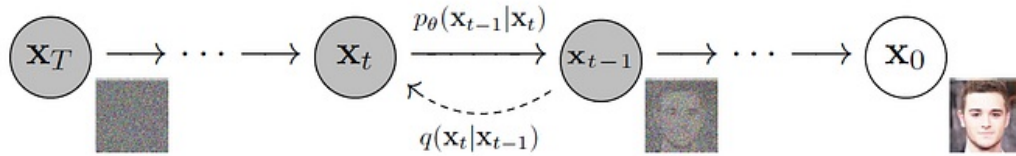
$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\tilde{\alpha}_t}x_0, (\sqrt{1 - \tilde{\alpha}_t})\mathbb{I}), \quad (2.4)$$

a qual representa a relação direta entre o dado original x_0 e o ruído a ser calculado na etapa t .

2.1.1.2 Processo de difusão reverso

Ao contrário do processo de difusão, o processo reverso (figura 2) tem como objetivo retirar os ruídos até conseguir reconstruir, o mais próximo possível, da amostra original x_0 . Para isso, é necessário treinar um modelo de rede neural com parâmetros θ a partir de uma amostra completamente ruidosa x_T , a qual segue uma distribuição normal padrão, $x_T \sim \mathcal{N}(0, 1)$, para estimar a distribuição p_θ referente à retirada de ruído.

Figura 2 – Processo de difusão reversa ao longo de T etapas



Fonte: Ho, Jain e Abbeel (2020).

Dado que μ_θ e Σ_θ são respectivamente a média e variância a serem aprendidas pela rede neural, defini-se, na equação 2.5, a distribuição gaussiana que define o passo reverso

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \quad (2.5)$$

A aprendizagem do modelo está sujeita a um problema de otimização de parâmetros θ em que função de perda $\mathcal{L}$ é calculada a partir de um valor esperado condicionado ao processo de difusão q ($\mathbb{E}_q$) tal que:

$$\mathcal{L} = \mathbb{E}_q \left[\log \left(\frac{p_\theta(x_0 : T)}{q(x_1 : T|x_0)} \right) \right] = \mathbb{E}_q \left[-\log p(x_T) - \sum_{t \geq 1} \log \left(\frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})} \right) \right] \quad (2.6)$$

Os parâmetros ótimos seriam aqueles capazes de aproximar a distribuição p_θ da distribuição q , isto é, $p_\theta(x_{t-1}|x_t) \approx q(x_t|x_{t-1})$, na equação 2.6. Todavia, isso é uma tarefa complexa (GUO et al., 2025).

Diante disso, Ho, Jain e Abbeel (2020) propuseram outra abordagem em que o modelo de rede neural $\epsilon_\theta(x_t, t)$ estimasse o ruído ϵ adicionado na etapa t (equação 2.7). Assim, para se encontrar uma amostra $x_{t-1} \sim p_\theta(x_{t-1}|x_t)$ - etapa 5 do algoritmo 2 - basta apenas calcular a seguinte relação

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \tilde{\alpha}_t}} \hat{\epsilon}_\theta(x_t, t) \right) + \sigma_t z, \quad (2.7)$$

em que $\sigma_t = \frac{1 - \tilde{\alpha}_{t-1}}{1 - \tilde{\alpha}_t}$, o modelo de rede neural $\epsilon_\theta(x_t, t)$ tem seus parâmetros otimizados a partir da função objetivo (etapa 5 do algoritmo 1) definida na equação 2.8 e $z \sim \mathcal{N}(0, \mathbb{I})$ se $t > 1$, $z = 0$ se caso contrário.

$$\mathcal{L}_{simple}(\theta) = \max_{\theta} \mathbb{E}_{t, x_0, \epsilon} \left[\|\epsilon - \epsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1 - \tilde{\alpha}_t}\epsilon, t)\|^2 \right] \quad (2.8)$$

O algoritmo 1 define o processo de estimação dos parâmetros θ do modelo de rede neural $\epsilon_\theta(x_t, t)$, o qual contém a informação da quantidade de ruído ϵ a ser acrescida na etapa t . Já, o algoritmo 2 define o processo de gerar uma imagem a partir de um ruído inicial $x_T \sim \mathcal{N}(0, \mathbb{I})$, sorteado aleatoriamente.

Algoritmo 1

- 1: Repita
 - 2: $x_0 \sim q(x_0)$
 - 3: $t \sim \text{Uniforme}(\{1, \dots, T\})$
 - 4: $\epsilon \sim \mathcal{N}(0, \mathbb{I})$
 - 5: Usar a passada do gradiente descendente para otimizar
 $\nabla_{\theta} \|\epsilon - \epsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1 - \tilde{\alpha}_t}\epsilon, t)\|^2$
 - 6: Até convergir
-

Fonte: Adaptado de Ho, Jain e Abbeel (2020).

Algoritmo 2

- 1: $x_T \sim \mathcal{N}(0, \mathbb{I})$
 - 2: Para $t = T, \dots, 1$ do
 - 3: Se $t > 1$
 $z \sim \mathcal{N}(0, \mathbb{I})$
 - 4: Caso contrário
 $z = 0$
 - 5: Retirando ruídos $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left[x_t - \frac{1 - \alpha_t}{\sqrt{1 - \tilde{\alpha}_t}} \epsilon_\theta(x_t, t) \right] + \sigma_t z$
 - 6: Fim para
 - 7: Retorne x_0
-

Fonte: Adaptado de Ho, Jain e Abbeel (2020).

2.2 Nightshade

O *Nightshade* é uma técnica de envenenamento direcionado do tipo caixa preta¹ de modelos de IAGen que visa auxiliar artistas e pequenas empresas na proteção de suas propriedades intelectuais contra as empresas de IA que treinam seus modelos a partir do *web scraping*. Seu principal alvo são modelos que se pautam na abordagem de difusão para criação de imagens.

2.2.1 Viabilidade de realizar envenenamento de modelos de difusão

Os modelos de difusão são treinados a partir de centenas de milhões ou, até mesmo, bilhões de imagens, logo é comum assumir que grandes quantidades de amostras envenenadas serão necessárias para realizar um envenenamento com sucesso, isto é, seriam necessários, no mínimo, dezenas de milhões de imagens para realizar um envenenamento em apenas 1% dos dados desses modelos (SHAN et al., 2024).

Uma das conclusões obtida pelos estudos dos autores do *Nightshade* mostra que esse pensamento não é válido. Apesar de o treinamento desses modelos utilizar uma amostra de treinamento gigantesca, ao avaliá-la minuciosamente, os autores perceberam que eles estão destilados em diversos conceitos. Para isso, eles estudaram um dos conjuntos de dados mais utilizados no treinamento de modelos de difusão TTI: o *LAION-Aesthetic* (SCHUHMANN et al., 2022).

A partir dele, foi observado que mais de 92% dos conceitos estão associados, cada um, a menos de 0,04% das imagens, ou seja, a grande maioria dos conceitos só abrange 240 mil imagens dentre o total de 600 milhões. Outro resultado importante é que menos de 0,2% de todo o conjunto de dados engloba semanticamente mais de 92% dos conceitos existentes. A junção de ambos os resultados fortalece a ideia de que é possível envenenar um conceito $\mathcal{C}$ e outros conceitos semanticamente parecidos com $\mathcal{C}$ de um modelo de difusão TTI apenas envenenando uma pequena porcentagem dos dados de treinamento referentes ao conceito $\mathcal{C}$.

2.2.2 Quem pode utilizar o *Nightshade*?

O indivíduo que optar pelo uso do envenenamento *Nightshade* deve atender a quatro suposições:

1. Ser capaz de encaixar pequenas quantidades de dados envenenados na amostra de treinamento de um modelo de difusão TTI;

¹ Não se tem conhecimento sobre o modelo de IAGen a ser atacado

2. Modificar arbitrariamente os conteúdos das imagens e textos de todos os dados envenenados;
3. Não ter acesso a qualquer parte da *pipeline* do modelo que se deseja atacar (códigos de treinamento, *deployment*, etc.)²;
4. Ter acesso a modelos TTI de código aberto.

Como os modelos genéricos de difusão são constantemente treinados e atualizados a partir de dados coletados da internet, logo qualquer pessoa que tenha a capacidade de armazenar ou compartilhar uma imagem na internet pode atender a todas as suposições. Diante disso, o *Nightshade* torna-se um ataque viável para qualquer usuário da Internet.

2.2.3 O algoritmo de envenenamento do Nightshade

A seguir, descrever-se-á o algoritmo do *Nightshade* para envenenar qualquer modelo TTI de código aberto, M_{TTI} .

- **Etapa 1:** Definir um conceito $\mathcal{C}$ que se deseja envenenar;
- **Etapa 2:** Definir o tamanho N_p do conjunto de imagens $I_{\mathcal{C}}$ do conceito $\mathcal{C}$ que sofrerão o envenenamento;
- **Etapa 3:** A partir de um conjunto $Data_{\mathcal{C}}$ que contém N imagens referentes ao conceito $\mathcal{C}$, encontrar as N_p imagens mais próximas do texto $t_{\mathcal{C}} = "A \text{ photography of } [\mathcal{C}]"$. Para isso, os autores utilizam a semelhança de cossenos entre imagem e texto definida por Radford et al. (2021), a qual eles denominam por pontuação CLIP (equação 2.9).

Supondo que E_{img} e E_{tex} são os codificadores imagens de dimensão 512×512 e de texto do modelo *Contrastive Language-Image Pre-Training* (CLIP), por conseguinte, a pontuação CLIP entre uma imagem do conceito $\mathcal{C}$, $I_{\mathcal{C}}$, e o texto $t_{\mathcal{C}}$ é definido da seguinte forma

$$CLIP = \frac{\langle E_{img}(I_{\mathcal{C}}), E_{tex}(t_{\mathcal{C}}) \rangle}{\|E_{img}(I_{\mathcal{C}})\| \cdot \|E_{tex}(t_{\mathcal{C}})\|} \quad (2.9)$$

em que $E_{img}(I_{\mathcal{C}})$ e $E_{tex}(t_{\mathcal{C}})$ são vetores de dimensões iguais.

Após comparar todas as imagens referentes ao conceito $\mathcal{C}$ do conjunto $Data_{\mathcal{C}}$, selecione as 5000 imagens $I_{\mathcal{C}}$ com a maior pontuação CLIP e, por fim, sorteie aleatoriamente N_p dessas imagens para obter o conjunto de imagens $Cand_p$ que serão envenenadas.

² Códigos de *deployment* são códigos de implantação, em tradução livre para o português

- **Etapa 4:** Definir um conceito âncora $\mathcal{A}$ que seja diferente do conceito $\mathcal{C}$.
- **Etapa 5:** Por meio de um modelo de difusão TTI, M_{TTI} , gerar uma imagem $I_{\mathcal{A}}$ do conceito âncora $\mathcal{A}$ a partir do texto $t_{\mathcal{A}} = "A \text{ photography of } [\mathcal{A}]"$. Isso deve ser realizado N_p vezes, de forma a gerar uma imagem âncora $I_{\mathcal{A}}$ para cada imagem $I_{\mathcal{C}}$ do conjunto $Cand_p$. Ao final desse processo, serão obtidos N_p pares $\{I_{\mathcal{C}}, I_{\mathcal{A}}\}$. Nessa etapa, os autores utilizaram a versão 2.1³ do modelo *Stable Diffusion* de Rombach et al. (2022).
- **Etapa 6:** Para cada par $\{I_{\mathcal{C}}, I_{\mathcal{A}}\}$, gere a versão envenenada da imagem origem original $I_{\mathcal{C}}$. Primeiro, a equação 2.10 define o processo de otimização utilizado com o intuito de encontrar a imagem envenenada $I_p = I_{\mathcal{C}} + \delta$.

$$\min_{\delta} D(E_{img}(I_{\mathcal{C}} + \delta), E_{img}(I_{\mathcal{A}})) \quad \text{sujeito a} \quad \|\delta\| < p \quad (2.10)$$

em que:

- E_{img} é mesmo codificador de imagens definido na etapa 3;
- $D(\cdot)$ é uma função de distância válida para o espaço de características estabelecido pelo codificador E_{img} ;
- $\|\delta\|$ é a perturbação adicionada à imagem $I_{\mathcal{C}}$;
- p é o maior valor que a perturbação $\|\delta\|$ pode assumir.

Em seguida, obter-se-á a imagem envenenada I_p ao resolver a otimização por meio do método da penalidade (equação 2.11), isto é,

$$\min_{\delta} \|E_{img}(I_{\mathcal{C}} + \delta) - E_{img}(I_{\mathcal{A}})\|_2^2 + \alpha \cdot \max(\|\delta\|_{LPIPS} - p, 0) \quad (2.11)$$

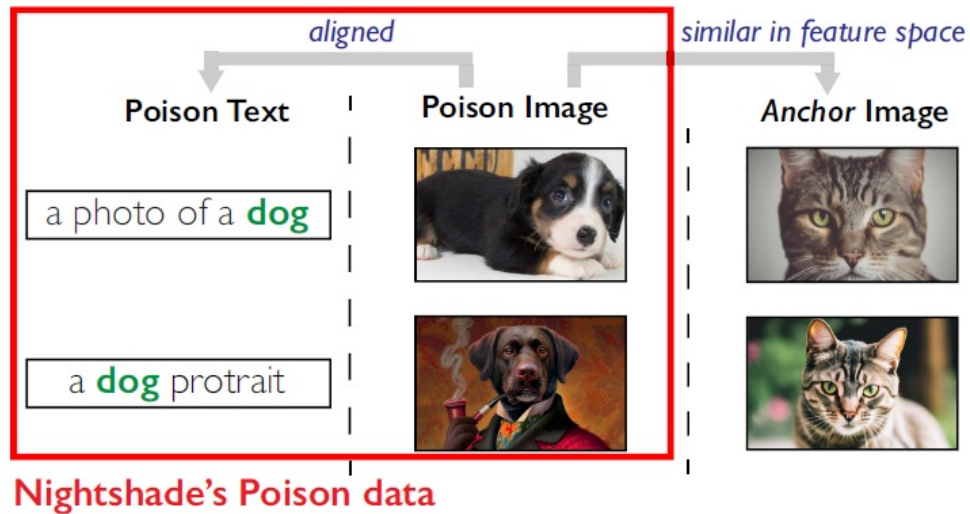
em que $\|\delta\|_{LPIPS} = LPIPS(I_{\mathcal{C}}, I_p)$ é a métrica definida por Zhang et al. (2018).

A figura 3 contém um exemplo visual das etapas 5 e 6.

- **Etapa 7:** Retreinar, do zero ou por meio da técnica de *fine-tune*, o mesmo modelo M_{TTI} utilizado na etapa 5, substituindo os N_p pares de imagens e suas respectivas descrições $\{I_{\mathcal{C}}, \text{prompt}\}$ por suas respectivas versões envenenadas $\{I_p, \text{prompt}\}$ na amostra de treinamento.

³ Disponível em: <https://huggingface.co/stabilityai/stable-diffusion-2-1>. Acesso em 12 set. 2025

Figura 3 – Exemplo de imagens envenenadas e suas respectivas imagens âncoras

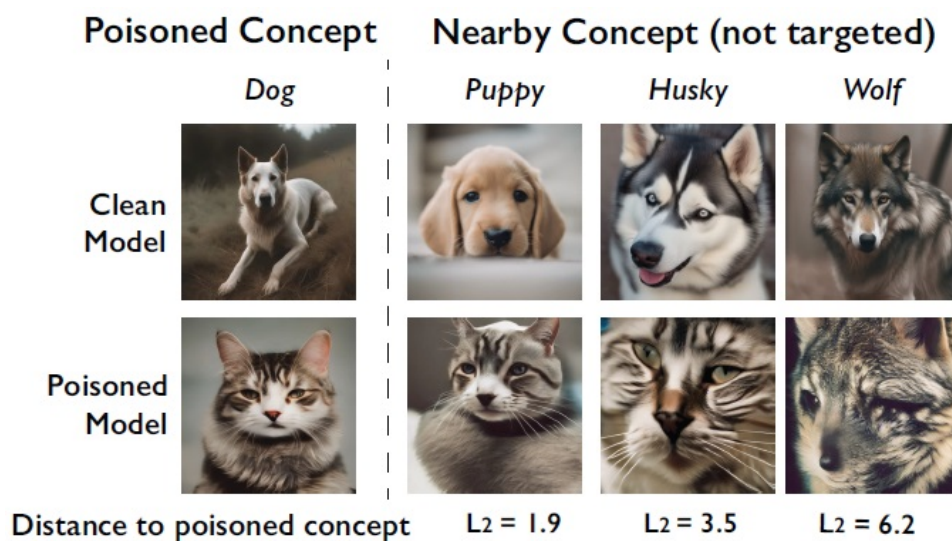


Fonte: Shan et al. (2024).

2.2.4 O efeito sangramento

Os autores do *Nightshade* observaram que conceitos que estão próximos do conceito $\mathcal{C}$ envenenado, mas que não foram escolhidos como alvo direto do ataque, também são afetados. Por exemplo, envenenar o conceito "dog" (em português, cachorro) também corrói a habilidade do modelo M_{TTI} de gerar imagens de "puppy" (em português, filhote de cachorro), "husky" ou, até mesmo, "wolf" (em português, lobo). A esse comportamento foi atribuída a denominação de "sangramento" (figura 4).

Figura 4 – Exemplo de sangramento no ataque "dog" para "cat"



Fonte: Shan et al. (2024).

Além disso, comparando as distâncias entre os *embeddings* de palavras do conceito $\mathcal{C}$ com os de conceitos relacionados, verificou-se que o sucesso do "sangramento"

diminuía conforme a distância aumentasse (tabela 1). Contudo, isso pode ser mitigado ao aumentar o tamanho da amostra envenenada, ou seja, inserir maiores quantidades de dados envenenados sobre um conceito $\mathcal{C}$ durante o treinamento de um modelo de difusão é capaz de aumentar a efetividade do ataque em relação ao conceito $\mathcal{C}$ (envenenamento) e aos conceitos que estão associados ao conceito $\mathcal{C}$ (sangramento).

Tabela 1 – Avaliando o efeito do sangramento a partir da pontuação média do CLIP

Distância L2 do conceito $\mathcal{C}$ envenenado	Conceitos próximos de $\mathcal{C}$ inclusos	CLIP score médio (em %)		
		100 $I_{\mathcal{C}}$ envenenadas	200 $I_{\mathcal{C}}$ envenenadas	300 $I_{\mathcal{C}}$ envenenadas
$D = 0$	1	85%	96%	97%
$0 < D \leq 3.0$	5	76%	94%	96%
$3.0 < D \leq 6.0$	13	69%	79%	88%
$6.0 < D \leq 9.0$	52	22%	36%	55%
$D \geq 9.0$	1929	5%	5%	6%

Fonte: Adaptado de Shan et al. (2024, p.10).

O “sangramento” é capaz de afetar o estilo artístico de imagens cujos *embeddings* de palavras estão afastados entre si. Por exemplo, as frases “a dragon” (em português, um dragão) e “fantasy art” (em português, arte fantástica) estão distantes no espaço dos *embeddings* de texto, porém elas estão relacionadas de outras maneiras. Para mostrar essa reação (figura 5), os autores exemplificaram com o envenenamento de dois conceitos: “fantasy art” e “Van Gogh style” (em português, estilo do Van Gogh).

Figura 5 – Envenenamento a partir de estilo artístico



Fonte: Shan et al. (2024).

O primeiro foi bem-sucedido, conseguindo realizar o envenenamento de conceitos relacionados ao estilo de arte fantástica. O segundo, em contrapartida, falhou. Os autores concluíram que não haverá envenenamento nem “sangramento” quando o conceito $\mathcal{C}$ alvo do ataque contém o nome de um artista. O ataque terá efetivo se $\mathcal{C}$ for o nome do estilo artístico - por exemplo, “Cubism” and “Impressionism” (em português, Cubismo e Impressionismo), “Anime”, “Cartoon”.

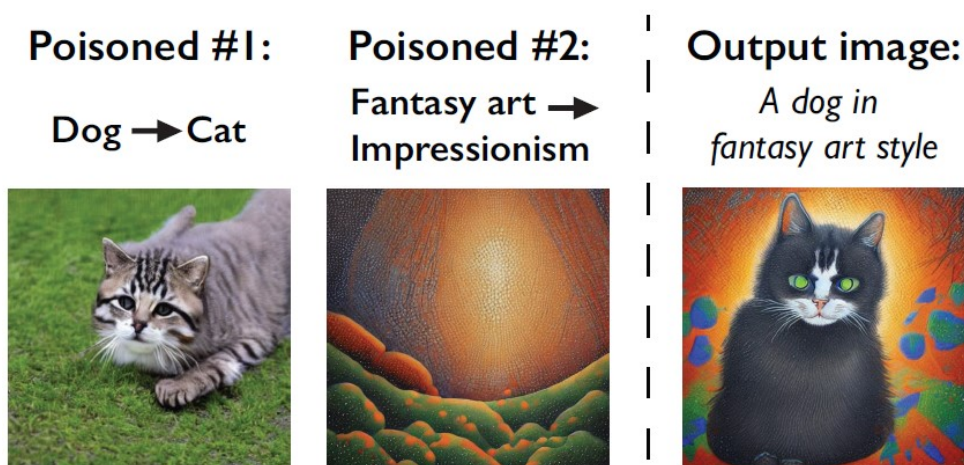
2.2.5 Composição de múltiplos envenenamentos

De acordo com os autores do *Nightshade*, é plausível que qualquer modelo TTI que utilize dados da internet será alvo de diferentes atacantes. Dessa forma, uma grande quantidade de conceitos $\mathcal{C}$ diferentes e sem relação estarão sendo envenenados ao mesmo tempo. Partindo desse pressuposto, eles realizaram dois ataques compostos para verificar quais consequências eles causam à capacidade de geração desses modelos.

Ao envenenar o conceito de “*dog*” para se tornar “*cat*” e o “*fantasy art*” para “*impressionism*” (figura 6), o modelo foi capaz de criar uma imagem de gato com traços artísticos do impressionismo a partir de um *prompt* cujo texto era “*A dog in fantasy art style*” (em português, um cachorro em estilo de arte fantástico).

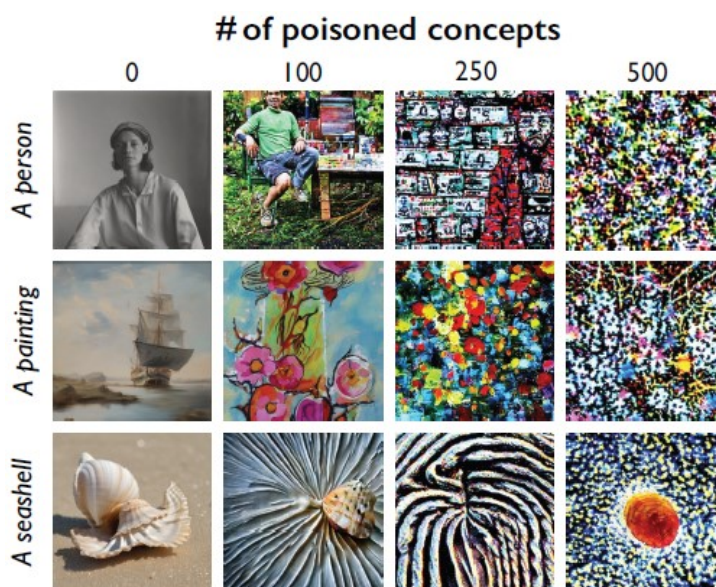
Por outro lado, quanto mais conceitos $\mathcal{C}$ envenenados vão alimentando o modelo, mais degradadas ficam suas imagens sintetizadas. A figura 7 demonstra esse resultado perfeitamente. A avaliação é feita por meio de três *prompts*, cujos textos são: “*A person*”, “*A painting*” e “*A seashell*” (em português, uma pessoa, uma pintura e uma concha). Inicialmente, mostra-se as imagens geradas pelo modelo intacto. Em seguida, o modelo é atacado por 100, 250 e 500 conceitos envenenados, respectivamente. No primeiro, os conceitos dos *prompts* ainda são perceptíveis nas imagens geradas. Já, no segundo, os conceitos dos *prompts* começam a sofrer deformação. Por fim, no terceiro caso, o modelo perde completamente a capacidade de representar os elementos exigidos pelos *prompts* fornecidos.

Figura 6 – Envenenamento composto de “*dog*” e “*fantasy art*”



Fonte: Shan et al. (2024).

Figura 7 – Degradação total do modelo



Fonte: Shan et al. (2024).

2.3 Áudio

O som, em sua essência, é um fenômeno físico que consiste na propagação de ondas de pressão através de um meio, como o ar (ZHANG et al., 2023). Quando captado por um microfone, essa variação de pressão é convertida em um sinal elétrico análogo, ou seja, uma tensão elétrica que varia continuamente no tempo, espelhando fielmente a forma da onda sonora original.

Um sinal analógico é, por definição, contínuo tanto no tempo quanto na amplitude. Isso significa que, para qualquer intervalo de tempo, por menor que seja, existem infinitos pontos de informação, e a amplitude pode assumir qualquer valor dentro de sua faixa de variação. A principal característica do sinal analógico é sua fidelidade direta ao fenômeno original, mas também sua suscetibilidade a ruídos e degradação ao longo de cada cópia ou transmissão realizada (POHLMANN, 2010).

Para superar essa vulnerabilidade, recorre-se à digitalização, o processo que converte a forma de onda contínua em uma representação numérica e discreta. A robustez dessa abordagem vem do fato de que uma sequência de bits, ao contrário de um sinal físico, pode ser copiada e regenerada perfeitamente, eliminando o problema da degradação cumulativa (SKLAR, 2001).

Essa conversão é realizada por um hardware conhecido como Conversor Analógico-Digital (em inglês, *Analogic Digital Conversor*, ADC), que opera através de duas etapas principais: a amostragem e a quantização (WATKINSON, 2001).

2.3.1 Amostragem

A amostragem consiste em medir a amplitude de um sinal analógico em intervalos de tempo constantes, convertendo-o em uma sequência de pontos discretos chamados de amostras. A velocidade dessas medições é determinada pela taxa de amostragem, cuja medida é expressa em *Hertz* (Hz).

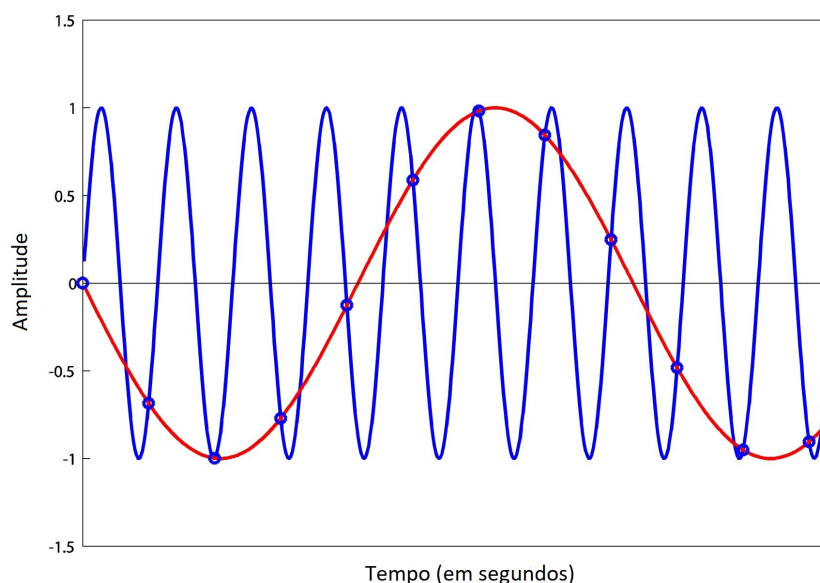
A escolha da taxa de amostragem não é arbitrária e é governada pelo Teorema da Amostragem de Nyquist-Shannon (LATHI; DING, 2018). Este teorema postula que, para reconstruir perfeitamente um sinal analógico a partir de suas amostras, a taxa de amostragem f_s deve ser, no mínimo, o dobro da frequência mais alta presente no sinal original f_{max} . Matematicamente, a condição está definida na equação 2.12

$$f_s \geq 2f_{max} \quad (2.12)$$

A faixa de audição humana se estende aproximadamente de 20 Hz a 20.000 Hz (20 kHz). Para representar digitalmente todas as frequências audíveis, a taxa de amostragem precisa ser superior a 40 kHz . Por essa razão, a taxa de 44.1 kHz foi estabelecida como padrão para o áudio em CDs (WATKINSON, 2001). Outras taxas são padrões em diferentes mídias, como 48 kHz para vídeo digital e taxas superiores como 96 kHz , utilizadas em produção de áudio de alta resolução para capturar nuances harmônicas complexas (HUBER, 2017).

Caso a taxa de amostragem seja inferior ao dobro da frequência máxima, ocorre um fenômeno indesejado chamado *aliasing*, onde frequências altas do sinal original são erroneamente interpretadas como frequências mais baixas no sinal digital e vice-versa, gerando distorção. Na figura 8, a onda azul, com uma frequência constante de 1000 Hz ($f_{max} = 1000$) foi erroneamente obtida ao utilizar uma taxa de amostragem de 880 Hz ($f_s = 880$), que resultou na leitura da onda vermelha.

Figura 8 – Onda azul sofre *aliasing*, resultando na mediação da onda vermelha



Fonte: Digital Sound & Music (s.d.).

2.3.2 Quantização

Após a amostragem definir os instantes de tempo, a quantização mede a amplitude de cada amostra e atribui a ela um valor numérico discreto, dentro de uma escala finita. Esse valor numérico é então representado em código binário (bits).

A precisão dessa medição é determinada pela profundidade de bits (*Bit Depth*, em inglês). Ela define o número de “degraus” ou níveis de amplitude disponíveis para representar o sinal. O número total de níveis é dado por 2^n , tal que n é o número de bits.

Na prática, os valores mais comuns de profundidade de bits são 8, 16 e 24 (quadro 1). Enquanto a profundidade de 8 bits foi essencial para a produção de trilhas sonoras e efeitos especiais na 3ª geração dos videogames (COLLINS, 2008), as outras duas são as mais utilizadas atualmente pela indústria musical (HUBER, 2017).

Quadro 1 – Informações sobre algumas das profundidades de bits mais conhecidas

Profundidade dos bits	Valores de amostragem	Intervalo de Amplitude	Aplicação
8	256	-128 a +127	Era dos consoles de 8-bits, por exemplo o Famicom da Nintendo e o Atari 7800
16	65536	-32.768 a +32.767	Em gravações de áudio para CD
24	16,777,216	-8,388,608 to +8,388,607	Em processos de mixagem e masterização realizados em estúdios de gravação profissionais

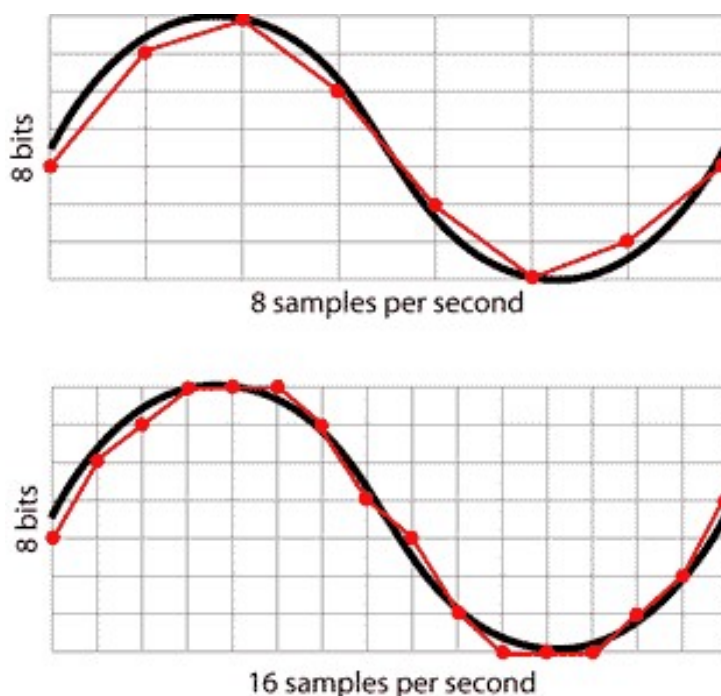
Fonte: Elaborada pelo próprio autor.

A diferença entre a amplitude real da amostra analógica e o nível discreto escolhido na quantização é chamada de erro de quantização. Esse erro se manifesta como um ruído de baixo nível no áudio digital. Quanto maior a profundidade de bits, menor o erro de quantização e, conseqüentemente, melhor a relação sinal-ruído (em inglês, *signal noise relation*, SNR) (WATKINSON, 2001).

Por meio das figuras 9 e 10, é possível compreender os efeitos causados por diferentes taxas de amostragem e profundidades de bits de forma isolada. A primeira imagem fixa o valor da profundidade de bits para avaliar o que ocorre quando se varia a taxa de amostragem, enquanto a segunda fixa a taxa de amostragem para observar o comportamento da quantização com diferentes profundidades de bits.

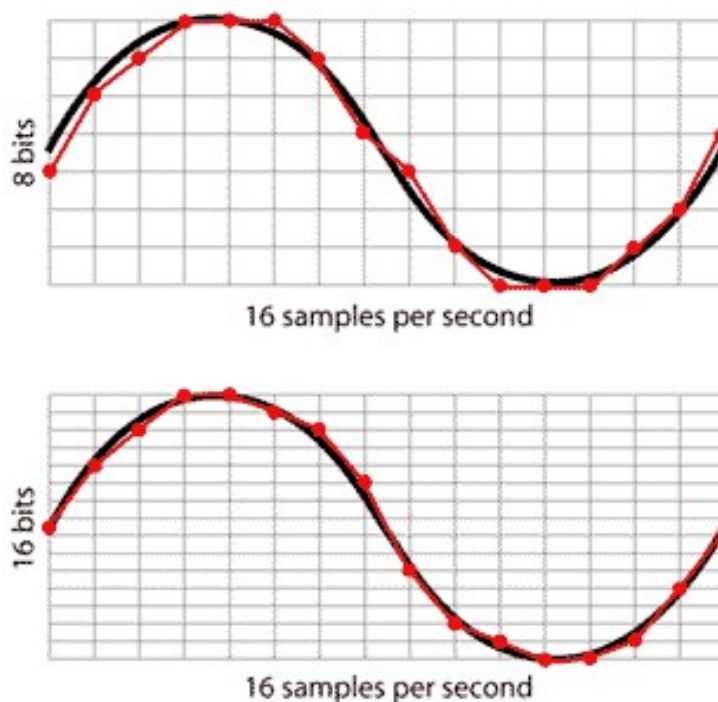
Pela primeira imagem, observa-se que quanto maior a taxa de amostragem, melhor será a aproximação do sinal original. Já, na segunda, compara-se o melhor resultado da figura 9 - que é a onda obtida com taxa de amostragem de 16 amostras por segundo e 8 bits de profundidade - com a estimativa do verdadeiro sinal quando se aumenta a profundidade de bit de 8 para 16. O resultado é uma estimativa ainda mais verossimilhante, mostrando que maiores os valores para taxa de amostragem e de profundidade de bits provocam melhores apurações dos dados reais.

Figura 9 – Exemplo de amostragem de áudio com profundidade de bit fixo



Fonte: Damien's Blog (2012).

Figura 10 – Exemplo de quantização de áudio com taxa de amostragem fixa



Fonte: Digital Audio Wiki (2014).

2.3.3 Tipos de arquivo de áudio

Ao final do processo de amostragem e quantização, o sinal sonoro analógico é convertido em um fluxo de dados binários, denominados de modulação pulso-código (em inglês, *pulse-code modulation*, PCM). Essa representação constitui a forma mais pura do áudio digital.

Esse fluxo de dados PCM, conforme explica Pohlmann (2010), pode ser armazenado em diferentes formatos de arquivo, os quais são agrupados em três categorias principais de acordo com o método de compressão utilizado:

- **Não Comprimido:** armazena os dados PCM brutos, sem qualquer alteração, garantindo a máxima fidelidade ao áudio original. Os exemplos mais comuns são Formato de Arquivo de Áudio em forma de Onda e Formato de Arquivo de Intercâmbio de Áudio (em inglês, *Waveform Audio File Format* e *Audio Interchange File Format*, WAV e AIFF, respectivamente). Eles são bastante utilizados em gravações de estúdio, pós-produção de áudio (edição, mixagem e masterização) e preservação digital de áudios com valor histórico. O tamanho desses arquivos é de 10 MB por minuto de gravação.
- **Comprimido Sem Perdas (*Lossless*):** reduz o tamanho do arquivo por meio de algoritmos que otimizam a sequência de dados, mas sem descartar nenhuma informação. Isso permite que o áudio original seja perfeitamente reconstruído. Os

formatos mais conhecidos nesta categoria são o Codec de Áudio Sem Perdas Gratuito e Codec de Áudio Sem Perdas da *Apple* (em inglês, *Free Lossless Audio Codec* e *Apple Lossless Audio Codec*, FLAC e ALAC, respectivamente) utilizados para o armazenamento de áudio em CD e em dispositivos da *Apple*, respectivamente. O tamanho desses arquivos varia de 4 a 7 MB por minuto de gravação.

- **Comprimido Com Perdas (Lossy):** reduz drasticamente o tamanho do arquivo ao eliminar dados sonoros que são considerados psicoacusticamente menos perceptíveis ao ouvido humano. Embora haja uma perda de qualidade, essa compressão é ideal para armazenamento e transmissão de áudio em serviços de *streaming*, como *Spotify*, *Apple Music* e *YouTube Music*, e durante a instalação e execução da cópia digital de um jogo. Os exemplos mais populares são MPEG Camada de Áudio III e Codificação Avançada de Áudio (em inglês, *MPEG Audio Layer III* e *Advanced Audio Coding*, MP3 e AAC, respectivamente) cujos tamanhos de arquivo podem variar entre 1 a 2.4 MB por minuto de gravação.

2.3.4 Espectrograma de log-mel

O grau de aprendizado da máquina é dependente das informações que lhe são fornecidas. No campo do áudio, os arquivos de áudio carregam todas as informações do som digitalizado por meio da representação em forma de onda. Ela contém apenas as informações sobre os valores de amplitude obtidos em relação ao tempo de gravação. Como o valor da amplitude carece de interpretabilidade, logo o formato de onda não é o formato ideal para que a máquina aprenda.

Apesar de a forma de onda não conter informações explícitas e interpretáveis, é possível obter informações relevantes sobre um áudio ao transformar a onda em uma representação de espectrograma de log-mel. Além de parecer uma “fotografia” do áudio, ela contém informações que mais sensibilizam a audição humana: a frequência (em escala mels) e a pressão (medida em decibels) do som ao longo do tempo de gravação (LACERDA et al., 2021). Por conseguinte, o uso de espectrograma de log-mel é essencial para problemas que envolvam o estudo da fala, como, por exemplo, a construção de modelos TTS e de vocoders (ZHANG et al., 2023).

De acordo com El-Khasawneh et al. (2025), a conversão da forma onda para a forma de espectrograma de log-mel consiste em quatro etapas: (i) dividir o sinal de onda em frames de mesmo tamanho, (ii) aplicar a Transformada rápida de Fourier (em inglês, *Fast Fourier Transform*, FFT) em cada um desses frames para obter o espectrograma do áudio, (iii) obter o espectrograma de magnitude ao aplicar a função modular em cada valor do espectrograma e, por último, (iv) aplicar o filtros mels seguidos pela função logarítmica na base 10 para obter o espectrograma de log-mel.

As duas primeiras etapas são contempladas pelo método Transforma de Fourier de curto termo (em inglês, *Short-Time Fourier Transform*, STFT) - definido pela equação 2.13 e representado visualmente pela figura 11.

$$X(n_{frame}, k) = \sum_{a=0}^{A_f-1} x[a + n_{frames}H] w[a] e^{\frac{-i2\pi kn_{frame}}{N_{frame}}}, \quad (2.13)$$

em que:

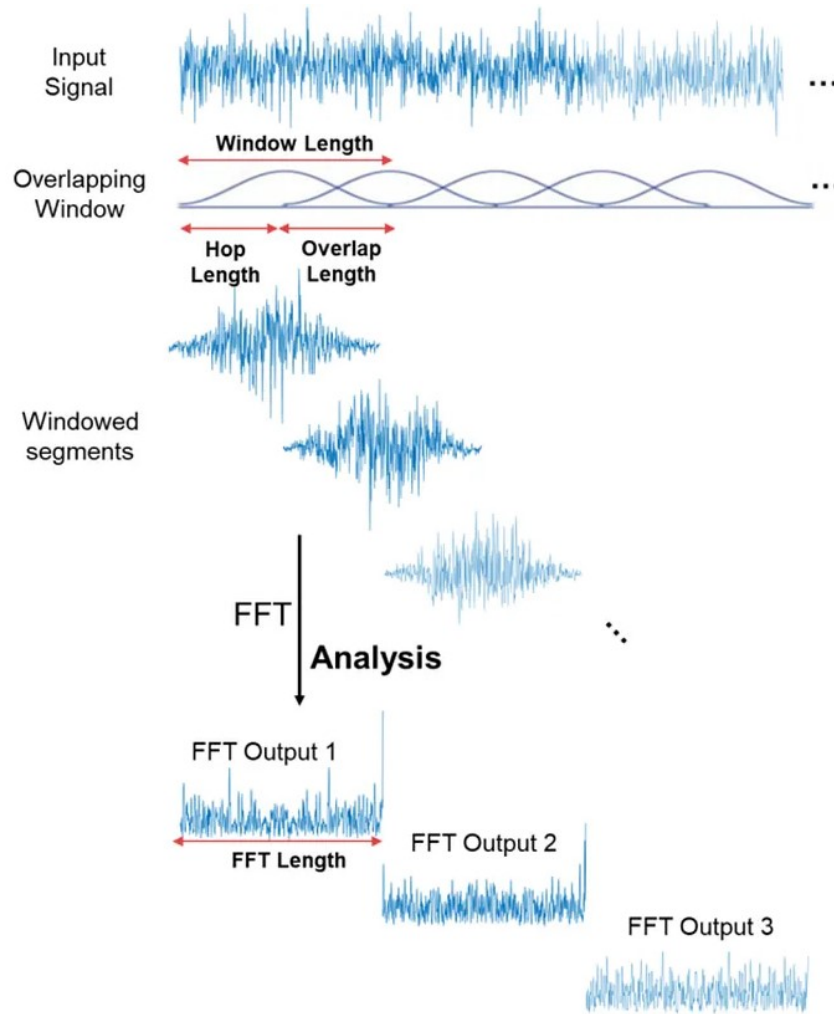
- $X(n_{frame}, k)$ é o valor do espectograma no frame n_{frame} e no intervalo de frequências k ;
- n_{frame} é o índice do n-ésimo frame;
- k é o índice referente ao k-ésimo intervalo de frequências;
- A_f é o tamanho do frame (quantidade de amostras que o frame deve conter);
- H distância entre os pontos (amostras) iniciais de dois frames sucessivos (em inglês, é conhecido como *hop length*);
- $x[a + n_{frame}H]$ é o sinal de áudio original na a-ésima amostra do frame n_{frame} ;
- $w[a]$ é a função janela aplicada na a-ésima amostra do frame n_{frame} ;
- $e^{\frac{-i2\pi kn_{frame}}{N_{frames}}}$ é a função base da Transformada de Fourier discreta (em inglês, *Discrete Fourier Transform*, DFT).

A função janela mais utilizada nesse processo é a janela de Hann (equação 2.14), definida por ALLEN e RABINER (1977):

$$w[a] = \begin{cases} 0.5 - 0.46 \cos\left(\frac{2\pi a}{A_f - 1}\right) & \text{se } 0 \leq a \leq A_f - 1 \\ 0 & \text{caso contrário.} \end{cases} \quad (2.14)$$

Além disso, nota-se que a equação 2.13 está definida a partir do método de resolução de DFT, o qual tem complexidade $O((N_{frames})^2)$, onde N_{frames} é o número total de frames. Para acelerar esse cálculo, essa equação deve ser resolvida por meio do método FFT, um algoritmo de divisão e conquista de complexidade $O(N_{frames} \log[N_{frames}])$ (OPPENHEIM; SCHAFER, 2012).

Figura 11 – Visualização do processo de STFT



Fonte: Math Works (2025).

Como o espectrograma é composto apenas por números complexos $X(n_{frame}, k)$ (equação 2.15), é necessário encontrar uma forma de espectrograma equivalente que pertença apenas ao campo dos reais.

Para isso, na terceira etapa, aplica-se a função modular aos valores do espectrograma $X(n_{frame}, k)$. Dessa forma, o resultado obtido é o espectrograma de magnitude, cujos valores são $S_{mag}(n_{frame}, k) = |X(n_{frame}, k)|$. Como $X(n_{frame}, k)$ é um número complexo

$$X(n_{frame}, k) = a + ib, \quad (2.15)$$

tal que $i^2 = -1$, a é parte real e b é a parte imaginária, então seu valor absoluto é dado por $|X(n_{frame}, k)| = \sqrt{a^2 + b^2}$.

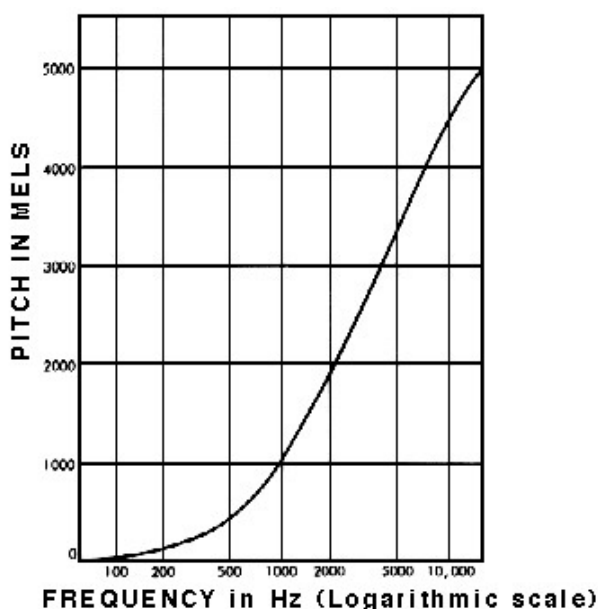
Caso a função aplicada seja uma função quadrática, o resultado obtido será o espectrograma de poder, cujos valores são $S_{pow}(n_{frame}, k) = |X(n_{frame}, k)|^2$.

Ambas as representações obtidas na terceira etapa já podem ser interpretadas como o áudio na forma de imagem. Porém, o conteúdo que o espectograma de magnitude ou o de poder carregam não condiz com a percepção humana a respeito dos sons. Stevens, Volkmann e Newman (1937) desenvolveram uma escala que é capaz de identificar qual é o intervalo de frequências (em *Hertz*) mais relevantes para o ouvido humano, denominada de escala mel.

Quando o interesse é identificar se um som tem *pitch* baixo (grave) ou alto (agudo), recorre-se ao estudo das frequências do som, tal que maiores frequências apontam para sons com *pitch* alto, em contrapartida, menores frequências, para sons com *pitch* baixo.

Pela figura 12, a diferença entre o *pitch* do som no intervalo de 1000 a 8000 *hertz* segue um comportamento linear, ou seja, o aumento ou decréscimo da frequência é facilmente interpretado como sons diferentes pelo ouvido humano. Contudo, no intervalo de frequências acima de 10000 *hertz*, esse comportamento é logarítmico, isto é, o ouvido humano possui dificuldades ao distinguir o *pitch* entre sons, por exemplo, de 12000 e 14000 *hertz*. Já, no intervalo de frequências menores do que 1000 *hertz*, uma pequena variação na frequência, por exemplo 10 *hertz*, é o suficiente para o ouvido humano distinguir o *pitch* entre dois sons.

Figura 12 – Relação logarítmica entre frequências em *hertz* e mels



Fonte: Stevens, Volkmann e Newman (1937).

Em termos matemáticos, a conversão da frequência expressa em *hertz* para aquela expressa em mels ocorre por meio da equação 2.16 (ALMUJALLY et al., 2024).

$$mel(f) = 2595 \log_{10} \left(1 + \frac{f}{700} \right) \quad (2.16)$$

Realizar essa conversão não terá como resultado o espectrograma de mel. Novamente, será fundamental utilizar uma técnica de janela, porém uma que seja aplicada no domínio da frequência mel.

Os bancos de filtro mels são responsáveis por efetuar essa tarefa. Primeiro, deve-se encontrar a frequência mínima f_{min} e máxima f_{max} do som, de forma que f_{max} atenda ao Teorema da Amostragem de Nyquist-Shannon (equação 2.12). Segundo, converter essas duas frequências para a escala mels, ou seja, $mel_{low} = mel(f_{min})$ e $mel_{high} = mel(f_{max})$. Terceiro, definir o número M de bancos de filtro mel a serem aplicados ao longo do som e encontrar os $M + 2$ pontos que irão formá-los ao dividir o intervalo $[f_{min}, f_{max}]$ em $M + 1$ partes iguais, conforme a relação estabelecida na equação 2.17.

$$mel_i = mel_{low} + i \left[\frac{mel_{high} - mel_{low}}{M + 1} \right], \quad i = 0, \dots, M + 1 \quad (2.17)$$

Em seguida, converter o valor de cada ponto do intervalo de filtro mel para a escala em hertz (equação 2.18), aplicando o processo inverso de 2.16, isto é, $f_i = f(mel_i)$ em que

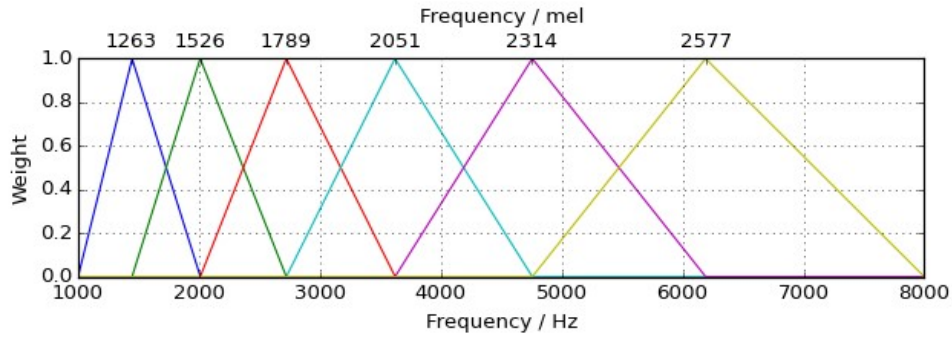
$$f(mel) = 700 \left(10^{\frac{mel}{2595}} - 1 \right) \quad (2.18)$$

Agora, com os $M + 2$ pontos no campo dos reais, é possível formar M triângulos com 3 pontos consecutivos. Esses triângulos são os bancos de filtro de mel $H_m[k]$ necessários para encontrar o espectrograma de mel. De acordo Bhuvanagiri e Kopparapu (2014), o tamanho dos triângulos segue a relação estabelecida na equação 2.19

$$H_m[f_k] = \begin{cases} 0 & \text{se } f_k < f_m \\ \frac{f_k - f_m}{f_{m+1} - f_m} & \text{se } f_m \leq f_k \leq f_{m+1} \\ \frac{f_{m+2} - f_k}{f_{m+2} - f_{m+1}} & \text{se } f_{m+1} \leq f_k \leq f_{m+2} \\ 0 & \text{se } f_k > f_{m+2} \end{cases} \quad (2.19)$$

em que $H_m[f_k]$ é o valor da função triangular obtido quando se aplicada a uma frequência f_k o m -ésimo banco de filtro mel, $m = 0, \dots, M - 1$. Conforme a figura 13, quanto maior for o valor da frequência, maior será o triângulo construído.

Figura 13 – Bancos de filtro de mel



Fonte: Ciapessoni (2017).

Aplicando os bancos de filtro mel ao espectrograma $S(n_{frame}, k)$, tem-se como resultado o espectrograma de mel $S_{mel}(n_{frame}, m)$ (equação 2.20).

$$S_{mel}(n_{frame}, m) = \sum_{k=0}^{K-1} H_m(f_k) S(n_{frame}, k), \quad (2.20)$$

em que n_{frame} é o índice do frame, m é índice do banco de filtro de mel, K é o número de frames e $S(n_{frame}, k) = S_{mag}(n_{frame}, k)$ quando estivermos fazendo a conversão a partir do espectrograma de magnitude e $S(n_{frame}, k) = S_{pow}(n_{frame}, k)$, do espectrograma de poder.

Por fim, o espectrograma de log-mel S_{mel_db} é a logaritmização do espectrograma mel, definida pela equação 2.21.

$$S_{mel_db} = 10 \log_{10}(S_{mel}(n_{frame}, m)) \quad (2.21)$$

2.3.5 Retornando espectrograma à forma de onda

Quando o áudio é convertido da forma de onda para a forma de espectrograma de magnitude, ocorre a perda da informação referente à fase da onda. Sem ela, torna-se impossível encontrar o formato de onda exato dado um espectrograma de magnitude qualquer. Isso ocorre porque a construção de um espectrograma é edificada sobre a ideia de coordenadas polares.

Os coeficientes do espectrograma de magnitude $|X(n_{frame}, k)|$ representam diferentes tamanhos de raio de circunferência e a fase (equação 2.22) representa o ângulo de inclinação $\theta(n_{frame}, k)$ de cada um desses raios de forma que

$$\theta(n_{frame}, k) = \arctan\left(\frac{b}{a}\right), \quad (2.22)$$

onde a é a parte real e b a parte imaginária dos coeficientes do espectrograma $X(n_{frame}, k)$ (equação 2.15).

Diversos valores de a e b podem resultar em um mesmo ângulo $\theta(n_{frame}, k)$ ou seja, um único ângulo pode estar atrelado a diferentes espectrogramas $X(n_{frame}, k)$. Apesar de a conversão de um espectrograma para a onda de áudio existir, não é possível, portanto, obter uma transformação inversa que retorne um espectrograma de magnitude para a sua verdadeira forma de espectrograma.

Masuyama, Ueno e Ono (2023) formalizam esse impasse pela definição matemática. Seja x um áudio, $X = STFT(x)$ é a forma de espectrograma do áudio x que resulta em F intervalos de frequência e T frames. Como os coeficientes do espectrograma X são números complexos (equação 2.15), logo o espectrograma X pertence ao campo dos complexos de forma que $X \in \mathbb{C}^{F \times T}$.

Dado que um número complexo é uma relação linear entre os coeficientes real a e imaginário b , logo define-se a reta $\mathcal{C} \subset \mathbb{C}^{F \times T}$ como a imagem da transformação $STFT$, em outras palavras, o subespaço que contém todos as representações de espectrogramas X com F intervalos de frequência e T frames de todos os áudios x . Se a reconstrução do espectrograma X (equação 2.23)

$$X_{rec} = STFT(iSTFT(X)), \quad (2.23)$$

não pertencer à reta $\mathcal{C}$, $X_{rec} \notin \mathcal{C}$, então o espectrograma de magnitude do áudio x reconstruído é diferente do original, ou seja, $|X_{rec}| \neq |X|$. Na equação 2.23, $iSTFT$ é a respectiva função inversa da $STFT$, também elaborada por GRIFFIN e LIM (1984).

A fim de garantir X_{rec} possua fase e magnitude iguais a do espectrograma original X , é proposto que se encontre o espectrograma de magnitude reconstruído $|A|$ mais parecido com o original $|X|$ ao efetuar a otimização definida na equação 2.24

$$\underset{A}{\operatorname{argmin}} || |X| - |A| ||^2, \quad \text{tal que } A \in (\mathcal{C} \cap \mathcal{A}), \quad (2.24)$$

onde $\mathcal{A} = \{A \in \mathbb{C}^{F \times T} \mid |A| = |X|\}$ é o conjunto de todos os espectrogramas reconstruídos A que possuem a mesma magnitude de X e $|| \cdot ||^2$ é a norma quadrática de Frobenius.

2.3.5.1 Algoritmo de Griffin-Lim

Proposto por GRIFFIN e LIM (1984), o algoritmo de Griffin-Lim foi capaz de solucionar essa otimização ao propor um método iterativo para a reconstrução da fase do espectrograma. A ideia principal desse método é utilizar projeções para os conjuntos $\mathcal{C}$ de todos os espectrogramas que possuem a mesma a fase e $\mathcal{A}$ de todos os

espectogramas que possuem mesma magnitude de forma alternada para encontrar o espectrograma reconstruído A com a fase e a magnitude mais próximos do espectrograma original X . De forma sucinta, o algoritmo busca encontrar um espectrogram $\hat{X}$ que pertença, simultaneamente, aos conjuntos $\mathcal{C}$ e $\mathcal{A}$.

Primeiro, sorteia-se um ângulo $\hat{\theta}^{(0)}(n_{frame}, k)$ no intervalo $[0, 2\pi]$. A partir de um espectrograma de magnitude $S_{mag}(n_{frame}, k)$ conhecido e $\hat{\theta}^{(0)} = \hat{\theta}^{(0)}(n_{frame}, k)$ o ângulo sorteado, é possível estimar o espectrograma inicial $\hat{X}^{(0)}(n_{frame}, k)$ do conjunto $\mathcal{A}$, utilizando a relação 2.25. A representação inicial do espectrograma $\hat{X}^{(0)}$ é dada pela figura 14.

$$\hat{X}^{(0)}(n_{frame}, k) = S_{mag}(n_{frame}, k) e^{i\hat{\theta}^{(0)}} \quad (2.25)$$

Em seguida, inicia-se o processo de intercalação de projeções entre os dois conjuntos. Esse processo é resumido, visualmente, pela figura 15 e, matematicamente, pela equação 2.26

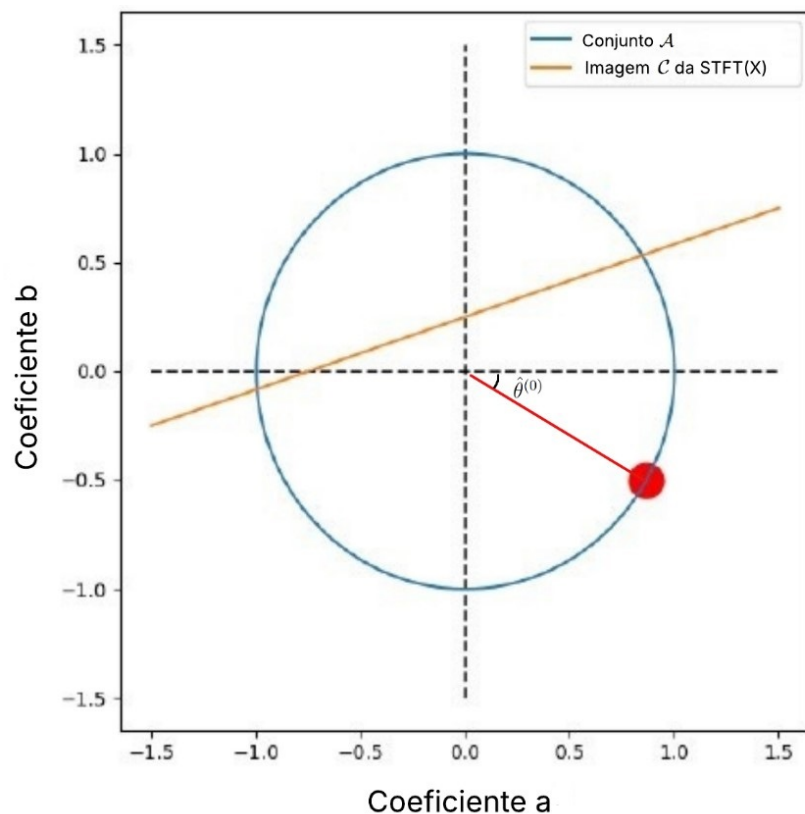
$$\hat{X}^{(i+1)}(n_{frame}, k) = P_C[P_A[\hat{\theta}^{(i)}]], \quad (2.26)$$

em que $P_A[\hat{\theta}^{(i)}] = \hat{X}^{(i)}(n_{frame}, k)$ encontrará, a partir do valor da fase $\hat{\theta}^{(i)}$, uma projeção no conjunto $\mathcal{A}$ - com magnitude igual a $S_{mag}(n_{frame}, k)$ para um espectrograma no conjunto $\mathcal{C}$ e $P_C[\hat{X}^{(i)}(n_{frame}, k)] = STFT[iSTFT[\hat{X}^{(i)}(n_{frame}, k)]]$ - será responsável por estimar o próximo espectrograma $\hat{X}^{(i+1)}$ do conjunto da imagem da STFT (conjunto $\mathcal{C}$) - que não, necessariamente, possui magnitude igual à desejada $S_{mag}(n_{dej}, k_{dej})$ - e, juntamente a ele, o valor de sua fase $\hat{\theta}^{(i+1)}$ por meio da equação 2.22. A partir dessas informações, verifica-se que a equação 2.25 é um caso particular de $P_A[\hat{\theta}^{(i)}]$. Por fim, esse processo irá ocorrer até que haja convergência do método, definida pelo critério de parada a partir da equação 2.27

$$\frac{\|\hat{X}^{(i+1)} - \hat{X}^{(i)}\|_2}{\|\hat{X}^{(i)}\|_2} < \varepsilon, \quad (2.27)$$

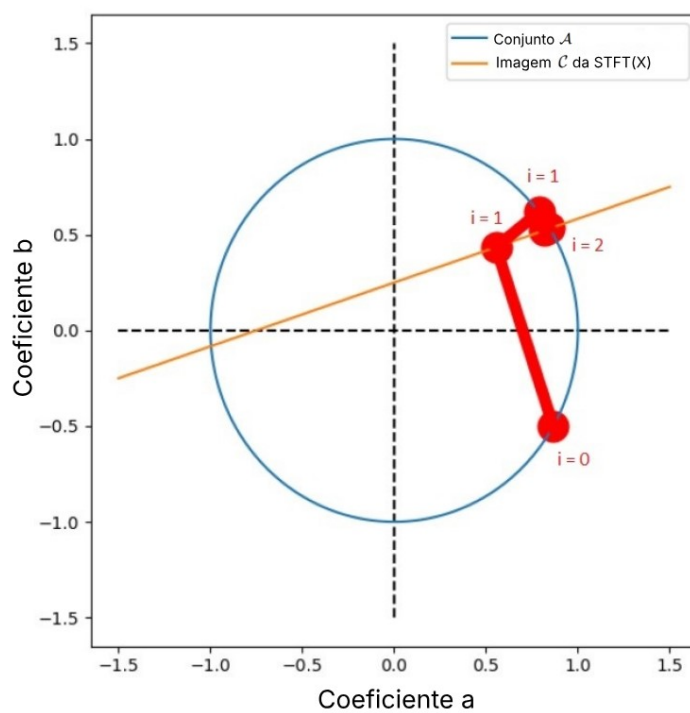
com $\|\hat{X}^{(i)}\|_2 = \sqrt{\sum_n \sum_k |\hat{X}^{(i)}(n_{frame}, k)|^2}$ sendo a norma de Froebius. Finalizada as interações, por conseguinte, o áudio resultante W_{res} que mais se aproxima do espectrograma de magnitude $S_{mag}(n_{dej}, k_{dej})$ conhecido é $W_{res} = P_A[\hat{\theta}^{(i+1)}]$.

Figura 14 – Espectrograma inicial $\hat{X}^{(0)}(n_{frame}, k)$ com magnitude igual a $S_{mag}(n_{frame}, k)$



Fonte: Adaptado de Hasegawa-Johnson (2023).

Figura 15 – Exemplo de execução do algoritmo de Griffin-Lim



Fonte: Adaptado de Hasegawa-Johnson (2023).

O algoritmo de Griffin-Lim para reconstrução de fases é utilizado apenas quando o interesse é converter um espectrograma de magnitude de volta para a forma de onda. Agora, quando se deseja fazer a conversão do espectrograma de mel para a forma de onda, essa conversão é composta por duas transformações, uma para estimar o espectrograma de magnitude equivalente para um espectrograma mel inicialmente conhecido e a outra dada pelo algoritmo de Griffin-Lim.

A primeira conversão é matematicamente formulada com base na função objetivo (equação 2.28)

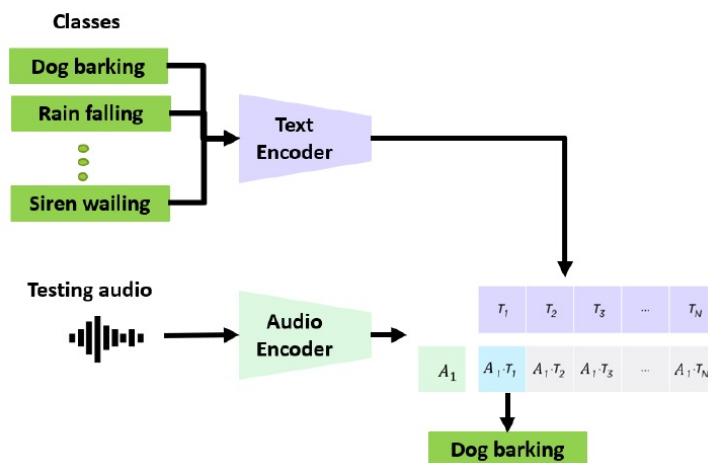
$$\min_Y = \frac{1}{2} ||EY - M||_2, \quad (2.28)$$

onde E são os bancos de filtro mel, Y é o espectrograma de magnitude que se deseja estimar e M , o espectrograma de mel a ser convertido para onda. Portanto, finalizada a conversão para espectrograma de magnitude, a próxima etapa consiste apenas em aplicar o algoritmo de Griffin-Lim no espectrograma de magnitude estimado.

2.4 CLAP

O pré-treinamento contrastivo entre linguagem e áudio (em inglês, *Contrastive Language-Audio Pre-Training*, CLAP) é um modelo de aprendizado profundo capaz de prever o quão próximos estão um áudio e um texto (WU* et al., 2023). Para isso, ele utiliza dois codificadores capazes de alinhar quaisquer pares de áudio e texto em um espaço latente de multimodalidade, de forma que ambos sejam vetores de mesmo tamanho. Por fim, a proximidade é dada pelo valor do cosseno obtido a partir do produto escalar entre esses dois vetores (da mesma forma que o modelo CLIP, pela equação 2.9).

Figura 16 – Calculando proximidade de um áudio em relação a uma lista de textos



Fonte: Wu* et al. (2023).

Sua atuação não é restrita apenas à áudios de fala humana. O CLAP, além de informar qual a proximidade entre uma fala com um ou mais textos (figura 16), é capaz de estender essa habilidade para sons de fundo (como barulho de chuva, de carro, de animais) e músicas.

2.4.1 Algoritmo de treinamento dos codificadores de áudio e texto

Suponha que $X_a \in \mathbb{R}^{F \times T}$ seja o espectograma mel de um áudio - o qual é uma matriz cujas F linhas são o número de bandas mel, T é o número de frames e X_t , um texto.

O treinamento dos codificadores é feito por lote, isto é, divide-se a amostra de treinamento em subamostras de tamanho reduzido. Se cada lote contém N pares de áudio e texto, então o lote pode ser definido como $B = \{\vec{X}_a, \vec{X}_t\}$, em que $\vec{X}_a = (X_{a1}, \dots, X_{aN})$ é uma lista de N espectogramas mels e $\vec{X}_t = (X_{t1}, \dots, X_{tN})$ é uma lista de N textos.

Definindo $f_a(\cdot)$ e $f_b(\cdot)$ como o codificador de áudio e texto, tais que $\hat{X}_{ai} = f_a(X_{ai})$ representa a codificação do i -ésimo espectograma mel enquanto $\hat{X}_{ti} = f_b(X_{ti})$, a codificação do i -ésimo texto do lote B . Após a codificação, o áudio pertence ao espaço latente dos áudios, ou seja, $\hat{X}_a \in \mathbb{R}^V$ e o texto, ao espaço latente dos textos, $\hat{X}_t \in \mathbb{R}^U$. Ao aplicá-los em todos os pares de espectograma mel e texto de um lote B , obter-se-á os vetores de áudio e texto codificados (*embeddings*) $\vec{\hat{X}}_a \in \mathbb{R}^{N \times V}$ e $\vec{\hat{X}}_t \in \mathbb{R}^{N \times U}$. Por consequência, o lote B estará codificado. Nessa etapa, os autores escolheram os modelos pré-treinados CNN14 (uma rede neural convulucional) e BERT (*Bidirecional Encoder Representations from Transformers*) como codificadores de áudio $f_a(\cdot)$ e texto $f_b(\cdot)$, respectivamente. Enquanto o codificador CNN14 transforma um áudio em uma representação vetorial de tamanho $V = 2048$, o codificador BERT, um texto em uma representação vetorial de dimensão $U = 768$, ou seja, $\hat{X}_a \in \mathbb{R}^{2048}$ e $\hat{X}_t \in \mathbb{R}^{768}$.

Para comparar áudios e textos, é preciso que ambos sejam elementos de um mesmo espaço. Diante disso, a proposição dos autores para possibilitar a comparação entre $\hat{X}_a$ e $\hat{X}_t$ comparáveis, foi a aplicação uma projeção linear após a etapa da codificação, de forma que $E_a = L_a(\hat{X}_a) \in \mathbb{R}^{1 \times d}$ e $E_t = L_t(\hat{X}_t) \in \mathbb{R}^{1 \times d}$ sejam as respectivas transformações dos espectogramas mels e textos codificados para o espaço latente de multimodalidade. Aqui, os autores definiram que os espaço latente multimodal será composto apenas por vetores de tamanho $d = 1024$, isto é, $E_a, E_t \in \mathbb{R}^{1 \times 1024}$.

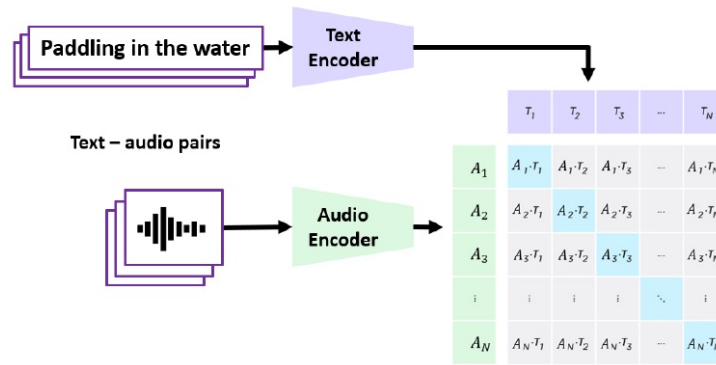
Uma vez que os áudios e os textos se encontram no espaço latente de multimodalidade, é possível compará-los um a um com uma medida de similaridade. A equação 2.29 define a pontuação CLAP como essa medida.

$$C = \tau * (E_t \cdot E_a^T), \quad C \in \mathbb{R} \quad (2.29)$$

Se $\tau = 1$, a pontuação CLAP (equação 2.29) será a similaridade de cosseno (equação 2.9), pois $\|E_t\| = \|E_a\| = 1$.

Ao aplicar essa medida para todas as combinações possíveis entre áudios e textos, será obtida a matriz de similaridade entre áudios e textos $\hat{C} \in \mathbb{R}^{N \times N}$ (figura 17).

Figura 17 – Calculando proximidade de um áudio em relação a uma lista de textos



Fonte: Wu* et al. (2023).

A diagonal principal da matriz de similaridade $\hat{C}$ representa a relação correta entre áudio e texto. Para treinar conjuntamente os codificadores de áudio L_a e texto L_t , os autores adotaram a função de perda de entropia cruzada simétrica descrita pela equação 2.30

$$\mathcal{L} = 0.5 \cdot l_{text}(\hat{C}) + 0.5 \cdot l_{audio}(\hat{C}), \quad (2.30)$$

em que l_k são as probabilidades dos logits do áudio ($k = audio$) ou do texto ($k = text$) definidas na equação 2.31.

$$l_k = \frac{1}{N} \sum_{i=0}^N \log[diag(softmax(\hat{C}))] \quad (2.31)$$

2.5 StyleTTS2

O StyleTTS 2, modelo proposto por Li et al. (2023) representa um avanço significativo na área de síntese de fala, alcançando um nível de naturalidade e expressividade comparável à fala humana. Sua arquitetura inovadora combina modelos de difusão, treinamento adversarial e o poder dos grandes modelos de linguagem de fala (em inglês, *speech language models*, SLMs) para superar as limitações de sistemas anteriores.

2.5.1 O problema de expressividade e o conceito de “estilo”

Sistemas TTS tradicionais, embora inteligíveis, frequentemente carecem da prosódia, entonação e variações rítmicas que caracterizam a fala humana. A síntese de fala não se resume apenas à correta pronúncia de fonemas, mas também à transmissão de emoções, ênfases e intenções. O StyleTTS 2 aborda este desafio tratando esses elementos paralinguísticos como um “estilo” generalizado.

Ao modelar o estilo como uma variável aleatória latente, isso permite gerar uma fala expressiva e apropriada para o texto de entrada sem a necessidade de um áudio de referência, um modo de operação conhecido como síntese não referenciada. Matematicamente, a equação 2.32 modela a fala x como uma distribuição condicional ao texto t , marginalizada sobre uma variável latente de estilo s :

$$p(x|t) = \int p(x|t, s)p(s|t)ds \quad (2.32)$$

Nesta equação, s representa o vetor de estilo, uma variável latente que encapsula todas as características da fala que não estão diretamente contidas no conteúdo fonético do texto t , como prosódia, ritmo, ênfase lexical e até mesmo o timbre do locutor.

A inovação central do StyleTTS 2 reside em como ele modela e gera essa variável s . Os autores propõe que s seja estimado por meio do método diferencial do elemento (em inglês, *Element Differential Method*, EDM) de Karras et al. (2022), apresentada na equação 2.33

$$s = \int -\sigma(\tau)[\beta(\tau)\sigma(\tau) + \dot{\sigma}(\tau)]\nabla_s \log[p_r(s|t)]d\tau + \int \sqrt{2\beta(\tau)}\sigma(\tau)d\tilde{W}_\tau, \quad (2.33)$$

que combina a soma entre as equações 2.34 e 2.35, isto é, uma equação diferencial ordinária (em inglês, *Ordinary Differential Equation*, ODE) com probabilidade fluida de Huang, Lim e Courville (2021)

$$-\sigma(\tau)\dot{\sigma}(\tau)\nabla_s \log[p_r(s|t)]d\tau, \quad (2.34)$$

e uma equação diferencial estocástica (em inglês, *Stochastic Differential Equation*, SDE) baseada em difusão de Langenvin com tempo variante de Song et al. (2021), respectivamente.

$$-\beta(\tau)\sigma(\tau)^2\nabla_s \log[p_r(s|t)]d\tau + \sqrt{2\beta(\tau)}\sigma(\tau)d\tilde{W}_\tau, \quad (2.35)$$

em que:

- $\tau \in [T, 0]$ é a variável tempo;
- $\sigma(\tau)$ é a função que controla o nível do ruído ao longo do tempo τ ;
- $\dot{\sigma}(\tau) = \frac{d\sigma(\tau)}{d\tau}$ é a função derivada de $\sigma(\tau)$ em relação ao tempo τ ;
- $\beta(\tau) = \frac{\dot{\sigma}(\tau)}{\sigma(\tau)}$ é o termo estocástico;
- $\nabla_s \log[p_r(s|t)]$ é a função *score* no tempo τ ;
- $\tilde{W}_\tau$ é o processo *backward* de Wiener para o tempo τ (WONG; HAJEK, 1985).

Além disso, a equação 2.36 define o redutor de ruído $K(s; t, \sigma)$ para a EDM (equação 2.33)

$$K(s; t, \sigma) = \left[\frac{0.2}{\sqrt{\sigma^2 + (0.2)^2}} \right]^2 s + \frac{0.2\sigma}{\sqrt{\sigma^2 + (0.2)^2}} \cdot V \left(\frac{s}{\sqrt{\sigma^2 + (0.2)^2}}; t, \frac{1}{4} \ln(\sigma) \right), \quad (2.36)$$

de forma que σ segue uma distribuição log-normal, ou seja, $\ln(\sigma) \sim \mathcal{N}(-1.2, 1.2^2)$, V é um *transformer* - proposto por Vaswani et al. (2017) - de três camadas e 0.2 é uma estimacão empírica feita pelos autores em relação ao desvio padrão dos vetores de estilo.

Para treinar redutor de ruído $K(s; t, \sigma)$, é proposta uma otimização de parâmetros do *transformer* V a partir da seguinte função de perda estabelecida na equação 2.37

$$\mathcal{L}_{edm} = \mathbb{E}_{x,t,\sigma,\mathcal{E}} [\lambda(\sigma) \|K(\mathbf{E}(x) + \sigma\mathcal{E}; t, \sigma) - \mathbf{E}(x)\|_2^2], \quad (2.37)$$

em que $\mathcal{E} \sim \mathcal{N}(0, I)$ é uma matriz de ruídos cujos valores segue, cada um, uma distribuição normal padrão. (σ) é o fator peso e E é o difusor de estilo, responsável por sintetizar um estilo $s = E(x)$ a partir de um áudio x .

Partindo de um vetor de estilo cheio de ruídos s , é possível encontrar um vetor de estilos s que contemple as características prosódicas da fala x por meio do processo de retirada de ruído, definido pela equação 2.38.

$$\frac{ds}{d\sigma} = \frac{s - K(s; t, \sigma)}{\sigma}, \quad s(\sigma(T)) \sim \mathcal{N}(0, [\sigma(T)]^2 I) \quad (2.38)$$

Por fim, os autores adotaram a solução proposta por Lu et al. (2022), uma solução de segunda ordem para modelos de difusão probabilísticos (em inglês, *Diffusion Probabilistic Model with second order solver*, DPM-2), a fim de resolver a equação 2.38. O resultado obtido por meio dessa escolha permitiu ao StyleTTS 2 gerar áudios que soassem naturais e bem expressivos apenas com 3 passos de difusão, isto é,

duas vezes mais rápido que um modelo edificado em autocodificadores variacionais condicionais (em inglês, *Conditional Variational Autoencoders*, CVAE), como VITS - proposto por Kim, Kong e Son (2021).

2.5.2 Arquitetura *end-to-end* e a duração do áudio sintetizado

Muitos sistemas de TTS operam em duas etapas: primeiro, geram uma representação intermediária, como um espectrograma de mel, e depois utilizam um *vocoder* para converter essa representação em uma forma de onda de áudio. Esse processo de duas fases pode introduzir artefatos e degradar a qualidade final, uma vez que o treinamento do método que gera os espectrogramas de mel e o *vocoder* ocorrem de forma separada.

O StyleTTS 2 se destaca por sua arquitetura de treinamento *end-to-end*, em que seus diferentes componentes são treinados em conjunto com o intuito de aprender a mapear diretamente o texto de entrada para o áudio de saída na forma de onda. O decodificador do modelo é treinado juntamente com o difusor de estilos e codificador acústico do texto para gerar o áudio diretamente a partir do vetor de estilo s e das representações fonéticas alinhadas.

Uma inovação crucial que viabiliza este treinamento é o modelamento de duração diferenciável não-paramétrico. O modelo aprende um alinhamento “suave” entre os fonemas do texto e os frames do áudio, permitindo que o gradiente flua através de todo o sistema durante o treinamento. Isso otimiza simultaneamente a pronúncia, a duração dos fonemas e a qualidade acústica, resultando em uma fala mais fluida e coesa.

A proposição matemática dos autores consiste em modelar a duração dos fonemas do texto $d_i \in 1, \dots, L$ e o alinhamento $a_i \in \mathbb{N}$ entre os fonemas e os frames do áudio como variáveis aleatórias.

A duração dos fonemas é limitada pelo hiperparâmetro L , o qual representa a duração máxima dos fonemas. Para $L = 50$, é estabelecido que a duração máxima de um fonema é de 1.25 segundos. Enquanto isso, para cada fonema t_j , a_i representa o alinhamento entre o frame i com o j -ésimo fonema.

Os autores do StyleTTS 2, Li et al. (2023), definem que o alinhamento a_i pode ser obtido a partir soma entre a duração do fonema t_i e a posição final do fonema anterior l_{i-1} (equação 2.39).

$$a_i = d_i + l_{i-1} \quad (2.39)$$

Supondo que d_i e l_{i-1} são independentes, logo a função massa de probabilidade (FMP) do alinhamento a_i , definida na equação 2.40, como o produto das FMP da

duração d_i e da posição final l_{i-1} dos fonemas, ou seja,

$$f_{a_i}[n] = f_{d_i}[n] + f_{l_{i-1}}[n] \quad (2.40)$$

Em seu estudo prévio, os autores adotaram a posição final l_i do fonema t_i como uma constante que representava a soma das durações desde o primeiro fonema t_1 até o i -ésimo fonema (LI; HAN; MESGARANI, 2023), isto é, $l_i = \sum_{k=1}^i d_k$. Essa abordagem não permitia à versão anterior do StyleTTS 2 realizar um treinamento *end-to-end*. A fim de solucionar esse entrave, a FMP da posição final l_i dos fonemas, $f_{l_{i-1}}[n]$ foi substituída por um kernel gaussiano com centro em l_i e hiperparâmetro $\sigma = 1.5$, apresentado na equação 2.41.

$$\mathcal{N}_{l_{i-1}}(n; 1.5) := \exp \left[-\frac{(n - l_{i-1})^2}{2(1.5)^2} \right] \quad (2.41)$$

Com isso, obtêm-se a FMP diferenciável do alinhamento a_i (equação 2.42).

$$f_{a_i}[n] = f_{d_i}[n] + \mathcal{N}_{l_{i-1}}(n; 1.5) \quad (2.42)$$

Em seguida, o preditor de duração $S(h_{BERT}, s)$ do StyleTTS 2 - responsável por ajustar a duração de todos fonemas de um texto de acordo com as características h_{BERT} do texto t e o estilo s gerado pelo difusor de estilos - tem como saída a probabilidade $q[k, i]$ de o fonema t_i ter uma duração mínima de $k \in 1, \dots, L$. Durante a fase de treinamento, $q[k, i] = 1$ quando a duração mínima k for menor ou igual à duração original de um fonema t_i , isto é, $k \leq d_{gt}$, $d_{gt} \in \mathbb{N}$. Além disso, os parâmetros do preditor de duração S são otimizados a partir da função de perda de entropia cruzada (equação 2.43)

$$\mathcal{L}_{ce} = \mathbb{E}_{d_{gt}} \left[\sum_{i=1}^N \sum_{k=1}^L -[\mathbb{I}(k \leq d_{gt}[i]) \cdot \log(q[k, i]) + [1 - \mathbb{I}(k \leq d_{gt}[i])] \cdot \log(q[k, i])] \right], \quad (2.43)$$

em que N é o número de fonemas presentes em um texto t e $\mathbb{I}(\cdot)$ é a função identidade.

Após o treinamento do modelo, a estimativa da duração de cada fonema t_i de um texto t é definida como a soma de todas as probabilidades $q[k, i]$, $\forall k \in \{1, \dots, L\}$ (equação 2.44)

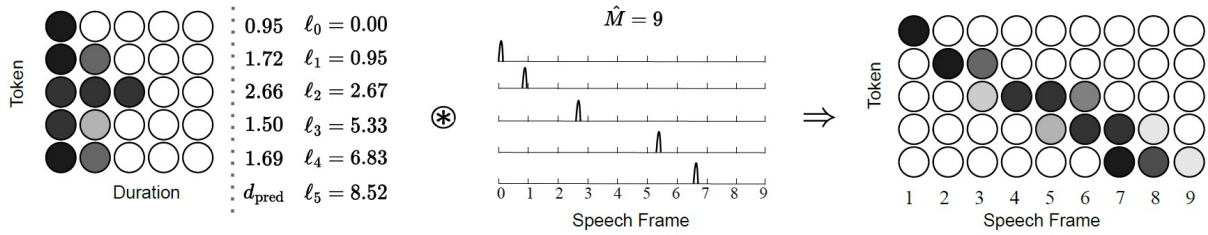
$$d_{pred}[i] = \sum_{k=1}^L q[k, i], \quad (2.44)$$

e estimativa da duração total da fala é dada por $\hat{M} = \lceil l_N \rceil$ e $l_N = \sum_{i=1}^N \sum_{k=1}^L q[k, i]$ é a posição final do último fonema do texto t .

Por fim, para cada tempo de duração $n \in \{1, \dots, \hat{M}\}$, a estimativa da FMP do alinhamento a_i é apresentada na equação 2.45

$$\tilde{f}_{a_i}[n] = \sum_{k=0}^{\hat{M}} q[n, i] \cdot \mathcal{N}_{l_i}(n - k; \sigma = 1.5) \quad (2.45)$$

Figura 18 – Estimativa da FMP do alinhamento a_i com 5 fonemas e $L = 5$



Fonte: Li et al. (2023).

e a estimativa do alinhamento a_i (figura 18) é obtida a partir da função de ativação *softmax*, aplicada na equação 2.46.

$$a_{pred}[n, i] = \frac{e^{\tilde{f}_{a_i}[n]}}{\sum_{i=1}^N e^{\tilde{f}_{a_i}[n]}} \quad (2.46)$$

2.5.3 Treinamento adversarial com SLMs

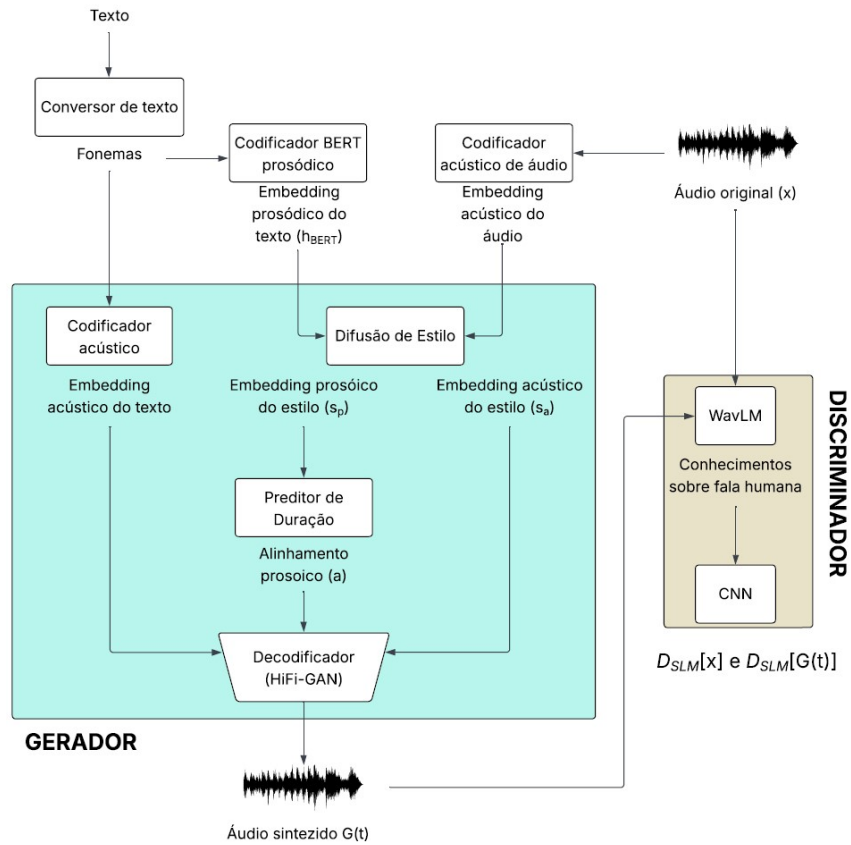
Para garantir que a fala sintetizada seja indistinguível da humana, o StyleTTS 2 emprega um treinamento adversarial no estilo das redes neurais adversariais generativas (em inglês, *Generative Adversarial Networks*, GANs). No entanto, em vez de usar um discriminador simples, ele utiliza o poder de grandes SLMs pré-treinados, como o WavLM ⁴, o qual foi treinado em vastas quantidades de dados de áudio não rotulados e, por causa disso, possui um vasto conhecimento sobre representações ricas e profundas da fala humana.

Na arquitetura do StyleTTS 2 (figura 19), o WavLM recebe tanto o áudio real quanto o áudio gerado e tenta diferenciar os dois. Como o SLM já possui um “entendimento” avançado sobre as características da fala humana, ele força o gerador - composto por um codificador acústico de texto, um modelo de difusão de estilos, um preditor de duração e um decodificador de características de texto e áudio para forma de onda - a produzir áudios que não apenas soem bem superficialmente, mas que também compartilhem as mesmas representações internas de alto nível da fala real.

⁴ Disponível em: <https://huggingface.co/microsoft/wavlm-base-plus>. Acesso em: 15 nov. 2025

Esse processo aprimora significativamente a naturalidade, a robustez e a qualidade perceptual da síntese.

Figura 19 – Arquitetura de treinamento do StyleTTS 2



Fonte: Adaptada de Li et al. (2023).

Além disso, o discriminador é o componente da arquitetura do StyleTTS 2 responsável por viabilizar, junto com o estimador diferenciável de áudios, a abordagem de treinamento *end-to-end* para o modelo. Sua é garantir que todas as partes do gerador G estejam aprendendo em conjunto e de forma correta.

Para encontrar os valores ótimos dos parâmetros dos componentes do gerador G e dos parâmetros do discriminador D_{SLM} , é necessário treinar o modelo com base na função de perda definida na equação 2.47

$$\mathcal{L}_{slm} = \min_G \max_{D_{SLM}} (\mathbb{E}_x [\log(D_{SLM}(x))] + \mathbb{E}[\log[1 - D_{SLM}(G(t))]]) , \quad (2.47)$$

em que $G(t)$ é o áudio de fala gerado pelo gerador G a partir do texto t e um áudio real de uma fala humana x .

A princípio, deve-se inicialmente treinar o discriminador D_{SLM} , cujos parâmetros são de uma CNN acoplada ao modelo WavLM. Para isso, os parâmetros do gerador G

são congelados. Em seguida, deve-se fornecer ao discriminador D_{SLM} uma gravação humana real x e um áudio ruidoso $G(t)$ do gerador G , calcular a perda $\mathcal{L}_{SLM}$ (equação 2.47) e atualizar os parâmetros pelo método do gradiente. Os parâmetros ótimos do discriminador D_{SLM} vão ser aqueles que identificam os áudios originais como verdadeiros, $D_{SLM}(x) \approx 1$, e os áudios sintetizados $G(t)$ pelo gerador como falsos, $D_{SLM}(G(t)) \approx 0$, garantindo a perda máxima na equação 2.47.

Na sequência, deve-se treinar o gerador G , isto é, treinar todos os parâmetros de seus componentes: o codificador acústico de texto, o modelo de difusão de estilos, o preditor de duração e o decodificador. Aqui, os parâmetros do discriminador D_{SLM} são congelados. Neste caso, o processo é o mesmo do treinamento dos parâmetros de D_{SLM} , contudo o método do gradiente será aplicado aos parâmetros de G . Por fim, os parâmetros ótimos do gerador G vão ser aqueles que identificam os áudios originais como falsos, $D_{SLM}(x) \approx 0$, e os áudios sintetizados $G(t)$ pelo gerador como verdadeiros, $D_{SLM}(G(t)) \approx 1$, garantindo, novamente, a perda mínima na equação 2.47.

Esse processo de oscilar entre os treinamentos do discriminador D_{SLM} e gerador G deve ocorrer diversas vezes até obter a convergência da função perda 2.47.

2.5.4 Desempenho do modelo após o treinamento

O StyleTTS 2 foi treinado em três conjuntos de dados diferentes: *LJ Speech* de Ito e Johnson (2017), *VCTK* de Yamagishi, Veaux e MacDonald (2019) e *LibriTTS* de Zen et al. (2019). O quadro 2 contém informações básicas de cada um deles e para qual propósito eles foram utilizados.

Quadro 2 – Informações sobre os *datasets* de treinamento

Dataset	Quantidade de áudios	Horas totais	Quantidade de locutores	Objetivo de treinamento
LJ Speech	13100	24	1	Avaliar qualidade e expressividade com áudio sintético de voz única
VCTK	44000	50	101	Avaliar qualidade e expressividade com áudio sintético para diversas vozes
LibriTTS	não informado	245	1151	Adaptação zero-shot ⁵

Fonte: Elaborada pelo próprio autor.

Para manter a uniformidade entre os três conjuntos de treinamento, todos tiveram seus textos convertidos para fonemas e seus áudios reamostrados com frequência de 24000 Hz .

O desempenho do modelo foi avaliado em duas categorias: a naturalidade e a similaridade. A primeira refere-se à capacidade de modelo produzir uma fala tão

⁵ Capacidade de replicar uma fala desconhecida pelo modelo (que não foi usada na fase de treinamento)

semelhante quanto a fala de uma pessoa. Já a segunda mede o quão próximo está o conteúdo da fala gerada do conteúdo presente na fala de referência.

A quantificação do desempenho foi feita por meio de uma pesquisa de opinião, em que os entrevistados deveriam atribuir uma nota que varia de 1 (fala não humana ou conteúdo completamente distinto do áudio referência) a 5 (é uma fala de uma pessoa ou o conteúdo é o mesmo do áudio referência) aos áudios originais, aqueles gerados pelo StyleTTS 2 e gerados por outros modelos (entre eles VITS, JETS, StyleTTS, ProDif, FastDiff e Vall-E). Por fim, os modelos foram comparados a partir de suas notas médias (MOS) de naturalidade e similaridade.

É esperado que a pontuação média de opinião (em inglês, *Mean Opinions Score*, MOS) dos áudios originais sejam sempre maiores que o dos áudios sintetizados por modelos TTS, esse, entretanto, não foi o caso para o StyleTTS 2 treinado a partir do *LJ Speech*. Durante a coleta de respostas, alguns ouvintes avaliaram os áudios do StyleTTS 2 como mais naturais que as próprias gravações originais, indicando que ele é capaz de produzir fala humana quando seu treinamento contém apenas dados de um único(a) locutor(a).

Em relação ao *dataset* VCTK, o modelo VITS era o que possuía as melhores avaliações de naturalidade e similaridade no campo de TTS. Ao realizarem a comparação de métricas entre seu modelo e o VITS, os autores concluíram que o StyleTTS 2 foi capaz de superá-lo tanto nesses dois quesitos quanto na velocidade de geração de áudios sintéticos (subseção 2.5.1).

Há um outro aspecto em que os modelos TTS são avaliados: a adaptação *zero-shot*. Conforme o quadro 2, é necessário avaliar o desempenho do modelo em relação ao *dataset* LibriTTS. O StyleTTS 2 foi capaz de superar todos os demais modelos nos dois aspectos, de forma que ele foi o único a atingir uma média maior que 4 para ambas as métricas. Destaca-se, também, a performance do StyleTTS 2 em comparação ao Vall-E de Wang et al. (2023), em que o primeiro foi capaz de superar o segundo utilizando apenas 245 horas de áudio contra 60000 horas de treinamento, ou seja, uma amostra de treinamento 250 vezes menor.

Em um último lugar, os autores avaliaram o impacto de cada componente da arquitetura do StyleTSS 2 na avaliação da naturalidade. Foi necessário treinar versões distintas do modelo em que o componente avaliado fora retirado da arquitetura principal.

A partir da tabela 2, nota-se que os três componentes inovadores do StyleTTS 2 apresentam impacto bastante significativo na avaliação da naturalidade do modelo, sendo o difusor de estilo o mais influente.

Tabela 2 – Impacto de cada componente na avaliação da naturalidade

Componente retirado	Impacto na naturalidade (MOS-N)
Difusor de estilo	-0.46
Preditor de duração	-0.21
Discriminador	-0.32

Fonte: Adaptado de Li et al. (2023).

Diante desses resultados, os autores concluíram que o StyleTTS 2 foi capaz de atingir um novo patamar entre os modelos de TTS. Ele é capaz de gerar e replicar, eficientemente, áudios que podem ser confundidos com falas humanas reais, ao utilizar uma menor quantidade de dados para treinamento e possuir o método mais rápido de sintetização de áudio que modelos TTS baseados na arquitetura de *transformers* e de difusão.

2.6 Trabalhos Correlatos

Nesta seção tem-se o levantamento de trabalhos relacionados ao envenenamento do áudio e a como tratar áudios como imagens.

2.6.1 Envenenamento de áudio

Atualmente, pode-se destacar os envenenamentos *VenoMave* de Aghakhani et al. (2023) e *WaveFuzz* de Ge et al. (2022) como o estado da arte em relação ao envenenamento de áudios de rótulos limpos. Ambos utilizam o conjunto de dados *Speech Commands* (WARDEN, 2018) para elaborar a *pipeline* e avaliar a eficiência de seus ataques.

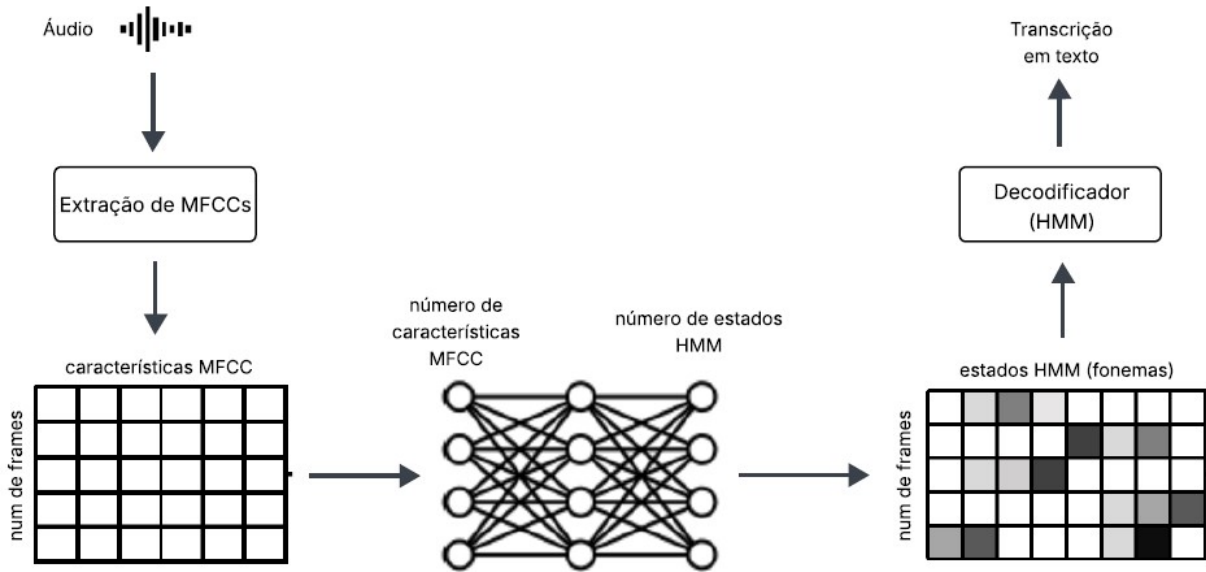
2.6.1.1 *VenoMave*

O *VenoMave* é um envenenamento que visa prejudicar o conhecimento dos sistemas híbridos de reconhecimento automático de fala (em inglês, *Automatic Speech Recognition*, ASR) - bastante utilizados em produtos como a Alexa da Amazon e o controle de voz da Sono - ao fazer com que uma palavra fonte específica de um áudio seja incorretamente transcrita como uma palavra alvo específica sem realizar troca de rótulos na amostra de treino.

Para gerar esse comportamento, o *VenoMave* emprega diferentes modelos substitutos $\mathcal{M}$ (originalmente conhecidos como *surrogate models*) como parte essencial do processo de geração transcrições (textos) envenenados a partir de um áudio limpo. Esses modelos são a junção de redes neurais profundas e modelos escondidos de Markov (em inglês, *Deep Neural Networks* e *Hidden Markov Models*, DNN-HMM,

respectivamente) treinadas pelo atacante sobre dados limpos e servem como aproximações do comportamento do modelo de reconhecimento de fala da vítima. A DNN é utilizada com o intuito de mapear as características do áudio na forma de coeficientes cepstrais de frequência mel (em inglês, *Mel Frequency Cepstral Coefficients*, MFCC) - transformação de áudio que sucede a transformação espectrograma de log-mel - para os fonemas definidos como estados no modelo de linguagem com arquitetura HMM. Em seguida, a HMM é responsável por traduzir a matriz de relação entre fonemas e frames (o mapa gerado pela DNN) em um texto (Figura 20).

Figura 20 – Pipeline de redes DNN-HMM



Fonte: Adaptado de Aghakhani et al. (2023).

Por meio de um ciclo iterativo de treinamento e refinamento, os modelos $\mathcal{M}$ permitem estimar como pequenas alterações em frames de áudio afetam as representações internas do modelo. Assim, o atacante pode otimizar, a partir do gradiente descendente, as amostras envenenadas para induzir erros direcionados, mesmo sem acesso direto à arquitetura ou aos parâmetros do sistema real. A equação 2.48 representa a função de perda da otimização adotada para o ataque $x^{(i)} = x_{\langle Y_i, Z_i \rangle}^{(i)}$.

$$\mathcal{L}_{\text{venom}} := \min_{x_{\gamma}^{(p)}} \frac{1}{2M} \sum_{m=1}^M \frac{\|\Phi^{(m)}(x^{(i)}) - \frac{1}{P} \sum_{p=1}^P \Phi^{(m)}(x_{\gamma}^{(p)})\|^2}{\|\Phi^{(m)}(x^{(i)})\|^2} \quad (2.48)$$

$\Phi^{(m)}(x^{(i)})$ é a saída desejada do modelo DNN, em que dado o frame do áudio original rotulado como Y_i , o modelo DNN irá classificá-lo como Z_i . Já, $\frac{1}{P} \sum_{p=1}^P \Phi^{(m)}(x_{\gamma}^{(p)})$ é o rótulo estimado pelos modelos substitutos $\mathcal{M}$, ou seja, a estimativa do ataque para os frames $x_{\gamma}^{(p)}$ selecionados para serem envenenados.

A avaliação do modelo no conjunto de dados *Speech Commands* foi feita por meio da proposição de quinze ataques. A taxa de sucesso do envenenamento foi de 73,33%, isto é, em 11 dos 15 ataques propostos, o modelo alvo realizou a transcrição equivocada do áudio original. Esse resultado foi obtido apenas aplicando o *VenoMave* em apenas 0,14% dos dados da amostra de treinamento, totalizando apenas 2 minutos de áudio.

Diante desses resultados, os autores foram capazes de elaborar um envenenamento de áudio em que o atacante é capaz de deteriorar o desempenho de quaisquer sistemas híbridos de ASR com apenas 2 minutos de áudios e, por conseguinte, demonstrar a vulnerabilidade desses modelos diante desse tipo de ataque.

2.6.1.2 WaveFuzz

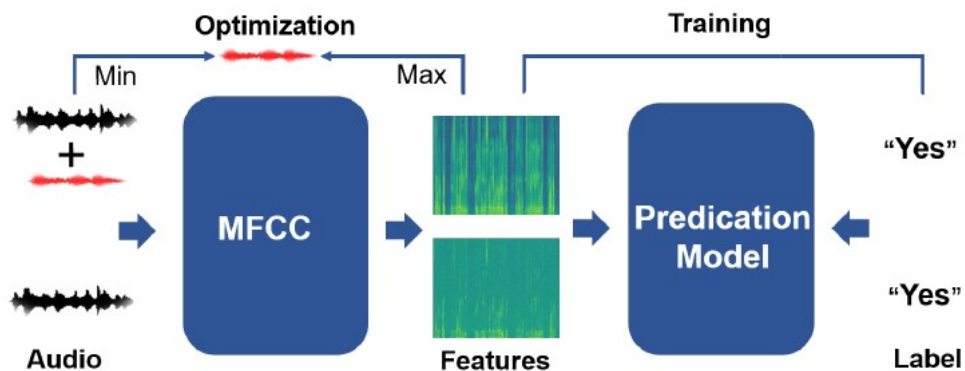
Segundo Ge et al. (2022), o *WaveFuzz* é um envenenamento de áudio que visa degradar a capacidade de os modelos baseados em atenção e CNN identificarem corretamente o comando expresso por um áudio ao inserir uma pequena quantidade de ruído diretamente na representação de onda de um áudio.

A estrutura do ataque (figura 21) consiste em gerar, aleatoriamente, uma onda ruidosa que será adicionada a um áudio que se deseja envenenar e por meio da comparação de MFCCs entre o áudio original e o áudio revestido pelo ruído, encontrar o menor valor ótimo para tornar esse ruído imperceptível. Em termos matemáticos, deve-se otimizar a função de perda dada pela equação 2.49

$$\mathcal{L}_{wave} = \arg \min_{\delta} \|\mathcal{MF}(x + \delta) - \mathcal{MF}(x)\|_2 + \alpha \cdot \|\delta\|_2, \quad (2.49)$$

em que x é a forma de onda do áudio a ser envenenado, $\mathcal{MF}(y)$ é a forma de MFCC de uma forma de onda y qualquer, $\|\cdot\|_2$ é a norma L_2 , α é o parâmetro que ajusta furtividade do ataque e o valor de δ é ajustado a partir do método do gradiente descendente.

Figura 21 – Pipeline do ataque *WaveFuzz*



Fonte: Ge et al. (2022).

A avaliação do modelo no conjunto de dados *Speech Commands* foi feita a partir de duas abordagens: uma em que os modelos baseados em atenção e CNN foram treinados do zero e outra em que foram treinados por meio do ajuste fino (em inglês, *fine-tuning*) com diferentes tamanhos de amostra de treinamento envenenada.

Na primeira abordagem, a perda de acurácia foi parecida para ambos os modelos com diferentes tamanhos de amostra. A acurácia do modelo baseado em atenção decresceu em 4,68% para uma amostra de treinamento em que apenas 0,5% dos dados estavam envenenados e, 7,89% com 10% dos dados envenenados. No modelo baseado em CNN, o decréscimo foi de 5,23% e 6,62% para o envenenamento de 0,5% e 10% dos dados, respectivamente. Portanto, aumentar a quantidade de dados envenenados na amostra de treinamento não aumentará o efeito do *WaveFuzz* sobre os modelos de atenção e CNN pelo método de treinamento do zero.

Já, na segunda abordagem, a perda de acurácia mostrou-se proporcional à quantidade de dados envenenados na amostra de treinamento. Para o modelo de atenção, as perdas de acurácia foram de 6,18% e 14,63% e para o modelo de CNN, de 5,75% e 14,37% com 0,5% e 10% dos dados envenenados, respectivamente. Logo, observa-se que o treinamento por ajuste fino é mais suscetível ao ataque *WaveFuzz* do que o treinamento do zero.

Por fim, os autores realizaram um estudo a respeito da qualidade dos áudios envenenados por meio de avaliação humana com vinte pessoas. A cada uma delas, foi apresentado um par de áudios, composto por um áudio envenenado e o mesmo áudio sem envenenamento, para que elas dissessem qual o comando expresso por cada um e a qual categoria eles pretendiam: “normal”, “com ruído, mas aceitável” e “extremamente ruidoso”. Os resultados obtidos foram: (i) o envenenamento *WaveFuzz* provoca uma redução média na acurácia dos modelos de 5,5% e (ii) 35% dos áudios adequirem um aspecto ruidoso após sofrerem o envenenamento.

Diante de todos esses resultados, os autores concluíram que as mensagens contidas nos áudios envenenados produzidos pelo *WaveFuzz* são preservadas, porém a qualidade desses áudios é um pouco comprometida pela adição de pequenas quantidades de ruídos e os modelos alimentados por esses mesmos áudios adquirem uma pequena chance de atribuir o rótulo errado aos áudios em avaliação.

2.6.2 Áudios como imagens

A detecção de violência em áudios tem sido explorada por meio de representações espectrográficas que capturam características temporo-frequenciais dos sinais acústicos. (BAKHSHI; SINGH; KUMAR, 2023) propuseram dois métodos de aprendizado profundo para classificar sinais de áudio contendo comportamentos violentos,

utilizando espectrogramas de mel como entrada para redes neurais profundas. Essa abordagem demonstrou eficácia na identificação de padrões acústicos associados a situações de violência, destacando a relevância dos espectrogramas de mel na análise de áudios em contextos de segurança pública.

Na área de detecção de *deepfakes* em áudios, diversas pesquisas têm investigado o uso de espectrogramas de mel para identificar manipulações sintéticas em sinais de voz. (PHAM et al., 2024) propuseram um sistema baseado em aprendizado profundo para a tarefa de detecção de deepfake em áudio, utilizando espectrogramas de mel como características de entrada para modelos de aprendizado profundo. Além disso, Alshehri et al. (2024) apresentaram o modelo *Sonic Sleuth*, que utiliza espectrogramas de mel como entrada para redes neurais convolucionais, alcançando alta precisão na detecção de áudios falsificados. Essas abordagens evidenciam a eficácia dos espectrogramas de mel na identificação de manipulações em sinais de áudio, contribuindo para o avanço das técnicas de detecção de *deepfakes*.

Ambas as linhas de pesquisa destacam a importância dos espectrogramas de mel como representações eficazes para análise de áudios em diferentes contextos. Seja na detecção de violência ou na identificação de *deepfakes*, o uso de espectrogramas de mel tem se mostrado uma estratégia promissora, permitindo a captura de informações temporo-frequenciais essenciais para a classificação e análise de sinais acústicos.

3 Metodologia

Neste capítulo, será apresentado o *dataset* a ser utilizado, o método proposto para o desenvolvimento deste projeto, detalhando quais foram as pequenas mudanças propostas no algoritmo do *Nightshade* que o permitissem envenenar áudios, e as métricas de avaliação do ataque adaptado.

3.1 *Dataset*

A proposta de adaptação do *Nightshade* para áudio visa fazer com que o modelo TTS escolhido para ser o modelo-alvo do ataque perca a capacidade de gerar certas palavras. Para este fim, emprega-se o *dataset Speech Commands v0.02* ¹.

Elaborado por Warden (2018), este conjunto de dados consiste em 105.829 enunciados de áudio de um segundo, cobrindo um vocabulário de 35 palavras em inglês, o que o torna ideal para a seleção de alvos específicos.

Desse modo, a imagem a ser envenenada e a imagem âncora são construídas como representações de espectrograma de log-mel da palavra-alvo e da palavra âncora. A escolha por este dataset é reforçada por seu uso em outros trabalhos da área, como nos ataques de envenenamento *VenoMave* e *WaveFuzz*.

3.2 Bibliotecas do Python

As bibliotecas *Python* utilizadas na adaptação do *Nightshade* são:

- ***Einops***: uma biblioteca que facilita operações de tensores;
- ***Librosa***: utilizada pa análise, processamento e extração de características de áudio;
- ***Numpy***: fornece suporte para arrays e matrizes multidimensionais de alta performance e um vasto conjunto de funções matemáticas para operar sobre eles;
- ***NLTK (Natural Language Toolkit)***: usada para tarefas como tokenização, *stemming* (radicalização), marcação de classe gramatical e análise de texto;

¹ Disponível em: <https://www.kaggle.com/datasets/mok0na/speech-commands-v002>. Acesso em: 15 nov. 2025

- **OS**: módulo nativo do Python para interagir com o sistema operacional, manipulando caminhos de arquivos, listando diretórios e navegando por pastas;
- **PIL (Python Imaging Library)**: biblioteca padrão para abertura, manipulação e salvamento de imagens em Python;
- **Pytorch** : um dos principais *frameworks* de aprendizado profundo, que utiliza tensores com forte aceleração por GPU para permitir a construção e o treinamento eficiente de redes neurais.

3.3 O algoritmo do *Nightshade* adaptado

Esta seção será dedicada à apresentação das modificações realizadas nas etapas 3, 5 e 6 do *Nightshade* (subseção 2.2.3) para que ele realize envenenamento do modelo StyleTTS 2 pré-treinado ² usando o conjunto de dados *Command Speeches*. Ademais, a adaptação foi realizada a partir do código disponível pelos autores em seu GitHub ³. Destaca-se, também, a troca do CLIP por um modelo CLAP pré-treinado ⁴. A figura 34 (apresentada no final da etapa 6) ilustra as diferenças entre o método do *Nightshade* original e a adaptação proposta por este trabalho.

A partir daqui, supõe-se que PE é a palavra escolhida para sofrer o envenenamento, PA é a palavra âncora e N_p é a quantidade de áudios referentes à PE que irão sofrer o envenenamento.

3.3.1 Etapa 3

O processo de seleção de candidatos ao envenenamento inicia-se com a identificação e coleta de todos os áudios referentes à palavra a ser envenenada PE , os quais formarão o conjunto inicial de dados.

Para realizar essa coleta de forma automatizada, utiliza-se a biblioteca *OS* do *Python*, conforme o código apresentado na figura 22. A função dessa ferramenta é vasculhar o diretório de dados específico da palavra PE , selecionando todos os arquivos no formato WAV e armazenando seus caminhos completos em uma lista, em que “<caminho_completo>/Audios/Command_Speech_v002/zero/<nome_arquivo>.wav” é o caminho de arquivo de quaisquer áudios no formato WAV que estejam contidos na pasta da palavra “zero”.

² Código e *checkpoints* da versão LibriTTS disponíveis em <https://github.com/yl4579/StyleTTS2>. Acesso em: 15 nov. 2025

³ Disponível em: <https://github.com/Shawn-Shan/nightshade-release> Acesso em: 15 nov. 2025

⁴ Disponível em: <https://github.com/LAION-AI/CLAP> Acesso em: 15 nov. 2025

Figura 22 – Função para armazenar todos os caminhos dos áudios referentes a *PE*

```
def get_all_audios(command_audios_path):
    audio_file_list = []
    for audio_file in os.listdir(command_audios_path):
        if audio_file.name.endswith(".wav"):
            audio_file_list.append(audio_file)
    return audio_file_list
```

Fonte: Elaborado pelo autor.

Segundo, é necessário realizar a remostragem de todos os áudios presentes na pasta da palavra *PE*, de forma que todos eles sejam convertidos de uma taxa de amostragem de 16000 *Hz* para 48000 *Hz* (taxa de amostragem exigida pelo modelo pré-treinado do CLAP). Isso pode ser realizado com a biblioteca *Librosa* por meio das funções *librosa.load()* e, na sequência, *librosa.resample()*. Após a conversão, os áudios serão denominados de $A48_i$.

Com os áudios $A48_i$ no formato correto, a próxima etapa prática é garantir que eles sejam, de fato, bons exemplos da classe *PE*. Para isso, o próprio modelo CLAP é empregado como um juiz de similaridade, em que a pontuação CLAP é calculada entre o prompt de texto “A person saying *PE*” e cada um dos áudios $A48_i$, estabelecido o seguinte limiar de qualidade: áudios que apresentarem uma pontuação CLAP ≤ 0.24 são considerados exemplos fracos ou atípicos da classe e, portanto, são descartados do processo.

Finalmente, a partir do conjunto de áudios filtrados e aprovados (aqueles com pontuação > 0.24), N_p candidatos de envenenamento são sorteados aleatoriamente, utilizando a função *random.sample()*. Os caminhos de arquivo desses áudios selecionados são salvos na lista *cand_list*, concluindo a preparação dos dados para a fase de envenenamento.

3.3.2 Etapa 5

Após a seleção dos áudios-alvo Aud_E , a próxima fase da metodologia consiste em gerar os áudios âncoras Aud_A correspondentes (figura 23). Cada áudio na *cand_list* (da palavra *PE*) deve ser pareado com um áudio recém-gerado da palavra *PA*. Este par $\{Aud_E, Aud_A\}$ é a unidade fundamental para o ataque de envenenamento.

Para esta tarefa, foi utilizado o modelo StyleTTS 2, que gera áudios com uma taxa de amostragem de 24000 *Hz*. No entanto, a simples geração de áudio a partir da palavra *PA* apresentou dois desafios técnicos significativos que exigiram adaptações: (i)

um relacionado à qualidade do áudio gerado Aud_A e (ii) outro relacionado à capacidade do modelo de processar os áudios-alvo Aud_E .

O primeiro desafio foi prático: observou-se que o StyleTTS 2 tende a interromper a pronúncia da última sílaba ao gerar áudio a partir de *strings* curtas. Para prevenir que os áudios âncoras PA sofressem esse corte abrupto, uma correção simples foi implementada. Um espaço em branco (" ") foi adicionado ao final da *string* de PA . Essa pequena modificação no input força o modelo a "completa" a fonética da palavra antes do silêncio, garantindo a integridade acústica de cada Aud_A gerado.

Figura 23 – Função gerar conceito âncora PA

```
def generate_target_audio(self, path_source_audio, tts_model,
                           source_concept, target, idx,
                           path_target_audio):

    target_concept = target + " "

    torch.manual_seed(123)
    with torch.no_grad():
        ref_s = tts_model.compute_style(path_source_audio)
        wav = tts_model.inference(target_concept,
                                   ref_s,
                                   alpha = 0.0,
                                   beta = 0.5,
                                   diffusion_steps = 5,
                                   embedding_scale = 1)

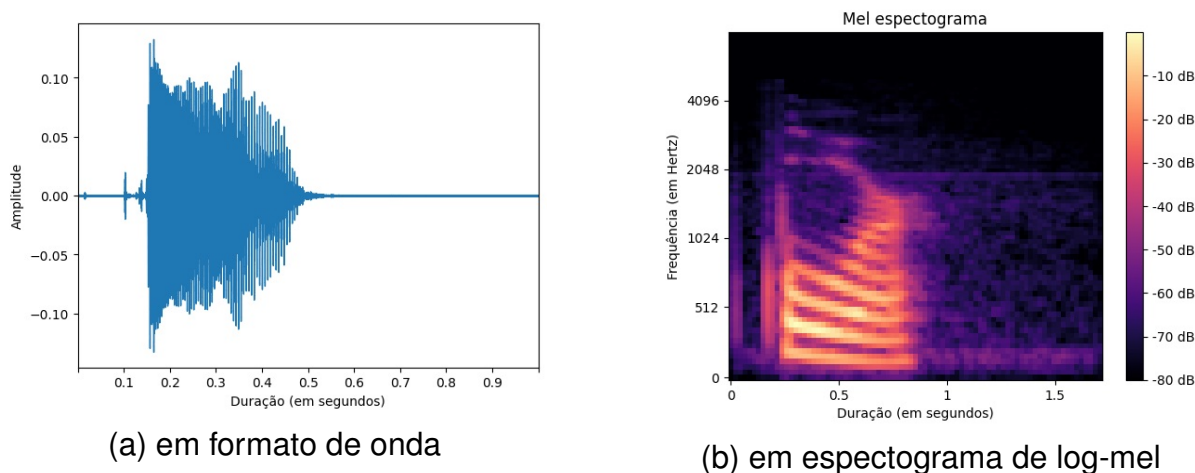
    file_name_target = path_target_audio + '{}/{}.wav'.format(idx)
    soundfile.write(file_name_target, wav, 24000)

    return file_name_target
```

Fonte: Elaborado pelo autor.

O segundo desafio foi uma incompatibilidade entre os dados de origem e a arquitetura do StyleTTS 2. Conforme documentado pelos autores do modelo (LI et al., 2023), o excelente desempenho do StyleTTS 2 na extração e transferência de estilo só é garantido quando o áudio de referência (neste caso, o Aud_E) possui, no mínimo, 3 segundos de duração.

Os áudios Aud_E do conjunto *Commands Speech* têm apenas 1 segundo. Para superar essa limitação, foi necessário modificar diretamente o código-fonte do modelo pré-treinado. Assim, a função `numpy.tile()` foi inserida no `compute_style()` (método do StyleTTS 2 responsável por analisar o áudio de referência). A função adicionada replica artificialmente o sinal do áudio de 1 segundo para criar um áudio de 4 segundos.

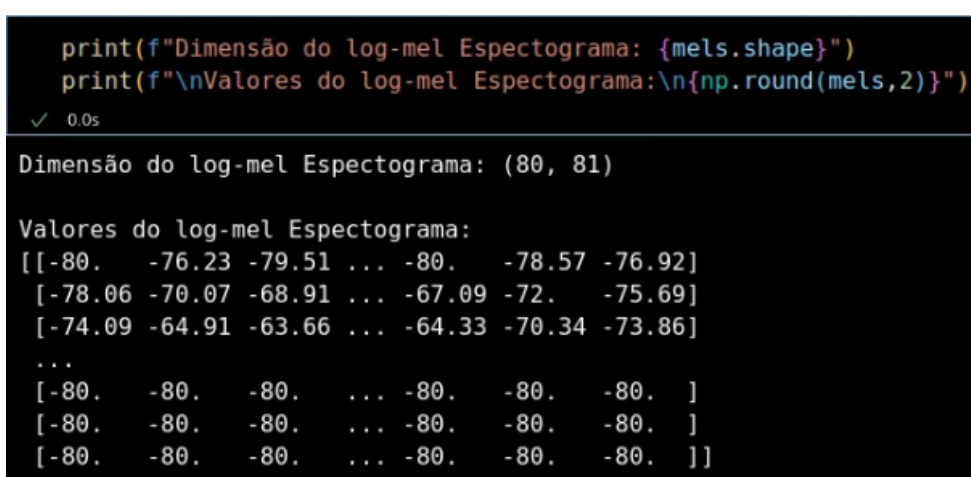
Figura 26 – Exemplo de áudio “four” do *Command Speechs*

Fonte: Elaborado pelo autor.

O *Nightshade* realiza o envenenamento em imagens de dimensão 512×512 , portanto é necessário encontrar uma maneira de representar o áudio em formato de uma imagem de mesma dimensão.

Não se pode utilizar o áudio no formato de onda, porque ele é um *array* de tamanho 24000, isto é, um elemento de uma única dimensão. Em contrapartida, a transformação obtida com os parâmetros definidos no código da figura 25 irá gerar espectrograma de log-mel na forma de matriz de dimensão 80×81 para qualquer áudio com duração de 1 segundo (figura 27).

Figura 27 – Dimensão e valores armazenados na matriz do log-mel espectrograma



Fonte: Elaborado pelo autor.

Apesar de o espectrograma de log-mel não atender aos requisitos de dimensão do *Nightshade*, ele ainda pode ser transformado em uma imagem na escala cinza segundo Donahue, McAuley e Puckette (2019). Como a matriz de espectrograma log-

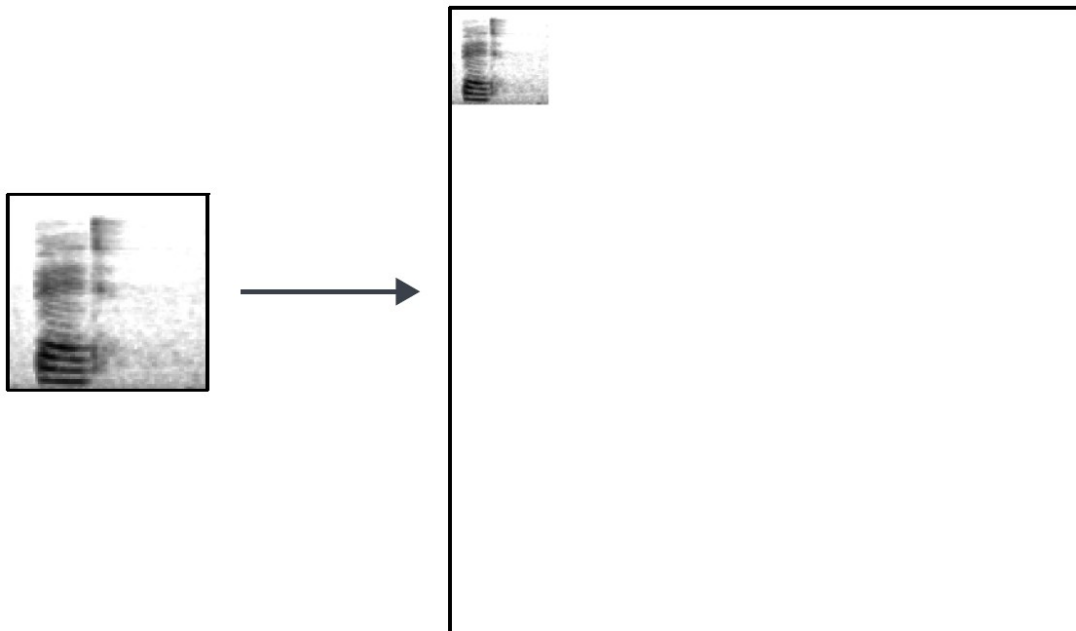
mel contém apenas valores reais x_{ij} que variam de 0 a -80 decibels, basta aplicar uma normalização do tipo Min-Max (equação 3.1)

$$y_{ij} = \frac{x_{ij} - \min(x_{ij})}{\max(x_{ij}) - \min(x_{ij})}, \quad (3.1)$$

para restringir os valores da matriz espectrograma de log-mel ao intervalo entre 0 e 1, $y_{ij} \in [0,1]$. A seguir, converter os dados do tipo *float* para o tipo *integer* e, na sequência, multiplicá-los por 255, de forma que se obtenha uma matriz com valores $p_{ij} = 255 \cdot y_{ij}$. Ao fim desse processo, o espectrograma de log-mel será uma imagem de dimensão 80×81 na escala cinza, pois valores entre 0 e 255 de uma matriz representam valores de pixels de uma imagem.

A partir daqui, tem-se uma representação de áudio em formato de imagem de dimensão 80×81 . Todavia, essa imagem não se adequa à resolução exigida pelo *Nightshade*, que deve ser 512×512 . Diante disso, foi necessário criar uma imagem de dimensão 512×512 totalmente branca (em que todos os pixels $b_{ij} = 255$) e, em seguida, inseriu-se os pixels da imagem de espectrograma de log-mel p_{ij} no canto superior esquerdo (figura 28). Essa nova imagem que contém o espectrograma de log-mel é denominada por M_{512} .

Figura 28 – Acoplando o espectrograma de log-mel de um áudio “bed” na imagem M_{512}



Fonte: Elaborado pelo autor.

A figura 29 contém o código em *Python* que descreve todo esse processo de conversão da imagem em escala cinza do espectrograma de log-mel para a imagem 512×512 adequada para o *Stable Diffusion*.

Figura 29 – Função que adiciona o espectrograma de log-mel a uma imagem 512 x 512 em branco

```
def converting_mel_to_img512(path_to_image, path_to_save, idx):

    mel_orig_img = Image.open(path_to_image) # Image Object (PIL library)

    black_img = np.zeros((512, 512), dtype = np.uint8) # pixels = 0
    white_matrix = Image.fromarray(255-black_img) # pixels = 255

    # Converting both Image Objects to numpy arrays
    mel_img_array = np.array(mel_orig_img)
    zero_img_array = np.array(white_matrix)

    mel_original_height = mel_img_array.shape[0]
    mel_original_width = mel_img_array.shape[1]

    # Putting the Mel_Spectrogram in the top-left
    # of the white 512x512 img
    zero_img_array[:mel_original_height,
    | | | :mel_original_width] = mel_img_array

    # Correct format input for Nightshade
    mel_512 = Image.fromarray(zero_img_array)

    # Saving the picture generated to disk:
    directory_to_save = path_to_save + '/{}.png'.format(idx)
    mel_512.save(directory_to_save)

    return directory_to_save
```

Fonte: Elaborado pelo autor.

Essa foi a abordagem escolhida, porque ela preserva todos os valores dos ruídos inseridos nos pixels envenenados ao redimensionar a imagem M_{512} envenenada (obtida ao final da otimização da equação 2.10) para seu seu tamanho original, 80×81 .

Como o envenenamento ocorre no espaço latente de imagens do Stable Diffusion 2.1, logo as imagens M_{512} deverão atender ao formato de dados exigidos pelo codificador embutido no modelo. Ele transforma tensores de 4 dimensões $[1, 3, h, w]$ cujos elementos estão contidos no intervalo $[-1, 1]$ em tensores de tamanho $[1, 512]$ com valores também limitados pelo mesmo intervalo.

A conversão direta da imagem M_{512} para tensor irá resultar em um elemento de dimensão $[h, w, 1]$ cujos valores estão contidos no intervalo $[0, 255]$. O valor 1 na terceira dimensão indica que o tensor está carregando informações de uma imagem em escala cinza, h é a largura da imagem e w , o comprimento. Porém, no tensor $[1, 3, h, w]$, o valor 3 na segunda dimensão indica que o tensor está carregando informações de uma imagem colorida na escala RGB. Logo, é necessário traduzir a imagem M_{512} da escala cinza para a escala RGB.

A biblioteca *PIL* do Python é capaz de realizar essa tarefa por meio do método *convert('RGB')*. Se a imagem M_{512} estiver em um arquivo PNG, ela será carregada no programa a partir do comando *Image.load("nome_arquivo.png").convert('RGB')* ou caso contrário, quando ela estiver na forma de *array*, a conversão ocorrerá pelo comando *Image.fromarray(M₅₁₂).convert('RGB')*. Assim, as dimensões do tensor de M_{512} serão atualizadas para $[h, w, 3]$.

Após a conversão da escala cinza para a escala RGB, deve-se garantir que os elementos da imagem M_{512} na escala RGB estejam contidos no intervalo $[-1, 1]$. Para isso, coloca-se as matrizes de pixels RGB de M_{512} no formato *array* a partir do comando *numpy.array()* da biblioteca *NumPy*, depois, dividi-se todos elementos por 127.5 e os subtrai por 1, conforme a equação 3.2.

$$M_{512_RGB[-1,1]} = \frac{M_{512_RGB[0,255]}}{127.5} - 1 \quad (3.2)$$

Pode-se efetuar o rearranjo das dimensões da imagem M_{512} na escala RGB por meio do método *rearrange* da biblioteca *Einops*, deslocando todas as dimensões para a direita e colocando a terceira dimensão (representada pelo valor 3) na primeira dimensão. Assim, as dimensões da matriz M_{512} na escala RGB serão $[3, h, w]$.

Por fim, é necessário converter a estrutura da matriz M_{512} de *array* para tensor de 4 dimensões e o tipo de dados de *float* de 32 bits para *float* de 16 bits. A biblioteca *PyTorch* oferece o método *tensor()* para conversão de *arrays* em tensores, o método *unsqueeze()* para aumentar o número de dimensões de um tensor e o método *half()* para transformar todos os dados em *float* de 16 bits. O tensor obtido nesse ponto será denominado de Te , sendo Te_E o tensor referente ao áudio que se deseja envenenar e Te_A o tensor referente ao áudio âncora.

Todas as transformações descritas até aqui estão descritas em forma de código na figura 30. Além disso, todas elas devem ser aplicadas em todos os áudios da lista de candidatos e em seus respectivos áudios âncoras. Em seguida, para cada par de tensores $\{Te_E, Te_A\}$, cria-se um tensor de zeros com as mesmas dimensões do Te_E . Esse tensor, simbolizado por δ , será utilizado para armazenar os valores ótimos dos ruídos de envenenamento.

Figura 30 – Código que converte imagem em escala cinza para o formato de *input* do *Stable Diffusion*

```
def img2tensor(cur_img):
    cur_img = np.array(cur_img)

    img = (cur_img / 127.5 - 1.0).astype(np.float32) # [0,255] to [-1,1]

    img = rearrange(img, 'h w c -> c h w')

    img = torch.tensor(img).unsqueeze(0) # shape [1,c,h,w]

    return img

def adequate_image_to_StableDiffusion_format(self, source_Mel512_path,
| | | | | | | | | | target_Mel512_path, device):
    # Loading source and target 512x512 mel-spectro
    source_mel_img_512 = Image.open(source_Mel512_path).convert('RGB')
    target_mel_img_512 = Image.open(target_Mel512_path).convert('RGB')

    source_tensor = img2tensor(source_mel_img_512).to(device)
    target_tensor = img2tensor(target_mel_img_512).to(device)

    source_tensor = source_tensor.half()
    target_tensor = target_tensor.half()

    return source_tensor, target_tensor
```

Fonte: Elaborado pelo autor.

Para realizar a etapa de otimização dos ruídos (código na figura 31), é necessário definir o número de interações j_{fim} , o valor inicial de acréscimo de ruído, cuja notação matemática é $step^{(0)}$ e os valores mínimo δ_{min} e máximo δ_{max} que os ruídos podem assumir. Foram utilizados os mesmos valores iniciais propostos por Shan et al. (2024), isto é, $j_{fim} = 500$, $step^{(0)} = 0.1$, $\delta_{min} = -0.1$ e $\delta_{max} = 0.1$. O código dessa etapa está representado pela figura 31.

Em cada interação j , calcula-se a quantidade de ruído $step^{(i)}$ a ser acrescida no *embedding* Emb_E pela equação 3.3

$$step^{(j)} = step^{(0)} - \left[\frac{(j-1)}{j_{fim}} \cdot 0.99 \cdot step^{(0)} \right] \quad (3.3)$$

Nesse ponto, cria-se tensor adversarial para armazenar a soma de δ e Te_E , isto é, $Te_{adv} = \delta + Te_E$, seguido pela extração dos *embeddings* Emb_{adv} do tensor Te_{adv} e Emb_A do tensor Te_A por meio do codificador do *Stable Diffusion*.

Obtidas as representações do áudio original, áudio âncora e do ruído no espaço latente do *Stable Diffusion*, aplica-se o método do gradiente descendente *torch.autograd.grad()* da biblioteca *PyTorch* para descobrir em quais pixels da imagem do espectrograma de log-mel os ruídos devem ser aplicados a fim de o áudio candidato obter características do áudio âncora a ele associado. A função perda $\mathcal{L}$ utilizada pelo gradiente é definida na equação 3.4.

$$\mathcal{L} = ||Emb_{adv} - Emb_A||_2 \quad (3.4)$$

Após isso, deve-se atualizar os valores dos ruídos $\delta^{(j+1)}$ ao informar a quantidade de ruídos $step^{(j)}$ a ser adicionada ou retirada em cada pixel. A equação 3.5 define, em termos matemáticos, o processo de atualização dos ruídos.

$$\delta^{(j+1)} = \delta^{(j)} + torch.sign(gradiente) \cdot step^{(j)}, \quad (3.5)$$

onde a função *torch.sign()* identifica em quais pixels a adição ou a remoção de ruídos deve ocorrer. Feita a atualização, deve-se truncar os valores do ruído no intervalo $[\delta_{min}, \delta_{max}]$. Por fim, repita esse processo até a interação $j = j_{max}$.

Com o fim da etapa de adição de ruído, obter-se-á o tensor Te_{adv} referente ao espectrograma de log-mel envenenado. Analogamente ao ruído, trunque seus valores para $[-1,1]$ para realizar o processo inverso de tensor para imagem (código disponível na figura 32).

Primeiro, converte-se os valores do tensor do intervalo $[-1,1]$ para $[0,1]$ com a transformação definida pela equação 3.6.

$$Te_{adv[0,1]} = \frac{Te_{adv[-1,1]} + 1}{2} \quad (3.6)$$

Utilize o método *rearrange* para efetuar a troca de dimensões de $[3, h, w]$ para as dimensões de uma imagem RGB $[h, w, 3]$. Em seguida, multiplica-se o tensor por 255 e converte-se seu tipo de variável para *unsigned integer* de 8 bits para se obter os valores dos pixels das matrizes RGB da imagem. Agora, utiliza-se a função *Image.fromarray.convert('L')* com a finalidade de transformar a imagem RGB em uma imagem em escala cinza de dimensões $[h, w]$. Essa é imagem de tamanho 512×512 que contém o espectrograma de log-mel envenenado na escala cinza, $M_{512(env)}$.

Figura 31 – Código referente à fase de otimização de ruídos

```

j_max = 500
noise_min_change = - 0.1
noise_max_change = 0.1
step_0 = 0.1

for j in range(j_max):

    step_j = step_0 - ( j/j_max * 0.99 * step_0 )

    modifier.requires_grad_(True) # noise
    adversarial_tensor = torch.clamp(modifier + tensor_source, -1, 1)

    with torch.no_grad():
        target_latent = sd_model.vae.encode(tensor_target).latent_dist.mean
        adv_latent = sd_model.vae.encode(adversarial_tensor).latent_dist.mean

    loss = (adv_latent - target_latent).norm()

    # Calculating the gradient
    grad = torch.autograd.grad(loss, modifier)[0]

    modifier = modifier + torch.sign(grad) * step_j # noise new values

    modifier = torch.clamp(modifier, # Truncate noise in [-0.1, 0.1]
                           min = noise_min_change,
                           max = noise_max_change)

```

Fonte: Elaborado pelo autor.

Figura 32 – Código referente à conversão do tensor $T_{e_{adv}}$ para a imagem $M_{512(adv)}$

```

def tensor2img(modifier):

    # rescalling values from [-1,1] to [0,1]
    cur_img = torch.clamp( (modifier.detach() + 1.0) / 2.0,
                           min = 0.0,
                           max = 1.0)

    # back to image shape and pixels values in [0,255]
    cur_img = 255. * rearrange(cur_img[0], 'c h w -> h w c').cpu().numpy()

    # convert image values to integers
    # and save image as Image Object
    cur_img = Image.fromarray(cur_img.astype(np.uint8))

    return cur_img

```

Fonte: Elaborado pelo autor.

A próxima etapa consiste na conversão da imagem $M_{512(env)}$ para sua representação em forma de onda envenenada, cujo código está ilustrado na figura 33.

O primeiro passo é o desacoplamento da imagem do espectrograma de log-mel envenenado $M_{cut(env)}$ da imagem $M_{512(env)}$ (o oposto do que ocorre na figura 28) e transformação dos valores de sua matriz para o tipo *float* de 32 bits.

No segundo passo, aplica-se uma normalização do tipo Min-Max (definida na equação 3.1) e multiplica-se esses valores no intervalo $[0,1]$ por -80 , transformações essas que resultam na verdadeira forma do espectrograma de log-mel envenenado.

No penúltimo passo, usam-se os métodos *librosa.db_to_power()* e o Griffin-Lim, cujo função em *Python* é *librosa.feature.inverse.mel_to_audio()*, para estimar o espectrograma de magnitude envenenado e, com ele, a forma de onda envenenada.

Por último, a onda envenenada deve ser transformada em um arquivo WAV com uma taxa de amostragem de 24000 Hz e este deve ser salvo em um diretório exclusivo para N os candidatos do ataque (PE , PA). A biblioteca *Soundfile* do *Python* salva os áudios a partir do método *soundfile.write()*.

Figura 33 – Código referente à conversão da imagem $M_{512(adv)}$ em seu respectivo áudio envenenado

```
def convert_poisoned_img512_to_audio(poison_img_512, h_orig_mel, w_orig_mel,
                                     path_to_save_poison_audio, orig_ref, idx):

    img512 = np.array(poison_img_512)

    mel_img_cut = img512[:h_orig_mel, :w_orig_mel] # cutting spectrogram

    mel_img = Image.fromarray(mel_img_cut).convert('L')

    mel_img_float32 = np.array(mel_img).astype(np.float32)

    img_to_mels_db = scale_minmax(mel_img_float32, 0, 80).astype(np.float32) # turning to mel scale
    mels_db = (-1) * img_to_mels_db # mels are negative numbers or zero

    mels_mag = librosa.db_to_power(mels_db) # Back to magnitude

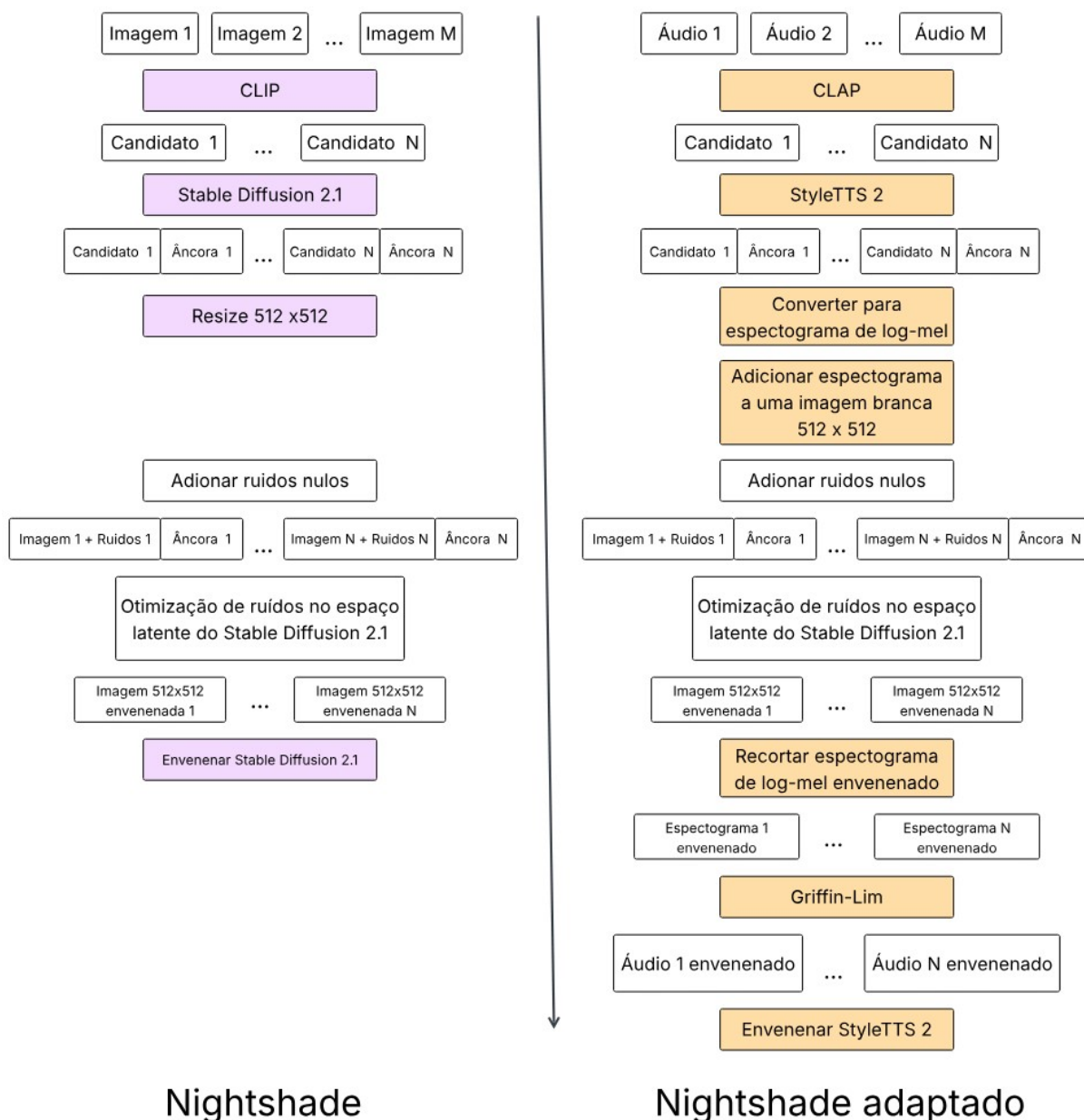
    waveform = librosa.feature.inverse.mel_to_audio( # Griffin-Lim method
        mels_mag,
        sr = 24000,
        n_fft = 2048, # old samples 1024
        hop_length = 300 # old samples 512
    )

    # save audio
    sf.write(path_to_save_poison_audio + "/" + "{}.wav".format(idx), waveform, 24000)
```

Fonte: Elaborado pelo autor.

A abordagem descrita na etapa 6 deve ser aplicada para todos os áudios listados na *cand_list()*. Dessa forma, ao final desta etapa, será obtida uma amostra de treinamento que conterà os N_p áudios candidatos envenenados destinados ao ataque (PE , PA) do modelo StyleTTS 2 na etapa 7.

Figura 34 – Diferenças entre o *Nightshade* original e o adaptado



Fonte: Elaborado pelo autor.

3.3.4 Etapa 7

O retreinamento do StyleTTS 2 ocorrerá por meio do *fine-tuning*. A escolha dessa técnica se baseia na ideia que ela é utilizada por atores mal-intencionados, mesmo com recursos computacionais limitados e poucos dados, com o propósito de adaptar modelos de código aberto para clonar vozes específicas em questão de horas e, assim, facilitar a propagação das *deepfakes*.

Antes de se realizar o *fine-tuning* do StyleTTS 2, é necessário separar os N áudios candidatos envenenados em amostras de treinamento e de teste. 10% deles são escolhidos aleatoriamente para compor a amostra de teste, enquanto os outros

90%, a amostra de treinamento envenenada.

Para cada uma dessas amostras é necessário criar um arquivo de texto que armazena: (i) caminhos de arquivo dos áudios que farão parte da amostra, (ii) a palavra *PE* em formato de fonema e (iii) um número inteiro distinto para cada áudio, tal que áudios com o mesmo falante devem compartilhar do mesmo número. A figura 35 mostra um exemplo de arquivo “*train_list.txt*” para o ataque (*no*, *yes*) e a figura 36 o código em Python para a conversão de texto em fonema.

Figura 35 – Arquivo *train_list.txt* para o ataque (*no*, *yes*)



```
84.wav | n'oʊ | 1201
59.wav | n'oʊ | 1202
167.wav | n'oʊ | 1203
186.wav | n'oʊ | 1204
187.wav | n'oʊ | 1205
205.wav | n'oʊ | 1206
153.wav | n'oʊ | 1207
284.wav | n'oʊ | 1208
208.wav | n'oʊ | 1209
61.wav | n'oʊ | 1210
88.wav | n'oʊ | 1211
12.wav | n'oʊ | 1212
295.wav | n'oʊ | 1213
136.wav | n'oʊ | 1214
```

Fonte: Elaborado pelo autor.

pontuação CLIP, basta utilizar a pontuação CLAP; já no caso da avaliação humana, manter-se-á a pontuação de 1 a 5 - denominada de MOS - e, ao invés de o áudio já vir acompanhado de uma descrição, a pessoa que o escutar deverá informar a transcrição ou dizer se o seu conteúdo é incompreensível.

Um par de palavra a ser envenenada e palavra âncora (PE, PA) simboliza um ataque de envenenamento direcionado. O desempenho de um ataque será avaliado ao se comparar a pontuação CLAP, o valor MOS e as transcrições obtidas em cada versão do modelo StyleTTS 2 (seja ela a original ou a envenenada). Cada um dos ataques estará atrelado a 3 frases de avaliação que contenham a palavra PE . Delas, criam-se 9 áudios tais que cada versão do StyleTTS 2 gere 3 deles, sendo 1 áudio por frase estipulada para avaliação.

O questionário coletará informações de todos os ataques estipulados por este trabalho. Todavia, ao invés de coletar respostas para as 3 frases de avaliação estipuladas para cada ataque - que totalizam $3 \cdot num_{modelos} \cdot num_{atqs}$ áudios para serem avaliados, onde $num_{modelos}$ é o número de modelos StyleTTS 2 em avaliação e num_{atqs} é a quantidade de ataques propostos - sortear-se-á uma das três frases para compor a coleta de informações acerca de cada ataque. Dessa forma, um único participante ficará encarregado de escutar e emitir suas opiniões sobre $num_{modelos} \cdot num_{atqs}$ áudios. Por fim, os resultados obtidos pelo questionário estarão resumidos em forma de tabelas, em que uma contenha tanto o valor MOS quanto a pontuação CLAP e outra, o número de transcrições corretas, erradas e incompreensíveis de cada modelo para cada ataque.

3.4.1 Pontuação CLAP

A pontuação CLAP de cada modelo é avaliada a partir de um único áudio que contém apenas a fala de uma pessoa dizendo PE em relação a dois textos: $text_1 = \text{"A person saying } PE\text{"}$ e $text_2 = \text{"A person saying } PA\text{"}$. O comando PE é o comando que se quer envenenar e PA é aquele que o substituirá. Defini-se a diferença CLAP (equação 3.7) como

$$Dif_{CLAP} = CLAP(PE, text_1) - CLAP(PE, text_2) , \quad (3.7)$$

tal que a diferença positiva implica que o áudio está mais próximo de PE e a diferença negativa, mais próximo de PA .

No modelo original, espera-se que a diferença CLAP deva ser positiva. Já, para os modelos envenenados, espera-se que o valor da diferença CLAP decresça, de forma que a diferença CLAP do modelo original seja a maior e a do modelo envenenado com uma amostra envenenada de tamanho 300, a menor. Se isso ocorrer, há um indicativo

de que o ataque de envenenamento (PE , PA) foi bem-sucedido, isto é, o áudio gerado pelo modelo envenenado trocou a palavra PE por PA .

3.4.2 Valor MOS acerca da qualidade do áudio

O valor MOS de determinado modelo é média das notas MOS obtidas em todos ataques efetuados nele. Se um modelo é avaliado ao longo de num_{atqs} ataques, tal que cada um possui num_{frases} frases de avaliação, logo sua MOS é média da MOS dessas $num_{atqs} \cdot num_{frases}$ frases.

Esse valor é utilizado para medir a furtividade de um ataque a um modelo TTS mediante a audição humana (CHEN et al., 2021). Quanto maior a furtividade, maior será o tempo que uma pessoa leva para perceber que seu modelo está envenenado. Se o valor da MOS é igual para os três modelos ou se existe um pequeno decrescimento da MOS conforme o tamanho da amostra de envenenamento cresce, há, portanto, indicativos de um ataque furtivo. Entretanto, se o valor da MOS vai decrescendo drasticamente na medida em que o tamanho da amostra envenenada aumenta, constata-se que o método de envenenamento proposto produzirá apenas áudios envenenados de qualidade inferior a suas respectivas versões originais (antes do envenenamento) e, como consequência direta, o modelo StyleTTS 2 perderá sua capacidade de replicar falas humanas com alta fidelidade.

3.4.3 Verificação da troca de palavras

Para verificar se houve troca das palavras PE por PA , isto é, um envenenamento direcionado, cada participante do questionário deverá transcrever a frase dita nos $n_{modelos}$ áudios presentes na página de cada ataque. Finalizado o período de coleta de respostas, as transcrições de cada ataque serão classificadas em três grupos: as transcrições corretas (A), transcrições erradas (E) e conteúdo do áudio incompreensível (I).

Se a quantidade de transcrições corretas for maior que a soma das transcrições erradas e do conteúdo do áudio incompreensível, isto é, $A > E + I$, então haverá um indicativo de que a troca da palavra PE por PA não ocorreu e, portanto, o envenenamento não é direcionado.

Agora, para o caso em que a quantidade de acertos é menor ou igual, $A \leq E + I$, somente haverá um indicativo de troca de palavra se $E > I$ e todas as transcrições coletadas realizarem única e exclusivamente a troca da palavra PE por PA , caso contrário, para $E < I$, haverá um indicativo de que o ataque resultou em um modelo StyleTTS 2 envenenado capaz de produzir apenas áudios incompreensíveis.

3.4.4 Questionário online e coleta de informações

A solução tecnológica para a aplicação do questionário foi estruturada em três camadas distintas.

- **Front-end:** responsável pela visualização e interação do usuário. A interface de visualização e interação com o usuário foi desenvolvida com o *framework Next.js*, garantindo uma aplicação dinâmica e performática.
- **Back-end:** desenvolvido em *Spring Boot*, serviu como o núcleo do sistema, expondo uma API para receber, validar e processar os dados enviados pelos participantes do questionário.
- **Banco de Dados:** para o armazenamento e a persistência das informações, optou-se pelo *MongoDB*, um banco de dados *NoSQL* que ofereceu flexibilidade na gestão das respostas.
- **Hospedagem:** a hospedagem do site do formulário e das respostas coletadas foram realizada na plataforma *Google Cloud Run*.

Todas as ferramentas escolhidas para a elaboração e hospedagem do formulário são gratuitas. Além disso, essa abordagem foi adotada porque ela, diferentemente do *Google Forms*, é capaz de permitir que áudios sejam executados em tempo real em uma página *web*.

O questionário foi divulgado para toda a comunidade da Unesp (isto é, em todos os email institucionais “@unesp.br”), por meio de um email de divulgação emitido pela Acessoria da Faculdades de Ciência (figura 37). O e-mail continha uma descrição breve sobre o conteúdo do questionário e o site para respondê-lo.

Figura 37 – Email de divulgação do questionário encaminhado pela Acessoria de FC



Fonte: Elaborado pelo autor.

3.4.4.1 Interface do questionário

A interface do questionário (ilustrada na figura 38) é organizada de forma simples e funcional, com a seguinte estrutura:

- **Título:** No topo, está o título "Pesquisa de Áudio", indicando que o objetivo do questionário é avaliar ou identificar o conteúdo de áudios;
- **Áudios:** A interface apresenta três áudios (Áudio 1, Áudio 2, Áudio 3) que podem ser ouvidos. Cada áudio possui um botão de *play*/pause, e um contador de tempo mostrando a duração de cada um. Esses áudios são referentes a uma frase de avaliação sortida ao acaso de um ataque (PE, PA) - em que cada ataque tem 3 frases de avaliação.
- **Campos para respostas:** Abaixo dos áudios, há uma caixa de texto para cada áudio, sendo a primeira para o Áudio 1, a segunda para o Áudio 2 e a terceira para o Áudio 3. Nelas, o participante deve escrever o que foi nos três áudios, colocando sua resposta na caixa referente ao áudio que escutou. Caso o entrevistado não compreenda aquilo que foi dito no áudio, ele deve ignorar a caixa de texto e, ao lado dela, marcar a opção "Incompreensível".
- **Avaliação de qualidade:** Após as respostas para cada áudio, o questionário solicita que o usuário avalie a qualidade dos áudios numa escala de 1 a 5, onde 1 representa uma qualidade muito baixa e 5, uma qualidade muito alta. Isso é feito por meio de campos de seleção com opções de avaliação.
- **Botões de navegação:** Ao final, há um botão para enviar as respostas referentes ao ataque (PE, PA) sortido ao acaso e passar para a próxima página do questionário (que representará outro ataque (PE, PA)).

Figura 38 – Interface do questionário utilizado na avaliação humana

Pesquisa de Áudio

Formulário 1 de 9

Audio 1

Audio 2

Audio 3

0:00 / 0:02

0:00 / 0:03

0:00 / 0:04

O que é dito em cada um dos áudios?

Audio 1

Escreva o que é dito em cada áudio

☐ Incompreensível

Audio 2

Escreva o que é dito em cada áudio

☐ Incompreensível

Audio 3

Escreva o que é dito em cada áudio

☐ Incompreensível

De 1 a 5, como você avalia a qualidade dos audios acima?

Audio 1

1

2

3

4

5

Audio 2

1

2

3

4

5

Audio 3

1

2

3

4

5

Enviar e próximo

Fonte: Elaborado pelo autor.

4 Experimentos

O capítulo 4 tem como objetivo apresentar e descrever os resultados obtidos ao realizar a adaptação de envenenamento proposto no Capítulo 3. Logo, serão apresentados as configurações da máquina utilizada para efetuar os experimentos, quais ataques foram ou não viáveis, como ficam os áudios assim que eles sofrem envenenamento, quais são os efeitos de realizar o fine-tuning do modelo StyleTTS 2 com áudios envenenados e, por último, as considerações finais.

4.1 Ambiente Experimental

Os componentes de *software* e *hardware* necessários para a execução deste projeto são os seguintes:

1) *Software*

- Sistema Operacional: Ubuntu 24.04 LTS 64 bits (*desktop*);
- Ambiente de Desenvolvimento: Microsoft Visual Studio Code;
- Linguagem de Programação: Python versão 3.12;

2) *Hardware*

- Processador: Intel Core i7-12700F de 12^a geração;
- GPU: GeForce RTX 3060 Lite Hash Rate (12 GB VRAM);
- Memória RAM: 64 GB DDR4;
- Armazenamento: 2 TB HDD.

4.2 Ataques realizados

Este trabalho propôs a realização de, pelo menos, um ataque para todas as categorias presentes no conjunto de dados *Commands Speech*. O plano de avaliação para cada ataque consistia em usar duas amostras de treinamento: uma com $N_p = 100$ áudios envenenados (1 minuto e 40 segundos) e outra, maior, com $N_p = 300$ áudios envenenados (5 minutos). A escolha desses tamanhos de amostra se justifica por replicar os valores utilizados no estudo *Nightshade* original e porque os 5 minutos representam um cenário de contaminação mais realista e perigoso para a fase de

fine-tuning, simulando o que um atacante conseguiria realisticamente injetar através de métodos como *web scraping* ou contribuição maliciosa.

Contudo, durante a fase de preparação dos dados e antes que o *fine-tuning* do StyleTTS 2 pudesse ser executado para cada ataque planejado, observou-se que algumas categorias apresentaram inconsistências. Especificamente, ocorreram falhas na etapa 3 (seleção de candidatos a serem envenenados) ou na etapa 5 (geração dos áudios âncoras pelo StyleTTS 2) do algoritmo adaptado (retomar seção 3.3). Diante dessa limitação prática, foi necessário separar os ataques em dois grupos: os não viáveis e os viáveis para a realização efetiva do *fine-tuning* com o StyleTTS 2.

4.2.1 Ataques não viáveis

4.2.1.1 Número insuficiente de candidatos

Os ataques que apresentaram inconsistência na etapa 3 do algoritmo do *Nightshade* adaptado são os ataques não viáveis que não foram capazes de encontrar candidatos o suficiente para compor as amostras de treinamentos envenenadas de tamanho $N_p = 100$ e $N_p = 300$.

Além disso, na etapa 3, ainda não se tem informação sobre a palavra âncora *PA* que irá substituir aquela que desejamos envenenar, portanto quaisquer ataques que desejam envenenar as palavras presentes na tabela 3 serão enquadrados como não viáveis.

Tabela 3 – Comandos que apresentaram problema na etapa 3

Categoria do áudio	Número de áudios no <i>dataset</i>	Candidatos encontrados
<i>Bird</i>	2064	14
<i>Dog</i>	2128	80
<i>Follow</i>	1579	58
<i>Forward</i>	1557	47
<i>House</i>	2113	158
<i>Left</i>	3801	87
<i>Off</i>	3745	64
<i>One</i>	3890	152
<i>Right</i>	3778	82
<i>Up</i>	3723	19
<i>Visual</i>	1592	32

Fonte: Elaborada pelo autor.

A tabela 3 contempla todos as categorias do *Command Speeches* que apresentam esse comportamento, com a quantidade total de áudios de cada categoria no *dataset* e suas respectivas quantidades de candidatos encontrados.

Segundo Wu* et al. (2023), esse problema ocorre devido ao péssimo desempenho do modelo CLAP em dados relacionados à fala humana. É importante lembrar que os candidatos são adicionados à amostra de envenenamento quando suas pontuações CLAP são maiores do que 0.24.

Esse valor adotado para a seleção de candidatos já é um valor pequeno para avaliar a proximidade entre um áudio um texto, ainda mais em uma pontuação que varia entre 0 e 1. Nota-se que, para uma pontuação baixa como essa, 11 das 35 categorias (aproximadamente 31%) tem 90% de seus áudios descartados pelo CLAP. Diante dessa enorme perda de informação, esse trabalho também corrobora com a avaliação dos autores do CLAP sobre o baixo desempenho do modelo para o contexto de fala humana.

4.2.1.2 Impossibilidade de gerar áudio âncora

Os ataques que apresentam inconsistência na etapa 5 do algoritmo do *Nightshade* adaptado são os ataques não viáveis que não foram completos porque a lista de candidatos a serem envenenados continha um par de áudios (PE , PA) para o qual o StyleTTS 2 não era capaz de gerar áudio âncora.

Essa incapacidade ocorre pois os *embeddings* gerados pelo preditor de duração e pelo codificador acústico texto do StyleTTS 2 (olhar arquitetura do modelo na Figura 19) possuem tamanhos diferentes. Antes de esses *embeddings* se tornarem *inputs* do decodificador do modelo, o preditor de duração calcula qual deve ser a dimensão dos *embeddings* o decodificador deve receber referente a duração do áudio e à acústica do texto. Caso o codificador acústico do texto gere um *embedding* de tamanho diferente do que é requisitado pelo decodificador, então o algoritmo de geração de áudio âncora PA é interrompido e, em seguida, divulga-se uma mensagem de erro com o tamanho do input que o decodificador deve receber e o tamanho do *embedding* gerado pelo codificador acústico de texto.

Por exemplo, para o ataque em que se deseja envenenar a palavra $PE = \text{"go"}$ de forma a modificá-la para $PA = \text{"yes"}$, o preditor de duração estima que a dimensão do *embedding* que o decodificador deva receber seja 5×4 . Já, o codificador acústico do texto fornece ao decodificador um *embedding* de dimensão 5×5 . Isso interrompe a execução do algoritmo do *Nightshade* adaptado com a mensagem de erro apresentada pela figura 39.

Figura 39 – Problema de *padding* no envenenamento de “go” para “yes”

```
RuntimeError: Calculated padded input size per channel: (5 x 4).
Kernel size: (5 x 5). Kernel size can't be greater than actual input size
```

Fonte: Elaborado pelo autor.

A tabela 4 contém os ataques que não foram finalizados devido a impossibilidade de se gerar o áudio âncora. Ademais, ela fornece a dimensão do *embedding* calculado pelo preditor de duração, a dimensão do *embedding* gerado pelo codificador acústico de texto e o número de pares (PE , PA) gerados pelo StyleTTS 2 até o erro ocorrer.

Tabela 4 – Ataques que apresentaram problema na etapa 5

Ataques		Preditor de	Codificador	Número de pares (PE, PA) até erro
PE	PA	duração	acústico de texto	
<i>Cat</i>	<i>Dog</i>	(5x4)	(5x5)	147
<i>Eight</i>	<i>Paint</i>	(5x4)	(5x5)	73
<i>Go</i>	<i>Yes</i>	(5x4)	(5x5)	72
<i>Happy</i>	<i>Tree</i>	(5x3)	(5x5)	135
<i>Six</i>	<i>Seven</i>	(5x4)	(5x5)	92
<i>Two</i>	<i>Up</i>	(5x4)	(5x5)	157
<i>Yes</i>	<i>Off</i>	(5x4)	(5x5)	15
<i>Zero</i>	<i>Stop</i>	(5x3)	(5x5)	178

Fonte: Elaborada pelo autor.

Neste experimento, o envenenamento está restrito apenas à uma palavra e não uma sentença com duas ou mais palavras, logo a geração de áudio fica limitada à geração de áudios curtos com menos de 3 segundos. Novamente, é importante reforçar que segundo os autores do StyleTTS 2, seu modelo não possui garantia de bom desempenho para gerar áudios com menos de 3 segundos. Assim, os ataques apresentados na tabela 4 corroboram com essa conclusão.

4.2.2 Ataques viáveis

Os ataques viáveis são aqueles capazes de gerar as amostras de envenenamento de tamanhos $N_p = 100$ e $N_p = 300$, sem apresentar problemas ao longo das etapas de 1 até a 6 do algoritmo do *Nightshade* adaptado.

A tabela 5 apresenta todos os ataques viáveis obtidos ao aplicar envenenamento do *Nightshade* adaptado até a etapa 6 (fase que coloca ruídos nos áudios) e seus respectivos valores de perda média durante a otimização de ruídos.

Tabela 5 – Ataques viáveis

Ataques		Perda média		
PE	PA	Inicial	Fim da interação	Diferença
<i>Bed</i>	<i>Cat</i>	127.85	76.03	51.82
<i>Down</i>	<i>Right</i>	128.82	80.22	48.6
<i>Four</i>	<i>Five</i>	128.37	82.47	45.91
<i>Four</i>	<i>Happy</i>	121.26	68.02	53.23
<i>Nine</i>	<i>Five</i>	134.54	84.63	49.91
<i>Nine</i>	<i>Tiny</i>	126.24	70.98	55.26
<i>No</i>	<i>Yes</i>	117.4	69.19	48.21
<i>Stop</i>	<i>Go</i>	127	78.05	48.95
<i>Tree</i>	<i>Destroy</i>	117.12	60.21	56.91

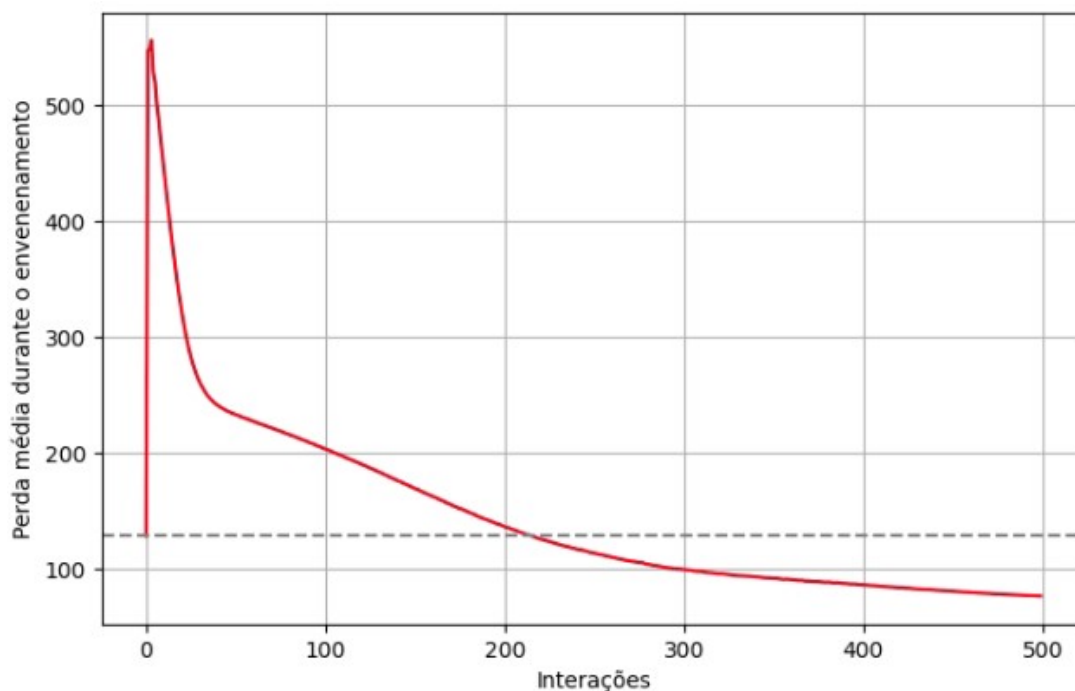
Fonte: Elaborada pelo autor.

O tempo médio para se envenenar apenas um áudio de 1 segundo e, em seguida, convertê-lo para um áudio envenenado utilizando o método proposto é de 2 minutos e 30 segundos. Já, para se gerar as amostras de treinamento envenenadas de tamanho $N_p = 100$ e $N_p = 300$, o tempo médio do envenenamento é de 3 horas e 20 minutos e 10 horas, respectivamente.

Os modelos de TTS, em sua maioria, são treinados a partir de amostras de treinamento com horas de informações. Dado que a amostra de tamanho 300 corresponde a 300 segundos de áudio e 1 hora de áudio corresponde a 3600 segundos, portanto seriam necessários, pelo menos, 5 dias de execução para se gerar 1 hora de informações envenenadas. Assim, é possível apontar que a adaptação proposta possui um tempo de execução consideravelmente elevado para a realização do envenenamento.

Esse tempo de execução está atrelado diretamente ao método de otimização de ruídos (subseção 3.3.3) com 500 interações. Pela figura 40, observa-se a perda média atrelada ao processo de adicionar e retirar ruído por difusão de forma que os áudios “*bed*” fiquem cada vez mais parecidos com o seus áudios âncoras “*cat*”. Essa aproximação começa a ocorrer, de fato, após a interação 200, porque são gerados ruídos até mesmo nos espaços em branco das imagens fornecidas ao *Stable Diffusion* (ilustradas na figura 28).

Figura 40 – Perda média obtida durante a fase de adição de ruídos no ataque (*bed*, *cat*)



Fonte: Elaborado pelo autor.

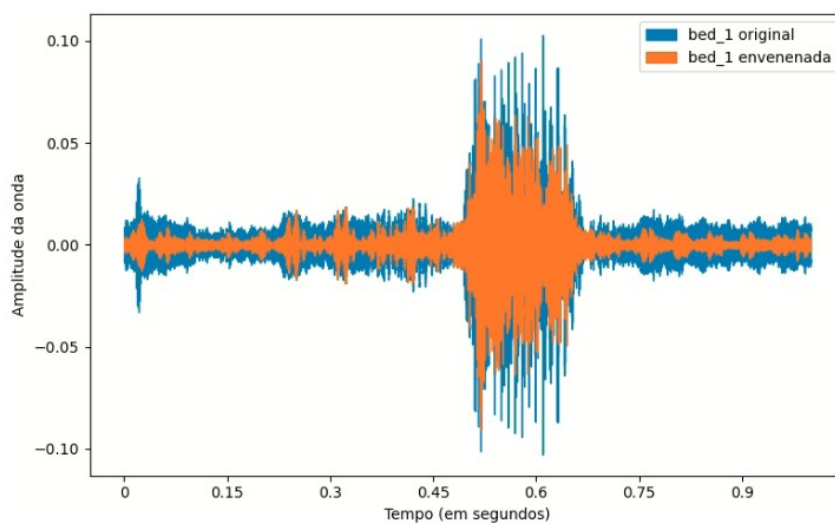
Esse comportamento da perda média durante a fase de otimização de ruídos também ocorre para todos os demais ataques. Para mostrar que a otimização de ruídos varia de ataque para ataque, os valores de perda média da primeira e última interação são apresentados na tabela 5.

As figuras 41 e 42 mostram a comparação entre os áudios originais e os obtidos ao aplicar o envenenamento *Nightshade* adaptado para três candidatos.

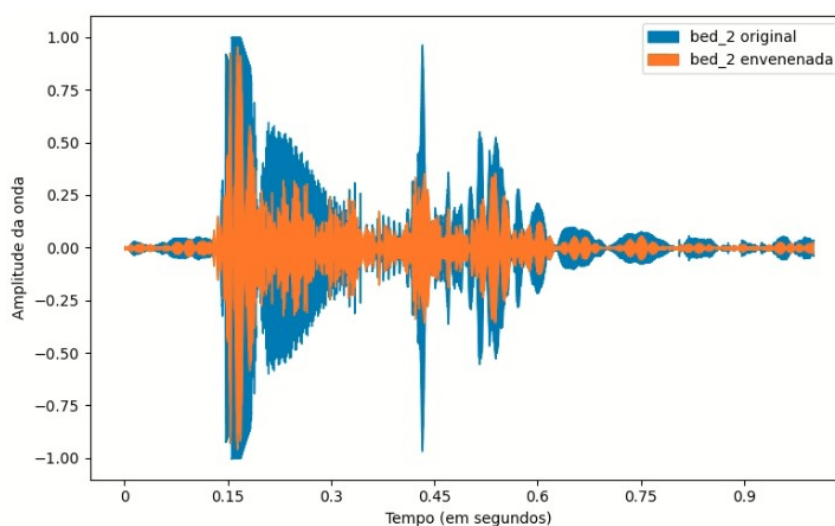
A falta de semelhança entre os áudios envenenados e o original, visível em ambas as figuras, sugere que a adaptação proposta produza áudios envenenados de qualidade inferior à de suas respectivas versões originais. A confirmação dessa queda na qualidade, no entanto, ocorrerá por meio da análise do valor MOS na seção 4.4.

Outro ponto que deve ser enfatizado é que ataques que almejam envenenar a mesma palavra *PE* com diferentes palavras âncoras *PA* geram áudios envenenados diferentes. A figura 42 retrata isso perfeitamente para os candidatos 0, 1 e 2 do ataque (*four*, *five*) e os candidatos 5, 88 e 258 do ataque (*four*, *happy*).

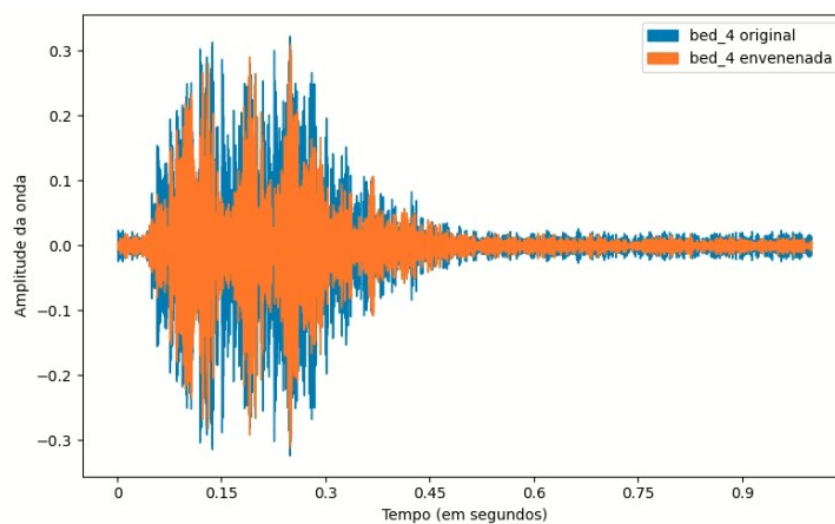
Figura 41 – Comparando as ondas originais e as envenenadas do áudio “bed”



(a)



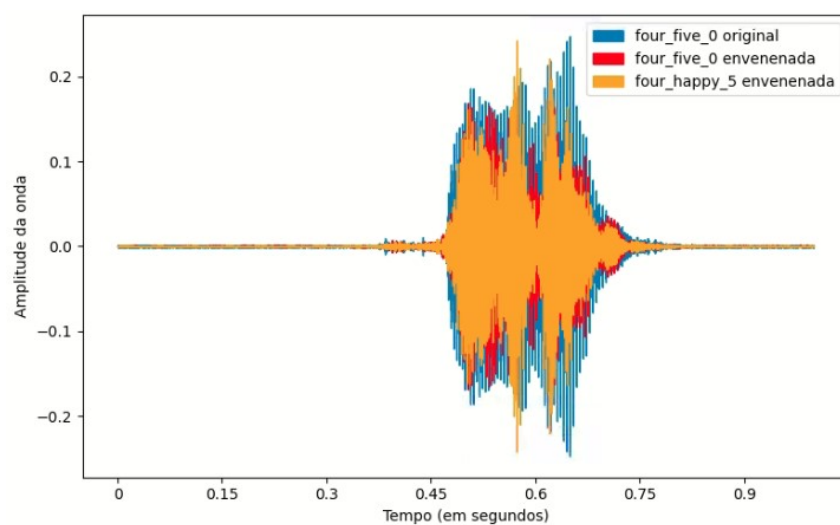
(b)



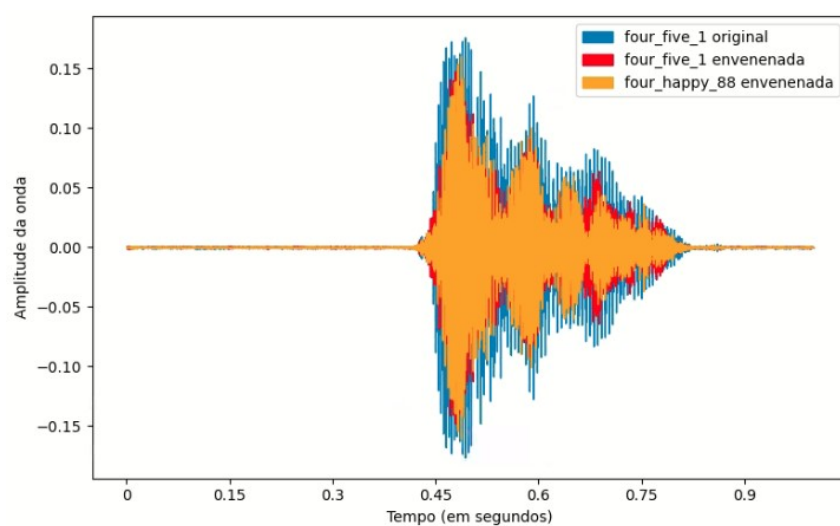
(c)

Fonte: Elaborado pelo autor.

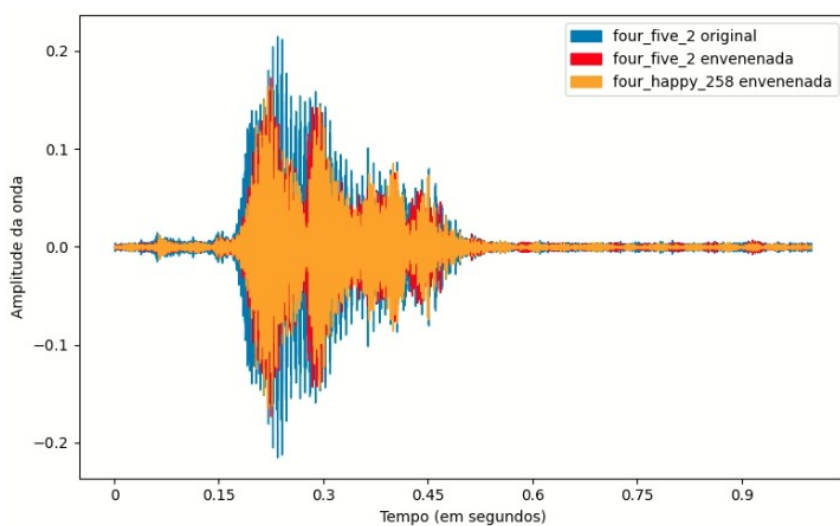
Figura 42 – Comparando onda do áudio “four” antes e depois do envenenamento com âncoras “five” e “happy”



(a)



(b)



(c)

Fonte: Elaborado pelo autor.

4.3 Funções de perda durante o *fine-tuning*

Para o contexto de envenenamento de dados (imagens ou áudios), o estudo das funções de perda é a primeira etapa de avaliação para identificar se o modelo treinado sofreu ou não envenenamento.

Esse estudo ocorre por meio de representações temporais dos valores de perda ao longo das épocas de treinamento. Caso elas apresentem um comportamento crescente e, ou, a ocorrência de picos acentuados, há, por conseguinte, uma sinalização de que o modelo está desaprendendo informações devido à aplicação de um envenenamento em sua base de treinamento. Entretanto, a própria evidência do sucesso do ataque — o crescimento na curva de perda — anula sua furtividade e o torna prontamente identificável.

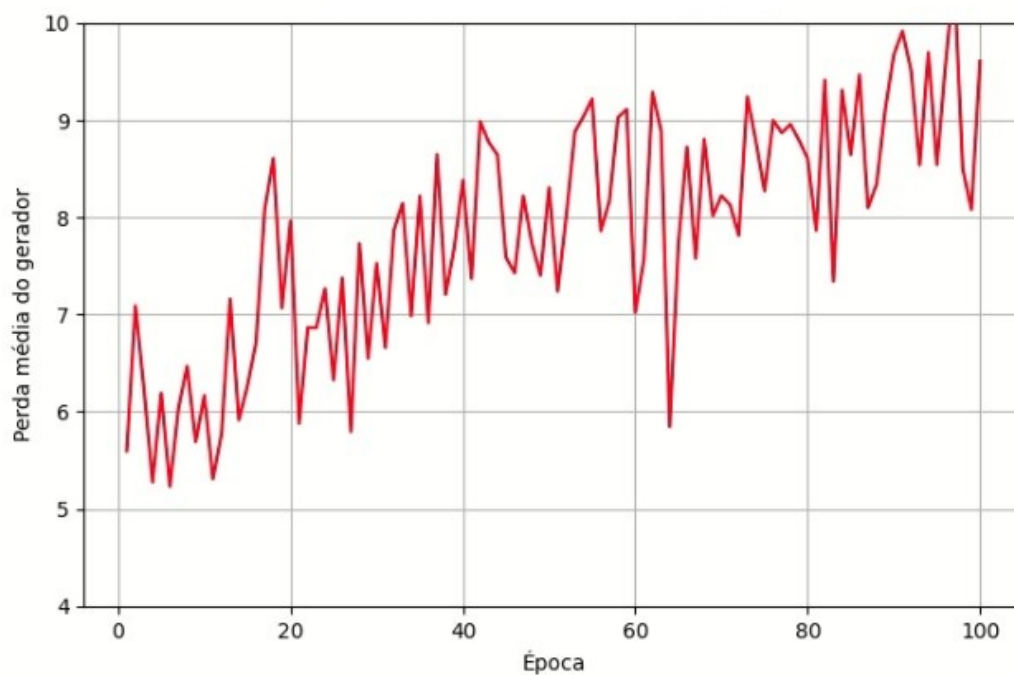
No modelo StyleTTS 2, o componente responsável pela do compreensão do conteúdo e geração dos áudios é o gerador, logo a identificação de um possível envenenamento do modelo ocorre por meio da avaliação humana sobre a representação visual da função de perda referente a este componente.

Neste trabalho, todos os ataques apresentaram, de forma quase igual, um comportamento crescente e com alta ocorrência de picos ao longo das funções de perda do *fine-tuning* definido para 100 épocas. Esse comportamento está ilustrado nas figuras 43 e 44, referentes aos ataques (*four*, *happy*) para as amostras envenenadas de tamanho $N_p = 100$ e $N_p = 300$.

Com base na análise das funções de perda do gerador, conclui-se que o método de envenenamento proposto é eficaz em induzir o desaprendizado no modelo StyleTTS 2, tanto para amostras envenenadas de tamanho $N_p = 100$ quanto $N_p = 300$. Contudo, essa mesma evidência que aponta para o sucesso do ataque serve como seu principal indicador, tornando-o facilmente identificável.

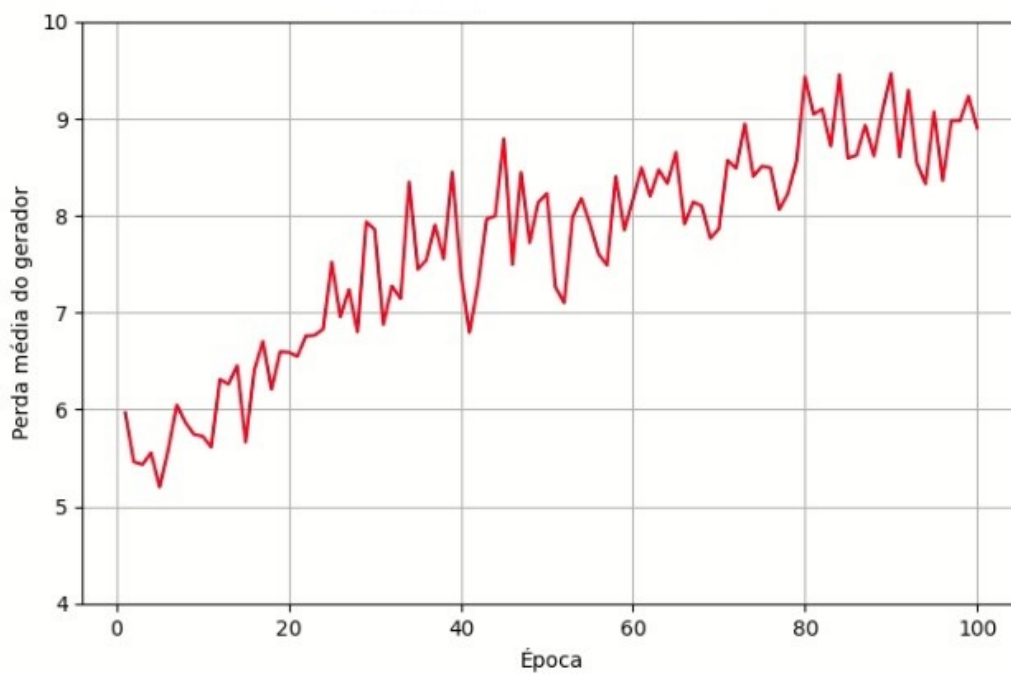
Por fim, a principal limitação desta etapa de avaliação é que, apesar de se confirmar o desaprendizado, não é possível determinar se os conceitos perdidos referem-se à qualidade intrínseca do áudio ou ao seu conteúdo semântico. Diante disso, é necessário avaliar as transcrições dos áudios envenenados, coletadas por meio do formulário (seção 4.4).

Figura 43 – Perda do gerador referente ao StyleTTS 2 envenado com $N_p = 100$ áudios



Fonte: Elaborado pelo autor.

Figura 44 – Perda do gerador referente ao StyleTTS 2 envenado com $N_p = 300$ áudios



Fonte: Elaborado pelo autor.

4.4 Avaliação do StyleTTS 2 envenenado

Nesta seção, avaliar-se-á os efeitos individuais de cada ataque de envenenamento possível sobre o modelo StyleTTS 2 por meio da pontuação CLAP, do MOS obtido a partir das respostas do questionário online e se houve troca das palavras *PE* pela *PA*. A comparação será feita entre o modelo pré-treinado do StyleTTS 2 (definido na subseção 3.3.2), o modelo StyleTTS 2 envenenado com amostra de áudios envenenados de tamanho $N_p = 100$ e o modelo StyleTTS 2 envenenado com amostra envenenada de tamanho $N_p = 300$.

Os ataques possíveis foram avaliados com base nas respostas dos 36 participantes do questionário. Para cada ataque, a avaliação humana está resumida em duas tabelas. A primeira contém as pontuações CLAP, diferença CLAP e MOS geral dos modelos StyleTTS 2 original (não sofreu envenenamento), StyleTTS 2 envenenado com uma amostra envenenada de tamanho $N = 100$ e o StyleTTS 2 envenenado com uma amostra de tamanho $N = 300$. Por sua vez, a segunda informa se as transcrições dos áudios gerados por cada um desses modelos está correta (A), errada (E) ou não fornecida (I, de incompreensível). Como há três modelos para serem avaliados, nove ataques possíveis por modelo e uma frase sortida ao acaso dentre as três frases que contém a palavra *PE*, logo cada participante escutará e avaliará 27 áudios ao todo.

Essas informações estão resumidas nas tabelas 6 a 23 (contempladas nas subseções de 4.4.1 a 4.4.9), em que as de número par se referem à pontuação CLAP e o valor MOS geral dos três modelos e as de número ímpar, a transcrição dos áudios gerados por cada modelo.

Pelas tabelas pares, é aferido que o valor MOS decresce drasticamente em todos os ataques realizados e a diferença CLAP aponta o ataque (*four*, *happy*) como o único bem-sucedido, porque ele é o único que apresentou decréscimo no valor da diferença CLAP para tamanhos de amostras de treinamento envenenadas maiores.

As tabelas ímpares complementam as inferências obtidas na análise das pares. Para as frases de avaliação que consistiam apenas da repetição da palavra a ser envenenada *PE* três vezes, é possível observar que $A > E + I$ para todos os modelos, indicando que não houve troca de *PE* por *PA*. Já, nas frases que não se repetiam palavras, observou-se que $A \leq E + I$ e $E < I$ ocorreu na maioria dos ataques, mostrando, assim, que a aplicação do envenenamento *Nightshade* adaptado no StyleTTS 2 resultou na extrema degradação do conteúdo presentes nos áudios desse modelo TTS, a ponto de ele ser capaz apenas de gerar áudios incompreensíveis e bastante ruidosos (aspecto este confirmado pelo decaimento abrupto do valor MOS).

De acordo com Kong, Kim e Bae (2020), a presença de pequenas inconsistências na magnitude do espectrograma, quando processadas pelo método iterativo

de reconstrução de fase do Griffin-Lim, degradam a qualidade sonora de um áudio a ponto de os ruídos introduzidos abafarem a própria fala. Pode-se argumentar, assim, que o método Griffin-Lim atua como a fonte principal da extrema degradação do áudio, contribuindo de forma mais significativa para o ruído final do que as próprias perturbações originalmente inseridas pelo próprio *Nightshade*.

Apesar de a quantidade de áudios incompreensíveis superar a quantidade de transcrições erradas, $E < I$, para as frases em que não se haviam repetições, é importante verificar se os erros correspondem à troca das palavras *PE* por *PA*. Dentre as transcrições erradas, apenas uma apresentou a troca esperada. Isso ocorreu na frase de avaliação “*I have nine dollars in my pocket*” para o ataque (*nine, five*), em que um dos participantes transcreveu o áudio para “*i have five five five*”. As demais transcrições erradas também registram trocas de palavras, todavia são trocas que não ocorrem sobre a palavra *PE* nem fazem as palavras trocadas se tornarem *PA*.

Por exemplo, na frase “*tree three tree*” do ataque (*tree, destroy*), predominou-se a transcrição “*tree tree tree*” nos StyleTTS 2 envenenados; na frase “*three four five*” dos ataques (*four, five*) e (*four, happy*), predominou-se a transcrição “*four four four*” após o envenenamento; na frase “*i’m feeling down*” do ataque (*down, right*), a palavra *feeling* se tornou uma palavra que começa com *d* (*don’t, dumb, down*) para o envenenamento com amostra de tamanho $N_p = 300$; na frase “*i have nine dollars in my pocket*”, a palavra “*dollars*” virou “*thousand*” nos modelos envenenados.

Outro ponto a se destacar é a presença de valores negativos nas diferenças CLAP do StyleTTS 2 original, os quais deveriam ser positivos antes do envenenamento do modelo. Diante dessa contradição, acrescida pela inconsistência na etapa de selecionar um número suficiente de candidatos para compor a amostra de treinamento envenenada (relatado na subseção 4.2.1.1), infere-se que o modelo CLAP adotado neste trabalho é ineficiente para mensuração da proximidade entre uma fala composta apenas por uma única palavra e um texto que representa a sua descrição - no caso deste trabalho, “*A person saying [word]*”.

Essa conclusão é plausível porque de acordo com Wu* et al. (2023), o CLAP é um modelo com baixo desempenho no campo dos áudios de falas humanas por dois motivos: (i) a maior parte dos dados de treinamento são áudios cujas legendas de texto são descrições de eventos sonoros, como “cachorro latindo”, “carro passando”, e (ii) não foi otimizado para considerar a ordem ou a estrutura temporal dos sons. Ao final de seu estudo, os autores apontam que a incorporação de um alinhamento temporal entre áudio e texto na arquitetura do CLAP possa melhorar seu desempenho na tarefa de medir a proximidade entre áudios de fala e textos.

4.4.1 Ataque (PE, PA) = (bed, cat)

As transcrições erradas registradas para esse ataque são:

- **Bed bed bed** : “dead dead dead”;
- **I’m going to the bed** : “bed bed bed bed bed”;
- **The cat is sleeping on the bed** : “the bed, sleeping on the bed”;

Tabela 6 – Pontuação CLAP e MOS para o ataque (bed, cat)

StyleTTS 2	Pontuação CLAP		Diferença CLAP	MOS
	<i>Bed</i>	<i>Cat</i>		
Original	0.069	0.198	- 0.130	4.65
Envenenado_100	0.185	0.086	+0.099	2.75
Envenenado_300	0.134	0.144	- 0.010	1.27

Fonte: Elaborada pelo autor.

Tabela 7 – Número de acertos, erros e de incompreensão acerca da transcrição dos áudios gerados pelo StyleTTS 2 após o envenenamento (bed, cat)

Transcrição do áudio	Original			Enve 100			Enve 300		
	A	E	I	A	E	I	A	E	I
<i>Bed bed bed</i>	14	0	1	13	0	2	9	1	5
<i>I’m going to the bed</i>	11	0	0	9	0	2	1	1	9
<i>The cat is sleeping on the bed</i>	10	0	0	8	1	1	0	0	10

Fonte: Elaborada pelo autor.

4.4.2 Ataque (PE, PA) = ($down, right$)

As transcrições erradas registradas para esse ataque são:

- **Down down down** : “gum gum gum”, “don’t don’t don’t”, “down down downed”, “dum dum dum”;
- **Go down the stairs** : “go down the stout”, “go down the stones”;
- **I am feeling down** : “i am feeling downed”, “i am feeling daunt”, “i am feeling doubt”, “bam coun count”, “bam coun count”.

Tabela 8 – Pontuação CLAP e MOS para o ataque (*down, right*)

StyleTTS 2	Pontuação CLAP		Diferença CLAP	MOS
	<i>Down</i>	<i>Right</i>		
Original	0.185	0.23	- 0.044	4.27
Envenenado_100	0.243	0.251	- 0.007	2.15
Envenenado_300	0.302	0.282	+0.020	1.62

Fonte: Elaborada pelo autor.

Tabela 9 – Número de acertos, erros e de incompreensão acerca da transcrição dos áudios gerados pelo StyleTTS 2 após o envenenamento (*down, right*)

Transcrição do áudio	Original			Enve 100			Enve 300		
	A	E	I	A	E	I	A	E	I
<i>Down down down</i>	12	0	0	10	2	0	7	2	3
<i>Go down the stairs</i>	11	1	0	2	2	8	0	4	8
<i>I'm feeling down</i>	10	0	0	5	3	2	0	6	4

Fonte: Elaborada pelo autor.

4.4.3 Ataque (*PE, PA*) = (*four, five*)

As transcrições erradas registradas para esse ataque são:

- **Three four five** : “*four four four*”;
- **I have four cards to trade** : “*i have four cars to trade*”, “*i have four chords to trade*”, “*i have four house to trade*”, “*i have four horts to trade*”, “*i have four fours to trade*”, “*check out four chords to print*”, “*four four, this train*”, “*i have four cards to train*”, “*i have four cards to train*”, “*four fours to trade*”;

Tabela 10 – Pontuação CLAP e MOS para o ataque (*four, five*)

StyleTTS 2	Pontuação CLAP		Diferença CLAP	MOS
	<i>Four</i>	<i>Five</i>		
Original	0.453	0.49	- 0.038	4.31
Envenenado_100	0.417	0.485	- 0.068	2.43
Envenenado_300	0.397	0.462	- 0.065	1.7

Fonte: Elaborada pelo autor.

Tabela 11 – Número de acertos, erros e de incompreensão acerca da transcrição dos áudios gerados pelo StyleTTS 2 após o envenenamento (*four, five*)

Transcrição do áudio	Original			Enve 100			Enve 300		
	A	E	I	A	E	I	A	E	I
<i>Three four five</i>	14	0	0	13	0	1	0	13	1
<i>I have four cards to trade</i>	15	0	0	5	9	1	2	4	9
<i>I scored a four in math class</i>	7	0	0	3	0	4	1	0	6

Fonte: Elaborada pelo autor.

4.4.4 Ataque (PE, PA) = (*four, happy*)

As transcrições erradas registradas para esse ataque são:

- **Three four five** : “three four four”, “four four four”, “through four ford”, “through four fourth”, “floor four ford”;
- **I have four cards to trade** : “i have four cars to trade”, “i have four chords to trade”, “i have four pour to trade”, “i have four horts to trade”, “i have four fours to trade”, “i have four fords to trade”, “i have four accords to trade”, “check out four chords to print”, “four four, this train”, “i have four cards to train”, “i have four cards to train”, “four fours to trade”, “i have four horse to threat”;
- **I scored a four in math class** : “four four, in the way”, “four four in my cloth”, “a four a four”, “my score are four in march watch”.

Tabela 12 – Pontuação CLAP e MOS para o ataque (*four, happy*)

StyleTTS 2	Pontuação CLAP		Diferença CLAP	MOS
	<i>Four</i>	<i>Happy</i>		
Original	0.453	0.049	+0.404	4.66
Envenenado_100	0.444	0.057	+0.388	2.48
Envenenado_300	0.346	0.03	+0.316	1.91

Fonte: Elaborada pelo autor.

Tabela 13 – Número de acertos, erros e de incompreensão acerca da transcrição dos áudios gerados pelo StyleTTS 2 após o envenenamento (*four, happy*)

Transcrição do áudio	Original			Enve 100			Enve 300		
	A	E	I	A	E	I	A	E	I
<i>Three four five</i>	11	0	0	0	8	3	0	9	2
<i>I have four cards to trade</i>	10	6	0	5	8	3	1	8	7
<i>I scored a four in math class</i>	9	0	0	1	3	5	0	2	7

Fonte: Elaborada pelo autor.

4.4.5 Ataque (PE, PA) = (*nine, five*)

As transcrições erradas registradas para esse ataque são:

- ***Eight nine ten*** : “*eight nine bend*”, “*nine nine nine*”, “*nine nine ten*”;
- ***Cats have nine lifes, right?*** : “*cat have nine knives, right?*”;
- ***I have nine dollars in my pocket*** : “*i have nine dar in my par*”, “*i have five five five*”.

Tabela 14 – Pontuação CLAP e MOS para o ataque (*nine, five*)

StyleTTS 2	Pontuação CLAP		Diferença CLAP	MOS
	<i>Nine</i>	<i>Five</i>		
Original	0.392	0.415	- 0.023	4.5
Envenenado_100	0.406	0.48	- 0.074	2.58
Envenenado_300	0.035	0.012	+0.023	1.43

Fonte: Elaborada pelo autor.

Tabela 15 – Número de acertos, erros e de incompreensão acerca da transcrição dos áudios gerados pelo StyleTTS 2 após o envenenamento (*nine, five*)

Transcrição do áudio	Original			Enve 100			Enve 300		
	A	E	I	A	E	I	A	E	I
<i>Eight nine ten</i>	11	0	0	4	2	5	3	2	6
<i>Cats have nine lifes, right?</i>	11	0	0	9	1	1	3	0	8
<i>I have nine dollars in my pocket</i>	14	0	0	13	0	1	5	2	7

Fonte: Elaborada pelo autor.

4.4.6 Ataque (PE, PA) = (*nine, tiny*)

As transcrições erradas registradas para esse ataque são:

- ***Cats have nine lifes, right?*** : “*nineth nine nineth knife right*”;
- ***I have nine dollars in my pocket*** : “*i have nine thousand in my pocket*”, “*i have nice thousand nine nine*”.

Tabela 16 – Pontuação CLAP para o ataque (*nine, tiny*)

StyleTTS 2	Pontuação CLAP		Diferença CLAP	MOS
	<i>Nine</i>	<i>Five</i>		
Original	0.392	0.061	+0.331	4.69
Envenenado_100	0.359	-0.004	+0.363	2.3
Envenenado_300	0.325	0.054	+0.271	1.54

Fonte: Elaborada pelo autor.

Tabela 17 – Número de acertos, erros e de incompreensão acerca da transcrição dos áudios gerados pelo StyleTTS 2 após o envenenamento (*nine, tiny*)

Transcrição do áudio	Original			Enve 100			Enve 300		
	A	E	I	A	E	I	A	E	I
<i>Eight nine ten</i>	13	0	0	7	0	6	6	0	7
<i>Cats have nine lifes, right?</i>	9	0	0	7	0	2	2	1	6
<i>I have nine dollars in my pocket</i>	14	0	0	10	1	3	5	3	6

Fonte: Elaborada pelo autor.

4.4.7 Ataque (PE, PA) = (*no, yes*)

As transcrições erradas registradas para esse ataque são:

- ***I said no*** : “*also no*”, “*i felt no*”, “*no no*”, “*oh no no*”, “*said no, no*”.

Tabela 18 – Pontuação CLAP e MOS para o ataque (*no, yes*)

StyleTTS 2	Pontuação CLAP		Diferença CLAP	MOS
	<i>No</i>	<i>Yes</i>		
Original	0.396	0.407	- 0.011	4.67
Envenenado_100	0.426	0.441	- 0.015	2.88
Envenenado_300	0.44	0.457	- 0.017	2.33

Fonte: Elaborada pelo autor.

Tabela 19 – Número de acertos, erros e de incompreensão acerca da transcrição dos áudios gerados pelo StyleTTS 2 após o envenenamento (*no, yes*)

Transcrição do áudio	Original			Enve 100			Enve 300		
	A	E	I	A	E	I	A	E	I
<i>No no no</i>	15	0	0	15	0	0	15	0	0
<i>I said no</i>	8	0	0	1	4	3	0	7	1

Fonte: Elaborada pelo autor.

4.4.8 Ataque (PE, PA) = (*stop, go*)

As transcrições erradas registradas para esse ataque são:

- **Can you stop this?** : “tell don’t stop us”, “no don’t stop don’t”, “stop dance stop dance”;
- **You can’t stop now** : “do don’t stop down”, “so don’t stop don’t”.

Tabela 20 – Pontuação CLAP e MOS para o ataque (*stop, go*)

StyleTTS 2	Pontuação CLAP		Diferença CLAP	MOS
	Stop	Go		
Original	0.222	0.112	+0.110	4.33
Envenenado_100	0.339	0.218	+0.121	2.10
Envenenado_300	0.269	0.165	+0.104	1.82

Fonte: Elaborada pelo autor.

Tabela 21 – Número de acertos, erros e de incompreensão acerca da transcrição dos áudios gerados pelo StyleTTS 2 após o envenenamento (*stop, go*)

Transcrição do áudio	Original			Enve 100			Enve 300		
	A	E	I	A	E	I	A	E	I
<i>Stop stop stop</i>	16	0	0	16	0	0	16	0	0
<i>Can you stop this?</i>	10	0	0	2	1	7	0	2	8
<i>You can’t stop now</i>	10	0	0	4	1	5	0	2	8

Fonte: Elaborada pelo autor.

4.4.9 Ataque (PE, PA) = (*tree, destroy*)

As transcrições erradas registradas para esse ataque são:

- **Tree three ree** : “tree tree tree”, “three three tree”, “three three three”, “tree three trick”;
- **Money does not grow on trees** : “my need is not growing trees”, “many this not grow on trees”, “my need is not to pray on church”, “money does not grow on treench”, “money does not throw in tree”, “money does not nate grow intriged”, “honey, there is not true, on tree”, “honey does not drug entree”, “free dows not throw in tree”.

Tabela 22 – Pontuação CLAP e MOS para o ataque (*tree*, *destroy*)

StyleTTS 2	Pontuação CLAP		Diferença CLAP	MOS
	<i>Tree</i>	<i>Destroy</i>		
Original	0.101	0.213	- 0.112	4.47
Envenenado_100	0.092	0.229	- 0.138	2.85
Envenenado_300	0.097	0.277	- 0.18	2.17

Fonte: Elaborada pelo autor.

Tabela 23 – Número de acertos, erros e de incompreensão acerca da transcrição dos áudios gerados pelo StyleTTS 2 após o envenenamento (*tree*, *destroy*)

Transcrição do áudio	Original			Enve 100			Enve 300		
	A	E	I	A	E	I	A	E	I
<i>Tree three tree</i>	9	6	0	2	13	0	0	15	0
<i>She is sleeping under that tree</i>	8	0	0	8	0	0	4	0	4
<i>Money does not grow on trees</i>	10	3	0	3	3	7	2	4	7

Fonte: Elaborada pelo autor.

4.5 Considerações Finais

Os principais resultados deste trabalho indicam que a adaptação do Nightshade para áudio falhou em seus objetivos, além de expor limitações nas ferramentas utilizadas:

- **Falha no Ataque Direcionado:** O envenenamento provou ser não direcionado e destrutivo. Em vez de trocar palavras-alvo, ele degradou severamente a qualidade geral do StyleTTS 2, o que foi confirmado pelo rápido decréscimo dos valores MOS e por transcrições de áudios incompreensíveis.
- **Falha na Furtividade:** O ataque é facilmente identificável. O *fine-tuning* com áudios envenenados fez a função de perda do gerador crescer (retomar seção 4.3), indicando um colapso do modelo perceptível por simples avaliação humana.
- **Custo Computacional:** A adaptação é computacionalmente custosa, exigindo 10 horas para envenenar 5 minutos de áudio ($N_p = 300$) em uma GPU GeForce RTX 3060 LTS, com estimativa de 5 dias para 1 hora de áudio.
- **Limitação do CLAP:** O modelo CLAP Wu* et al. (2023) mostrou-se ineficiente para fala humana. Isso foi evidenciado pela falha em encontrar candidatos em 11 de 35 categorias do Command Speeches (retomar subseção 4.2.1.1) e pelos valores negativos da “diferença CLAP” apresentados no modelo StyleTTS 2 sem envenenamento (retomar subseções 4.4.1 a 4.4.9).

- **Limitação do StyleTTS 2:** O StyleTTS 2 Li et al. (2023) confirmou sua inconsistência com áudios curtos (menos de 3 segundos de duração). O preditor de duração falhou ao gerar matrizes incompatíveis, forçando o descarte de 8 ataques (retomar subseção 4.2.1.2).
- **Limitação do Griffin-Lim:** As observações de Kong, Kim e Bae (2020) sugerem que o Griffin-Lim foi o agente causador da alta distorção ao amplificar o efeito das perturbações adversariais (previamente inseridas pelo ataque no espectrograma) durante a síntese da forma de onda.

5 Conclusões

O avanço das IAGens impulsionou o desenvolvimento de *deepfakes* de áudio e a clonagem de voz não autorizada, problemas frequentemente alimentados por *web scraping*. Este trabalho propôs uma solução inédita para esse problema: a adaptação do algoritmo de envenenamento *Nightshade*, estado da arte para imagens, ao domínio do áudio.

O objetivo era tratar o áudio como um problema de imagem, visando um ataque direcionado e imperceptível contra o modelo TTS StyleTTS 2. Para isso, a metodologia converteu áudios de palavras únicas de 1 segundo (do conjunto *Speech Commands*) em espectrogramas de log-mel. Essas representações foram acopladas a imagens de 512×512 (formato do *Nightshade*), e os componentes do ataque foram adaptados: o CLIP foi substituído pelo CLAP (para seleção de candidatos) e o próprio StyleTTS 2 foi usado para gerar os áudios âncora correspondentes. Por fim, o algoritmo Griffin-Lim foi adotado para reverter o espectrograma envenenado à forma de onda.

Contudo, a adaptação falhou em seu objetivo principal. Em vez de um ataque direcionado e furtivo, o envenenamento degradou severamente a qualidade do modelo, resultando em áudios incompreensíveis e facilmente detectáveis, como confirmado pela análise da função de perda (com comportamento crescente) e pela queda drástica no MOS. As causas dessa falha residem diretamente nas limitações das ferramentas escolhidas: o CLAP mostrou-se inadequado para selecionar candidatos de fala (falhando em 11 de 35 classes); o StyleTTS 2 apresentou inconsistências com áudios curtos (inviabilizando oito ataques); e, crucialmente, o Griffin-Lim intensificou as perturbações adversariais durante a reconstrução, ampliando o ruído e destruindo a qualidade sonora.

Em resumo, a adaptação proposta do *Nightshade* para áudios falhou em atingir seus objetivos. O método mostrou-se destrutivo, detectável e não direcionado, isto é, o oposto dos resultados obtidos pelo *Nightshade* original. Conclui-se que o alto custo computacional, combinado com as inadequações das ferramentas escolhidas (CLAP, StyleTTS 2 e Griffin-Lim), foram os fatores determinantes que impediram o êxito da abordagem.

5.1 Trabalhos Futuros

Considerando a continuidade desse trabalho, esta seção tem por objetivo pontuar os aspectos técnicos para serem considerados na metodologia do *Nightshade*

adaptado e as questões éticas levantadas quanto ao seu uso.

Quanto à parte técnica, são necessários considerar os seguintes aspectos:

- Retreinar o modelo CLAP utilizando datasets focados em fala humana e suas transcrições, ou buscar na literatura alternativas que melhorem a comparação entre áudio de fala e texto;
- Seguindo as observações feitas por (KONG; KIM; BAE, 2020), trocar o algoritmo do Griffin-Lim por *vocoders*, isto é, redes neurais intensivamente treinadas para realizar uma conversão precisa do espectrograma de log-mel para áudio;
- Explorar outros modelos TTS de difusão de código aberto para a geração dos áudios âncora, priorizando modelos com melhor desempenho em áudios curtos;
- Testar o envenenamento em amostras de treinamento compostas por frases completas, aplicando o ataque a palavras específicas dentro de um contexto, em vez de palavras isoladas.

Quanto à parte ética, que não foram o escopo deste trabalho, são necessários considerar os seguintes aspectos:

- Investigar os fundamentos legais e éticos da “autodefesa de dados”, definindo quem (por exemplo, o criador original, o público, pequenas ou grandes empresas, etc.) tem o direito de aplicar o envenenamento e em quais circunstâncias isso é considerado uma proteção legítima.
- Analisar o impacto social e de segurança de um ataque bem-sucedido, levantando casos reais em que o envenenamento foi utilizado de forma defensiva e ofensiva em áreas como *deepfake audio* e biometria de voz.
- Discutir políticas de governança para a pesquisa em segurança de IA. Este estudo avaliaria os riscos da publicação irrestrita (por exemplo, *open-source*) de métodos de ataque e, se possível, propor diretrizes de “publicação responsável” para o áudio.

Referências

- AGHAKHANI, H.; SCHÖNHERR, L.; EISENHOFER, T.; KOLOSSA, D.; HOLZ, T.; KRUEGEL, C.; VIGNA, G. *VenoMave: Targeted Poisoning Against Speech Recognition*. 2023. Disponível em: <https://arxiv.org/abs/2010.10682>. Acesso em: 20 out. 2025.
- ALLEN, J. B.; RABINER, L. R. A unified approach to short-time fourier analysis and synthesis. *Proceedings of the IEEE*, IEEE, New York, v. 65, n. 11, p. 1558–1564, 1977.
- ALMUJALLY, N. A.; KHAN, D.; MUDAWI, A.; ALONAZI, A.; ALAZEB, A.; ALGARNI, A.; LIU, H. Biosensor-driven iot wearables for accurate body motion tracking and localization. *Sensors*, v. 24, n. 10, p. 3032, 2024. Disponível em: <https://www.mdpi.com/1424-8220/24/10/3032>. Acesso em: 20 out. 2025.
- BAKHSHI, A.; SINGH, S.; KUMAR, A. Violence detection in real-life audio signals using lightweight deep neural networks. *Procedia Computer Science*, v. 187, p. 122–129, 2023. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1877050923009274>. Acesso em: 20 out. 2025.
- BHUVANAGIRI, K. K.; KOPPARAPU, S. K. *Modified Mel Filter Bank to Compute MFCC of Subsampled Speech*. 2014. Disponível em: <https://arxiv.org/abs/1410.7382>. Acesso em: 20 out. 2025.
- CHEN, G.; FENG, S.-w.; KIM, W.-h.; QU, S.; LIU, T.; YIN, X.-c. BadTTS: a backdoor attack against text-to-speech models. In: *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Toronto: IEEE, 2021. p. 2660–2664.
- CHEN, H.; MAGRAMO, K. *Finance worker pays out \$25 million after video call with deepfake ‘chief financial officer’*. 2024. CNN World. Asia. Disponível em: <https://edition.cnn.com/2024/02/04/asia/deepfake-cfo-scam-hong-kong-intl-hnk/index.html>. Acesso em: 19 out. 2025.
- CIAPESSONI, G. G. *Mel-frequency filterbank*. 2017. Documentação da biblioteca pyfilterbank. Disponível em: <https://siggigue.github.io/pyfilterbank/melbank.html>. Acesso em: 3 out. 2025.
- COLLINS, K. *Game Sound: An introduction to the history, theory, and practice of video game music and sound design*. Cambridge: The MIT Press, 2008. ISBN 978-0262270694.
- Damien's Blog. *Blog 6 - The properties of Digital Images VS those of Digital Audio*. 2012. Disponível em: <https://damienoleary.wordpress.com/2012/11/18/blog-6-the-properties-of-digital-images-vs-those-of-digital-audio/>. Acesso em: 30 set. 2025.
- Digital Audio Wiki. *Bit Depth*. 2014. Disponível em: https://digital-audio.fandom.com/wiki/Bit_Depth. Acesso em: 30 set. 2025.
- Digital Sound & Music. *Mathematics and Algorithms for Aliasing*. s.d. Disponível em: <https://digitalsoundandmusic.com/5-3-4-mathematics-and-algorithms-for-aliasing/>. Acesso em: 30 set. 2025.

- DONAHUE, C.; MCAULEY, J.; PUCKETTE, M. *Adversarial Audio Synthesis*. 2019. Disponível em: <https://arxiv.org/abs/1802.04208>. Acesso em: 20 out. 2025.
- EL-KHASAWNEH, M.; AL-ADWAN, M. A. S.; SMADI, M. A.; AL-ZOUBI, R. An overview of quantum computing and its impact on communication systems. *Journal of Combinatorial Optimization*, Springer US, v. 49, 2025.
- GANGWAL, A.; ANSARI, A.; AHMAD, I.; AZAD, A. K.; KUMARASAMY, V.; SUBRAMANIYAN, V.; WONG, L. S. Generative artificial intelligence in drug discovery: basic framework, recent advances, challenges, and opportunities. *Frontiers in Pharmacology*, v. 15, p. 1331062, 2024. ISSN 1663-9812.
- GAYED, J. M.; CARLON, M. K. J.; ORIOLA, A. M.; CROSS, J. S. Exploring an ai-based writing assistant's impact on english language learners. *Computers and Education: Artificial Intelligence*, v. 3, p. 100055, 2022. ISSN 2666-920X.
- GE, Y.; WANG, Q.; ZHANG, J.; ZHOU, J.; ZHANG, Y.; SHEN, C. *WaveFuzz: A Clean-Label Poisoning Attack to Protect Your Voice*. 2022. Disponível em: <https://arxiv.org/abs/2203.13497>. Acesso em: 20 out. 2025.
- GRIFFIN, D. W.; LIM, J. S. Signal estimation from modified short-time fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, v. 32, n. 2, p. 236–243, 1984.
- GUO, Z.; LANG, J.; HUANG, S.; GAO, Y.; DING, X. *A Comprehensive Review on Noise Control of Diffusion Model*. 2025. Disponível em: <https://arxiv.org/abs/2502.04669>. Acesso em: 20 out. 2025.
- HASEGAWA-JOHNSON, M. *Lecture 6: Griffin-Lim Algorithm*. 2023. Notas de aula. Material da disciplina ECE 417: Multimedia Signal Processing, Outono de 2023. Disponível em: <https://courses.grainger.illinois.edu/ece417/fa2023/lectures.html>. Acesso em: 20 out. 2025.
- HO, J.; JAIN, A.; ABBEEL, P. *Denoising Diffusion Probabilistic Models*. 2020. Disponível em: <https://arxiv.org/abs/2006.11239>. Acesso em: 20 out. 2025.
- HUANG, C.-W.; LIM, J. H.; COURVILLE, A. *A Variational Perspective on Diffusion-Based Generative Models and Score Matching*. 2021. Disponível em: <https://arxiv.org/abs/2106.02808>. Acesso em: 20 out. 2025.
- HUBER, D. M. *Modern Recording Techniques*. 9. ed. New York: Routledge, 2017. ISBN 978-1138954373.
- ITO, K.; JOHNSON, L. *The LJ Speech Dataset*. 2017. Disponível em: <https://keithito.com/LJ-Speech-Dataset/>. Acesso em: 20 out. 2025.
- KARRAS, T.; AITTALA, M.; AILA, T.; LAINE, S. *Elucidating the Design Space of Diffusion-Based Generative Models*. 2022. Disponível em: <https://arxiv.org/abs/2206.00364>. Acesso em: 20 out. 2025.
- KIM, J.; KONG, J.; SON, J. *Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech*. 2021. Disponível em: <https://arxiv.org/abs/2106.06103>. Acesso em: 20 out. 2025.

- KONG, J.; KIM, J.; BAE, J. *HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis*. 2020. Disponível em: <https://arxiv.org/abs/2010.05646>. Acesso em: 16 set. 2025.
- KREUK, F.; SYNNAEVE, G.; POLYAK, A.; SINGER, U.; DÉFOSSEZ, A.; COPET, J.; PARIKH, D.; TAIGMAN, Y.; ADI, Y. *AudioGen: Textually Guided Audio Generation*. 2023. Disponível em: <https://arxiv.org/abs/2209.15352>. Acesso em: 16 set. 2025.
- LACERDA, T.; MIRANDA, P.; CÂMARA, A.; FURTADO, A. Deep learning and mel-spectrograms for physical violence detection in audio. In: *Anais do XVIII Encontro Nacional de Inteligência Artificial e Computacional*. Porto Alegre, RS, Brasil: SBC, 2021. p. 268–279. ISSN 2763-9061. Disponível em: <https://sol.sbc.org.br/index.php/eniac/article/view/18259>. Acesso em: 30 set. 2025.
- LATHI, B. P.; DING, Z. *Modern Digital and Analog Communication Systems*. 5. ed. New York: Oxford University Press, 2018. ISBN 978-0190686840.
- LI, Y. A.; HAN, C.; MESGARANI, N. *StyleTTS: A Style-Based Generative Model for Natural and Diverse Text-to-Speech Synthesis*. 2023. Disponível em: <https://arxiv.org/abs/2205.15439>. Acesso em: 20 out. 2025.
- LI, Y. A.; HAN, C.; RAGHAVAN, V. S.; MISCHLER, G.; MESGARANI, N. *StyleTTS 2: Towards Human-Level Text-to-Speech through Style Diffusion and Adversarial Training with Large Speech Language Models*. 2023. Disponível em: <https://arxiv.org/abs/2306.07691>. Acesso em: 20 out. 2025.
- LU, C.; ZHOU, Y.; BAO, F.; CHEN, J.; LI, C.; ZHU, J. *DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps*. 2022. Disponível em: <https://arxiv.org/abs/2206.00927>. Acesso em: 20 out. 2025.
- MACHADO, S. *'Eram meu rosto e minha voz, mas era golpe': como criminosos 'clonam pessoas' com inteligência artificial*. 2024. BBC NEWS Brasil. Disponível em: <https://www.bbc.com/portuguese/articles/cd1jv45dq3go>. Acesso em: 19 out. 2025.
- MASUYAMA, Y.; UENO, N.; ONO, N. *Signal Reconstruction from Mel-spectrogram Based on Bi-level Consistency of Full-band Magnitude and Phase*. 2023. Disponível em: <https://arxiv.org/abs/2307.12232>. Acesso em: 20 out. 2025.
- Math Works. *Short-time Fourier transform*. 2025. Disponível em: <https://www.mathworks.com/help/dsp/ref/dsp.stft.html>. Acesso em: 2 out. 2025.
- NGUYEN, T. T.; NGUYEN, Q. V. H.; NGUYEN, D. T.; NGUYEN, D. T.; HUYNH-THE, T.; NAHAVANDI, S.; NGUYEN, T. T.; PHAM, Q.-V.; NGUYEN, C. M. Deep learning for deepfakes creation and detection: A survey. *Computer Vision and Image Understanding*, Elsevier BV, v. 223, p. 103525, out. 2022. ISSN 1077-3142.
- OPPENHEIM, A. V.; SCHAFER, R. W. Processamento em tempo discreto de sinais. In: _____. 3. ed. São Paulo: Pearson Education do Brasil, 2012. cap. 9, p. 694–811. ISBN 978-8581431024.
- PHAM, L.; LAM, P.; NGUYEN, T.; NGUYEN, H.; SCHINDLER, A. *Deepfake Audio Detection Using Spectrogram-based Feature and Ensemble of Deep Learning Models*. 2024. ArXiv preprint. Disponível em: <https://arxiv.org/abs/2407.01777>. Acesso em: 16 set. 2025.

POHLMANN, K. C. *Principles of Digital Audio*. 6. ed. New York: McGraw-Hill, 2010. ISBN 978-0071663472.

QUEIROZ, G. 'Deepfake' de voz e desinformação em áudio se espalham pelo WhatsApp e desafiam checagem nas eleições. 2024. DW Brasil. Disponível em: <https://www.dw.com/pt-br/uso-de-ia-pode-ser-uma-amea%C3%A7a-no-brasil-%C3%A0s-v%C3%A9speras-da-elei%C3%A7%C3%A3o/a-69891136>. Acesso em: 19 out. 2025.

RADFORD, A.; KIM, J. W.; HALLACY, C.; RAMESH, A.; GOH, G.; AGARWAL, S.; SASTRY, G.; ASKELL, A.; MISHKIN, P.; CLARK, J.; KRUEGER, G.; SUTSKEVER, I. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. Disponível em: <https://arxiv.org/abs/2103.00020>. Acesso em: 20 out. 2025.

REISS, T.; CAVIA, B.; HOSHEN, Y. *Detecting Deepfakes Without Seeing Any*. 2023. Disponível em: <https://arxiv.org/abs/2311.01458>. Acesso em: 20 out. 2025.

ROMBACH, R.; BLATTMANN, A.; LORENZ, D.; ESSER, P.; OMMER, B. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. Disponível em: <https://arxiv.org/abs/2112.10752>. Acesso em: 20 out. 2025.

SANTOS, E. *O que os nudes falsos em Itararé ensinam sobre o aumento de casos de deepfakes pornográficos em escolas*. 2024. G1. Disponível em: <https://g1.globo.com/educacao/noticia/2024/10/11/o-que-os-nudes-falsos-em-itarare-ensinam-sobre-o-aumento-de-casos-de-deepfakes-pornograficos-em-escolas.ghtml>. Acesso em: 19 out. 2025.

SCHUHMANN, C.; BEAUMONT, R.; VENCU, R.; GORDON, C.; WIGHTMAN, R.; CHERTI, M.; COOMBES, T.; KATTA, A.; MULLIS, C.; WORTSMAN, M.; SCHRAMOWSKI, P.; KUNDURTHY, S.; CROWSON, K.; SCHMIDT, L.; KACZMARCZYK, R.; JITSEV, J. *LAION-5B: An open large-scale dataset for training next generation image-text models*. 2022. Disponível em: <https://arxiv.org/abs/2210.08402>. Acesso em: 1 set. 2025.

SHAN, S.; DING, W.; PASSANANTI, J.; WU, S.; ZHENG, H.; ZHAO, B. Y. *Nightshade: Prompt-Specific Poisoning Attacks on Text-to-Image Generative Models*. 2024. Disponível em: <https://arxiv.org/abs/2310.13828>. Acesso em: 1 set. 2025.

SHEN, K.; JU, Z.; TAN, X.; LIU, Y.; LENG, Y.; HE, L.; QIN, T.; ZHAO, S.; BIAN, J. *NaturalSpeech 2: Latent Diffusion Models are Natural and Zero-Shot Speech and Singing Synthesizers*. 2023. Disponível em: <https://arxiv.org/abs/2304.09116>. Acesso em: 20 out. 2025.

SKLAR, B. *Digital Communications: Fundamentals and Applications*. 2. ed. [S.l.]: Prentice Hall, 2001. ISBN 978-0130847881.

SOHL-DICKSTEIN, J.; WEISS, E. A.; MAHESWARANATHAN, N.; GANGULI, S. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. 2015. Disponível em: <https://arxiv.org/abs/1503.03585>. Acesso em: 20 out. 2025.

SONG, Y.; SOHL-DICKSTEIN, J.; KINGMA, D. P.; KUMAR, A.; ERMON, S.; POOLE, B. *Score-Based Generative Modeling through Stochastic Differential Equations*. 2021. Disponível em: <https://arxiv.org/abs/2011.13456>. Acesso em: 20 out. 2025.

STEVENS, S. S.; VOLKMANN, J.; NEWMAN, E. B. A scale for the measurement of the psychological magnitude of pitch. *The Journal of the Acoustical Society of America*, v. 8, n. 3, p. 185–190, 1937.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, L.; POLOSUKHIN, I. *Attention Is All You Need*. 2017. Disponível em: <https://arxiv.org/abs/1706.03762>. Acesso em: 20 out. 2025.

WANG, C.; CHEN, S.; WU, Y.; ZHANG, Z.; ZHOU, L.; LIU, S.; CHEN, Z.; LIU, Y.; WANG, H.; LI, J.; HE, L.; ZHAO, S.; WEI, F. *Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers*. 2023. Disponível em: <https://arxiv.org/abs/2301.02111>. Acesso em: 20 out. 2025.

WARDEN, P. *Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition*. 2018. Disponível em: <https://arxiv.org/abs/1804.03209>. Acesso em: 20 out. 2025.

WATKINSON, J. *The Art of Digital Audio*. 3. ed. Oxford: Focal Press, 2001. ISBN 978-0240515878.

WONG, E.; HAJEK, B. *Stochastic Processes in Engineering Systems*. New York: Springer Verlag, 1985. ISBN 978-1-4612-5060-9.

WU*, Y.; CHEN*, K.; ZHANG*, T.; HUI*, Y.; BERG-KIRKPATRICK, T.; DUBNOV, S. *Large-scale Contrastive Language-Audio Pretraining with Feature Fusion and Keyword-to-Caption Augmentation*. 2023. Disponível em: <https://arxiv.org/abs/2211.06687>. Acesso em: 31 aug. 2025.

YAMAGISHI, J.; VEAUX, C.; MACDONALD, K. Cstr vctk corpus: English multi-speaker corpus for cstr voice cloning toolkit (version 0.92). In: . [S.l.: s.n.], 2019. Disponível em: <https://api.semanticscholar.org/CorpusID:213060286>. Acesso em: 20 out. 2025.

ZEN, H.; DANG, V.; CLARK, R.; ZHANG, Y.; WEISS, R. J.; JIA, Y.; CHEN, Z.; WU, Y. *LibriTTS: A Corpus Derived from LibriSpeech for Text-to-Speech*. 2019. Disponível em: <https://arxiv.org/abs/1904.02882>. Acesso em: 19 out. 2025.

ZHANG, C.; ZHANG, C.; ZHENG, S.; ZHANG, M.; QAMAR, M.; BAE, S.-H.; KWEON, I. S. *A Survey on Audio Diffusion Models: Text To Speech Synthesis and Enhancement in Generative AI*. 2023. Disponível em: <https://arxiv.org/abs/2303.13336>. Acesso em: 30 set. 2025.

ZHANG, J.; JAYASURIYA, S.; BERISHA, V. Restoring degraded speech via a modified diffusion model. In: *Interspeech 2021*. [S.l.]: ISCA, 2021. p. 221–225. Disponível em: <http://dx.doi.org/10.21437/Interspeech.2021-1889>. Acesso em: 17 set. 2025.

ZHANG, R.; ISOLA, P.; EFROS, A. A.; SHECHTMAN, E.; WANG, O. The unreasonable effectiveness of deep features as a perceptual metric. 2018. Disponível em: <http://arxiv.org/abs/1801.03924>. Acesso em: 12 set. 2025.

ZHANG, Y.; IKEMIYA, Y.; XIA, G.; MURATA, N.; MARTÍNEZ-RAMÍREZ, M. A.; LIAO, W.-H.; MITSUFUJI, Y.; DIXON, S. *MusicMagus: Zero-Shot Text-to-Music Editing via Diffusion Models*. 2024. Disponível em: <https://arxiv.org/abs/2402.06178>. Acesso em: 16 set. 2025.

ZHANG, Z.; LI, L.; CONG, G.; YIN, H.; GAO, Y.; YAN, C.; HENGEL, A. v. d.; QI, Y. From speaker to dubber: Movie dubbing with prosody and duration consistency learning. In: *Proceedings of the 32nd ACM International Conference on Multimedia*. New York, NY, USA: Association for Computing Machinery, 2024. p. 7523–7532. ISBN 9798400706868.